

OZNAL Capstone - LoL Esports Early-Game Prediction

Martin Vančo, Adriana Gemelová

2026-05-08

Introduction

League of Legends is the most-played and most-watched esport in the world. Predicting which team will win from the early state of the match is a long-standing question in esports analytics: by 15 minutes - well before half of a typical game has elapsed - gold, experience, and objective leads have already begun to compound. We use the 2025 season of professional play to test how reliably this 15-minute snapshot forecasts the eventual winner, and which features actually drive the prediction.

The analysis runs two scenarios on the same data:

- **Scenario 1 - Model Comparison.** We train and tune five baseline classifiers (Logistic Regression, Random Forest, Naive Bayes, KNN, CART) on the early-game feature set, plus XGBoost as an additional model to mirror the gradient-boosted approach used in the published @15 literature. All six are compared on accuracy, AUC-ROC, precision, recall, and F1. The goal is to see whether any model can extract signal beyond the trivial “team with the gold lead at 15 min wins” baseline (~73% accuracy), and whether different model families agree on what matters.
- **Scenario 3 - Feature Selection.** With 14 candidate features carrying residual redundancy, we apply forward stepwise selection (AIC), LASSO, and Elastic Net to find the minimal subset that retains predictive performance. This both improves interpretability and stress-tests which features are genuinely informative versus carrying duplicate signal.

Hypotheses

H1: *“Early-game performance metrics and objective control can reliably predict match outcome in professional League of Legends.”* We test whether gold, XP, and CS differentials at 10 and

15 minutes, combined with first objective captures (first blood, first dragon, first tower, void grubs), contain enough signal to classify wins and losses.

H2: *“Bot lane (ADC) gold advantage is the most impactful role-specific early-game factor for predicting match outcome.”* We test whether the bot lane player’s early-game differentials carry stronger predictive signal than those of other roles (top, jungle, mid, support).

Motivation

Both of us on this project are long-time League of Legends players and regular viewers of the professional scene. Watching pro matches, you very quickly develop a feel for when a game is “already over” - a significant gold lead at early game mostly means a quick and decisive game. We had been wondering for a while how much of that intuition is actually backed by data: how reliably do those early signals predict the eventual winner and which signals matter the most.

Dataset

We use the 2025 season export from **Oracle’s Elixir** [1], the standard public source for professional League of Legends match data. The raw file contains **120,636 rows across 165 columns**, covering **10,053 unique matches** played by 445 teams across 45 professional leagues (LPL, LCK, LEC, LFL, LAS, EM, ...) on patches 15.01-15.24.

The row layout is **12 rows per game**: one per player (5 per team) plus one aggregated team-level row per side (Blue and Red, 50/50). About **91.9% of rows are marked complete**, meaning the early-game snapshots at 10/15/20/25 min are populated; the remaining 8.1% are missing these values.

Columns group into metadata (gameid, league, patch, side, position, ...), draft (pick1-5, ban1-5), outcome (result, gamelength), combat (kills/deaths/assists, first blood), objectives (dragons, heralds, void grubs, towers, turret plates), economy (gold, CS), vision (wards, vision score), and the **early-game timeline snapshots** (goldatX, xpatX, csatX, golddiffatX, etc. for X in {10, 15, 20, 25}) which form the core of our feature set.

Game introduction

League of Legends (LoL) is a 5v5 team game. Two teams (Blue and Red side) try to destroy each other’s base. A single match is typically around 30 minutes but can last as long as 50 minutes. In this work, we focus on the **early game** stage (the first 15 minutes). Each team has five fixed roles - **top**, **jungle (jng)**, **mid**, **bot (ADC)**, and **support (sup)** - each playing a different lane or area of the map.

Three continuous resources accumulate throughout the match:

- **Gold** - earned from minions, champion kills, and objectives, and spent on items that strengthen champions.
- **XP (experience)** - levels champions up and unlocks abilities.
- **CS (creep score)** - the number of minions a player has killed

The early game also has a set of one-off objective races, each won by exactly one team: **first blood** (the first champion kill of the match), **first dragon**, **first herald**, **first tower** (each granting a stacking buff, a tower-damaging summon, or a large gold bonus), **turret plates** (five plates on each outer tower, all paid out as gold before 14:00), and **void grubs** (a shared pit of up to 6 grubs that despawns at 13:45 on early-2025 patches and 14:45 mid-season).

Related work

Published work on early-game LoL match prediction at the **15-minute mark** converges tightly, with reported accuracy clustering in the **76-78% band** across model families and dataset types.

Study	Year	Dataset	Model	Acc. @15
Spaargaren, MSc thesis (Tilburg) [2]	2022	Oracle’s Elixir pro, 2021	Logistic Regression	77.99%
Spaargaren	2022	Oracle’s Elixir pro, 2021	Random Forest	77.26%
Spaargaren	2022	Oracle’s Elixir pro, 2021	Naïve Bayes	76.82%
Spaargaren	2022	Oracle’s Elixir pro, 2021	kNN (k=9)	74.83%
Lafrance & Grewal, <i>Entertainment Computing</i> [3]	2026	Riot API solo queue, Diamond	XGBoost	76.9%
Junior & Campelo, arXiv 2309.02449 [4]	2023	Riot API solo queue	Random Forest	~74%*

*Interpolated to 50% PET (~ 15 min for a ~30-min median match) from Junior & Campelo’s PET-bucket accuracy curves.

Professional play is not materially easier to predict than solo queue at 15 minutes: Spaargaren’s pro-data LR sits within one percentage point of Lafrance & Grewal’s solo-queue XGBoost. Algorithm choice matters little - pro-data classifiers all land in the 77 +/- 1% band, with

solo-queue ~3 points lower. Across every study, gold differential is the strongest single feature, with first tower or first herald typically ranked next.

Tsang (2025, Medium) [5] adds a temporal signal: the coefficient of `golddiffat15` has **declined** from above 1.0 in 2020-2023 to 0.83 in 2025, suggesting the 2025 season is harder to predict from a single gold-at-15 signal than any previous season. We additionally include two features absent from the @15-minute literature - **turret plate differential** and **void grubs (introduced in 2025 season)**.

Finally, we note a framing gap: the trivial rule “team with a gold lead at 15 min wins” already delivers ~73% accuracy on professional data (we compute the exact figure on our own set in the EDA), so the 76-78% models above only beat this naive baseline by 3-5 points. We adopt this framing directly in our results.

Setup

```
.required_pkgs <- c(
  "tidyverse", "corrplot", "caret", "glmnet", "randomForest",
  "naivebayes", "rpart", "rpart.plot", "xgboost", "pROC",
  "gridExtra", "doParallel", "MASS"
)
.missing_pkgs <- setdiff(.required_pkgs, rownames(installed.packages()))
if (length(.missing_pkgs)) {
  install.packages(.missing_pkgs, repos = "https://cloud.r-project.org")
}

library(tidyverse)
library(corrplot)
library(caret)
library(glmnet)
library(randomForest)
library(naivebayes)
library(rpart)
library(xgboost)
library(pROC)
library(gridExtra)
library(doParallel)

detected_cores <- parallel::detectCores(logical = TRUE)
if (is.na(detected_cores) || detected_cores < 2) {
  detected_cores <- 2
}
```

```

}

cl <- tryCatch(
  parallel::makePSOCKcluster(max(1, detected_cores - 1)),
  error = function(e) {
    message("Parallel backend unavailable; continuing sequentially: ", conditionMessage(e))
    NULL
  }
)

if (!is.null(cl)) {
  doParallel::registerDoParallel(cl)
} else {
  foreach::registerDoSEQ()
}

```

Load data

```

raw <- read_csv("2025_LoL_esports_match_data_from_OraclesElixir.csv")
glimpse(raw)

```

Rows: 120,636

Columns: 165

\$ gameid	<chr> "LOLTMNT03_179647", "LOLTMNT03_179647", "LOLT~
\$ datacompleteness	<chr> "complete", "complete", "complete", "complete~
\$ url	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
\$ league	<chr> "LFL2", "LFL2", "LFL2", "LFL2", "LFL2", "LFL2~
\$ year	<dbl> 2025, 2025, 2025, 2025, 2025, 2025, 2025, 202~
\$ split	<chr> "Winter", "Winter", "Winter", "Winter", "Wint~
\$ playoffs	<dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
\$ date	<dtm> 2025-01-11 11:11:24, 2025-01-11 11:11:24, 20~
\$ game	<dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
\$ patch	<dbl> 15.01, 15.01, 15.01, 15.01, 15.01, 15.01, 15.~
\$ participantid	<dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, 200, 1, 2~
\$ side	<chr> "Blue", "Blue", "Blue", "Blue", "Blue", "Red"~
\$ position	<chr> "top", "jng", "mid", "bot", "sup", "top", "jn~
\$ playername	<chr> "PatkicaA", "Joinze", "Sayn", "Shiganari", "L~
\$ playerid	<chr> "oe:player:c659697694306de62d978569b84c344", ~
\$ teamname	<chr> "IziDream", "IziDream", "IziDream", "IziDream~
\$ teamid	<chr> "oe:team:84bc703e28859788770611d94cf02ac", "o~

\$ firstPick	<dbl> 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, ~
\$ champion	<chr> "Gnar", "Maokai", "Hwei", "Jinx", "Leona", "R~
\$ ban1	<chr> "Vi", "Vi", "Vi", "Vi", "Vi", "Yone", "Yone", ~
\$ ban2	<chr> "Skarner", "Skarner", "Skarner", "Skarner", "~
\$ ban3	<chr> "Corki", "Corki", "Corki", "Corki", "Corki", ~
\$ ban4	<chr> "K'Sante", "K'Sante", "K'Sante", "K'Sante", "~
\$ ban5	<chr> "Sylas", "Sylas", "Sylas", "Sylas", "Sylas", ~
\$ pick1	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, "Maok~
\$ pick2	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, "Jinx~
\$ pick3	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, "Leon~
\$ pick4	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, "Hwei~
\$ pick5	<chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, "Gnar~
\$ gamelength	<dbl> 1592, 1592, 1592, 1592, 1592, 1592, 1592, 159~
\$ result	<dbl> 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, ~
\$ kills	<dbl> 1, 0, 1, 1, 0, 3, 1, 1, 8, 0, 3, 13, 1, 3, 6, ~
\$ deaths	<dbl> 2, 3, 2, 3, 3, 2, 0, 1, 0, 0, 13, 3, 3, 0, 0, ~
\$ assists	<dbl> 1, 1, 0, 1, 2, 4, 10, 8, 3, 11, 5, 36, 3, 17, ~
\$ teamkills	<dbl> 3, 3, 3, 3, 3, 13, 13, 13, 13, 13, 3, 13, 21, ~
\$ teamdeaths	<dbl> 13, 13, 13, 13, 13, 3, 3, 3, 3, 3, 13, 3, 11, ~
\$ doublekills	<dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, ~
\$ triplekills	<dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
\$ quadrakills	<dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
\$ pentakills	<dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
\$ firstblood	<dbl> 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, ~
\$ firstbloodkill	<dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, NA, NA, 0, 0, 0, ~
\$ firstbloodassist	<dbl> 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, NA, NA, 0, 1, 0, ~
\$ firstbloodvictim	<dbl> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, NA, NA, 0, 0, 0, ~
\$ `team kpm`	<dbl> 0.1131, 0.1131, 0.1131, 0.1131, 0.1131, 0.489~
\$ ckpm	<dbl> 0.6030, 0.6030, 0.6030, 0.6030, 0.6030, 0.603~
\$ firstdragon	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ dragons	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 2, ~
\$ opp_dragons	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 2, 0, ~
\$ elementaldrakes	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 2, ~
\$ opp_elementaldrakes	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 2, 0, ~
\$ infernals	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ mountains	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 0, ~
\$ clouds	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ oceans	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 0, ~
\$ chemtechs	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 0, ~
\$ hextechs	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 0, ~
\$ `dragons (type unknown)`	<lg1> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
\$ elders	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 0, ~
\$ opp_elders	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 0, ~

\$ firstherald	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ heralds	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ opp_heralds	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 1, 0, ~
\$ void_grubs	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 6, ~
\$ opp_void_grubs	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 6, 0, ~
\$ firstbaron	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ barons	<dbl> 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, ~
\$ opp_barons	<dbl> 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, ~
\$ atakhans	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ opp_atakhans	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 1, 0, ~
\$ firsttower	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ towers	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 3, 9, ~
\$ opp_towers	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 9, 3, ~
\$ firstmidtower	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ firstttothreetowers	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, 1, ~
\$ turretplates	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 1, 8, ~
\$ opp_turretplates	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 8, 1, ~
\$ inhibitors	<dbl> 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 2, 0, 2, 1, ~
\$ opp_inhibitors	<dbl> 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, ~
\$ damagetochampions	<dbl> 20156, 4963, 13952, 6898, 4174, 14166, 4924, ~
\$ dpm	<dbl> 759.6482, 187.0477, 525.8291, 259.9749, 157.3~
\$ damageshare	<dbl> 0.4019700, 0.0989769, 0.2782440, 0.1375670, 0~
\$ damagetakenperminute	<dbl> 681.2186, 906.9724, 366.8593, 479.2085, 474.5~
\$ damagemitigatedperminute	<dbl> 629.7362, 954.6106, 241.0553, 282.9271, 534.1~
\$ damagetotowers	<dbl> 7451, 0, 0, 333, 0, 4339, 4231, 3495, 8851, 1~
\$ wardsplaced	<dbl> 9, 7, 12, 6, 40, 11, 6, 11, 8, 42, 74, 78, 11~
\$ wpm	<dbl> 0.3392, 0.2638, 0.4523, 0.2261, 1.5075, 0.414~
\$ wardskilled	<dbl> 2, 7, 3, 2, 10, 4, 9, 11, 6, 5, 24, 35, 7, 11~
\$ wcpm	<dbl> 0.0754, 0.2638, 0.1131, 0.0754, 0.3769, 0.150~
\$ controlwardsbought	<dbl> 3, 6, 4, 1, 9, 4, 4, 5, 5, 15, 23, 33, 5, 7, ~
\$ visionscore	<dbl> 17, 29, 20, 10, 82, 26, 34, 35, 21, 80, 158, ~
\$ vspm	<dbl> 0.6407, 1.0930, 0.7538, 0.3769, 3.0905, 0.979~
\$ totalgold	<dbl> 10668, 7429, 9032, 9407, 5719, 10814, 10740, ~
\$ earnedgold	<dbl> 7145, 3906, 5509, 5884, 2196, 7291, 7217, 795~
\$ `earned gpm`	<dbl> 269.2839, 147.2111, 207.6256, 221.7588, 82.76~
\$ earnedgoldshare	<dbl> 0.2899810, 0.1585200, 0.2235810, 0.2388010, 0~
\$ goldspent	<dbl> 9793, 7200, 8050, 8925, 4825, 8650, 9250, 937~
\$ gspd	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, -
0.11~	
\$ gpr	<dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, -
3.34~	
\$ `total cs`	<dbl> 234, 143, 209, 251, 38, 214, 166, 254, 265, 2~
\$ minionkills	<dbl> 234, 11, 209, 239, 38, 212, 11, 254, 253, 23, ~

```

$ monsterkills <dbl> 0, 132, 0, 12, 0, 2, 155, 0, 12, 0, 144, 169,~
$ monsterkillsownjungle <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ monsterkillsenemyjungle <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ cspm <dbl> 8.8191, 5.3894, 7.8769, 9.4598, 1.4322, 8.065~
$ goldat10 <dbl> 3058, 2977, 3020, 3097, 2114, 3394, 3451, 342~
$ xpat10 <dbl> 4466, 3153, 4584, 3403, 2913, 4603, 3687, 478~
$ csat10 <dbl> 75, 62, 75, 81, 20, 79, 71, 91, 92, 12, 313, ~
$ opp_goldat10 <dbl> 3394, 3451, 3425, 3519, 2199, 3058, 2977, 302~
$ opp_xpat10 <dbl> 4603, 3687, 4789, 3682, 2420, 4466, 3153, 458~
$ opp_csat10 <dbl> 79, 71, 91, 92, 12, 75, 62, 75, 81, 20, 345, ~
$ golddiffat10 <dbl> -336, -474, -405, -422, -85, 336, 474, 405, 4~
$ xpdiffat10 <dbl> -137, -534, -205, -279, 493, 137, 534, 205, 2~
$ csdiffat10 <dbl> -4, -9, -16, -11, 8, 4, 9, 16, 11, -8, -
32, 3~
$ killsat10 <dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 2, 2, ~
$ assistsat10 <dbl> 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 6, 3, ~
$ deathsat10 <dbl> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, ~
$ opp_killsat10 <dbl> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, ~
$ opp_assistsat10 <dbl> 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, ~
$ opp_deathsat10 <dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 3, 2, ~
$ goldat15 <dbl> 4531, 4461, 4581, 4687, 3238, 5372, 5289, 566~
$ xpat15 <dbl> 6777, 5316, 6530, 5589, 4058, 6968, 5332, 703~
$ csat15 <dbl> 119, 97, 116, 133, 31, 125, 91, 141, 138, 17,~
$ opp_goldat15 <dbl> 5372, 5289, 5666, 5598, 3410, 4531, 4461, 458~
$ opp_xpat15 <dbl> 6968, 5332, 7031, 5789, 3619, 6777, 5316, 653~
$ opp_csat15 <dbl> 125, 91, 141, 138, 17, 119, 97, 116, 133, 31,~
$ golddiffat15 <dbl> -841, -828, -1085, -911, -172, 841, 828, 1085~
$ xpdiffat15 <dbl> -191, -16, -501, -200, 439, 191, 16, 501, 200~
$ csdiffat15 <dbl> -6, 6, -25, -5, 14, 6, -6, 25, 5, -14, -
16, 1~
$ killsat15 <dbl> 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 3, 0, 3, 2, ~
$ assistsat15 <dbl> 0, 0, 0, 0, 0, 2, 3, 1, 1, 2, 0, 9, 0, 7, 4, ~
$ deathsat15 <dbl> 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 3, 0, 1, 0, 0, ~
$ opp_killsat15 <dbl> 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 3, 0, 1, 2, 1, ~
$ opp_assistsat15 <dbl> 2, 3, 1, 1, 2, 0, 0, 0, 0, 0, 0, 9, 0, 1, 1, 2, ~
$ opp_deathsat15 <dbl> 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 3, 1, 3, 3, ~
$ goldat20 <dbl> 6473, 5668, 6332, 6782, 4220, 7012, 7357, 799~
$ xpat20 <dbl> 9072, 6978, 9111, 8435, 5063, 9562, 8317, 105~
$ csat20 <dbl> 154, 118, 166, 183, 35, 154, 133, 195, 202, 1~
$ opp_goldat20 <dbl> 7012, 7357, 7990, 8489, 4530, 6473, 5668, 633~
$ opp_xpat20 <dbl> 9562, 8317, 10550, 9683, 4749, 9072, 6978, 91~
$ opp_csat20 <dbl> 154, 133, 195, 202, 18, 154, 118, 166, 183, 3~
$ golddiffat20 <dbl> -539, -1689, -1658, -1707, -310, 539, 1689, 1~

```



```

$ xpdiffat20      <dbl> -490, -1339, -1439, -1248, 314, 490, 1339, 14~
$ csdiffat20      <dbl> 0, -15, -29, -19, 17, 0, 15, 29, 19, -17, -
46~
$ killsat20       <dbl> 1, 0, 0, 1, 0, 2, 0, 1, 3, 0, 2, 6, 0, 3, 4, ~
$ assistsat20     <dbl> 1, 1, 0, 0, 1, 2, 5, 2, 1, 4, 3, 14, 1, 10, 5~
$ deathsat20      <dbl> 2, 1, 1, 1, 1, 2, 0, 0, 0, 0, 6, 2, 2, 0, 0, ~
$ opp_killsat20   <dbl> 2, 0, 1, 3, 0, 1, 0, 0, 1, 0, 6, 2, 1, 2, 2, ~
$ opp_assistsat20 <dbl> 2, 5, 2, 1, 4, 1, 1, 0, 0, 1, 14, 3, 1, 3, 2, ~
$ opp_deathsat20  <dbl> 2, 0, 0, 0, 0, 2, 1, 1, 1, 1, 2, 6, 1, 4, 3, ~
$ goldat25        <dbl> 9244, 7040, 8488, 9097, 5357, 9020, 9403, 100~
$ xpat25          <dbl> 12552, 8877, 10669, 11763, 6259, 12553, 10321~
$ csat25          <dbl> 217, 139, 200, 251, 38, 200, 157, 241, 257, 2~
$ opp_goldat25    <dbl> 9020, 9403, 10040, 11710, 6019, 9244, 7040, 8~
$ opp_xpat25      <dbl> 12553, 10321, 13134, 12919, 6993, 12552, 8877~
$ opp_csat25      <dbl> 200, 157, 241, 257, 20, 217, 139, 200, 251, 3~
$ golddiffat25    <dbl> 224, -2363, -1552, -2613, -662, -224, 2363, 1~
$ xpdiffat25      <dbl> -1, -1444, -2465, -1156, -734, 1, 1444, 2465,~
$ csdiffat25      <dbl> 17, -18, -41, -6, 18, -17, 18, 41, 6, -18, -
3~
$ killsat25       <dbl> 1, 0, 1, 1, 0, 2, 1, 1, 6, 0, 3, 10, 0, 3, 4,~
$ assistsat25     <dbl> 1, 1, 0, 1, 2, 4, 7, 5, 2, 8, 5, 26, 2, 11, 7~
$ deathsat25      <dbl> 2, 2, 2, 2, 2, 2, 0, 1, 0, 0, 10, 3, 2, 0, 0,~
$ opp_killsat25   <dbl> 2, 1, 1, 6, 0, 1, 0, 1, 1, 0, 10, 3, 1, 2, 2,~
$ opp_assistsat25 <dbl> 4, 7, 5, 2, 8, 1, 1, 0, 1, 2, 26, 5, 1, 3, 2,~
$ opp_deathsat25  <dbl> 2, 0, 1, 0, 0, 2, 2, 2, 2, 2, 3, 10, 1, 5, 3,~

```

```

# select only complete data - with @10, 15, 20, 25 metrics
complete <- raw %>% filter(datacompleteness == "complete")

```

```

teams <- complete %>% filter(position == "team")
players <- complete %>% filter(position != "team")

```

```

cat(sprintf(
  "Total rows: %d\nFiltered to complete: %d (removed %d incomplete, %.1f%%)\nTeam rows: %d\n",
  nrow(raw), nrow(complete),
  nrow(raw) - nrow(complete),
  (nrow(raw) - nrow(complete)) / nrow(raw) * 100,
  nrow(teams), nrow(players),
  mean(teams$result == 1) * 100, mean(teams$result == 0) * 100
))

```

Total rows: 120636

Filtered to complete: 110832 (removed 9804 incomplete, 8.1%)
Team rows: 18472
Player rows: 92360
Target balance: 50.0% win / 50.0% loss

Restructure: one row per game

The raw `teams` table stores two mirror rows per game, one per side. For EDA we pivot to one row per game with `blue_*` and `red_*` columns and a binary `blue_win` target, following the **tidy data** convention of one observation per row (Wickham, 2014, *JSS* 59(10)) [6].

After the pivot, the Blue and Red copies of each signed differential and each mutually-exclusive binary flag are pure mirrors: `red_golddiffat15 = -blue_golddiffat15` and `red_firstblood = 1 - blue_firstblood`. We drop the `red_*` side of those columns, since the model gains nothing from a feature it can recover from the one next to it. Independent counts (kills, deaths, assists, turret plates) and the shared-pool grub count stay on both sides - neither side can be reconstructed from the other.

```
game_cols <- c(
  "result",
  "goldat10", "xpat10", "csat10",
  "goldat15", "xpat15", "csat15",
  "golddiffat10", "xpdiffat10", "csdiffat10",
  "golddiffat15", "xpdiffat15", "csdiffat15",
  "killsat10", "deathsat10", "assistsat10",
  "killsat15", "deathsat15", "assistsat15",
  "firstblood", "firstdragon", "firstherald", "firsttower",
  "void_grubs", "turretplates"
)

games_full <- teams %>%
  select(gameid, side, all_of(game_cols)) %>%
  mutate(side = tolower(side)) %>%
  pivot_wider(
    id_cols      = gameid,
    names_from   = side,
    values_from  = all_of(game_cols),
    names_glue   = "{side}_{.value}"
  ) %>%
  mutate(blue_win = blue_result) %>%
  select(-blue_result, -red_result)
```

```

# Sanity: blue/red signed diffs must be exact negatives of each other,
# and mutually exclusive binary flags must satisfy red = 1 - blue whenever
# the objective fires.
stopifnot(all.equal(games_full$blue_golddiffat15, -games_full$red_golddiffat15))
stopifnot(all.equal(games_full$blue_xpdiffat15, -games_full$red_xpdiffat15))
stopifnot(all.equal(games_full$blue_csdiffat15, -games_full$red_csdiffat15))
stopifnot(all(games_full$blue_firstblood + games_full$redirstblood <= 1))
stopifnot(all(games_full$blue_firsttower + games_full$red_firsttower <= 1))

# Drop the strict-mirror columns identified above. We keep both sides for
# independent counts (kills/deaths/assists/turret plates) and for the
# shared-pool grub count, since those carry information beyond their pair.
redundant_mirrors <- c(
  "red_golddiffat10", "red_xpdiffat10", "red_csdiffat10",
  "red_golddiffat15", "red_xpdiffat15", "red_csdiffat15",
  "red_firstblood", "red_firstdragon", "red_firstherald", "red_firsttower"
)

games <- games_full %>% select(-all_of(redundant_mirrors))

cat(sprintf(
  "Games (one row per game): %d\nFeatures dropped as strict mirrors: %d\nFeatures retained\n",
  nrow(games),
  length(redundant_mirrors),
  ncol(games) - 2, # excluding gameid and blue_win
  mean(games$blue_win) * 100,
  mean(1 - games$blue_win) * 100
))

```

```

Games (one row per game): 9236
Features dropped as strict mirrors: 10
Features retained: 38
Blue win rate: 53.28% (Red: 46.72%)

```

```

# Statistical test for side imbalance
side_test <- prop.test(sum(games$blue_win), nrow(games), p = 0.5)
cat(sprintf(
  "\nTwo-sided proportion test vs 50%:\n  chi^2 = %.2f, df = 1, p = %.3g\n  95% CI for B",
  side_test$statistic, side_test$p.value,
  side_test$conf.int[1], side_test$conf.int[2],
  ifelse(side_test$p.value < 0.05, "statistically significant", "not statistically significant")
))

```

Two-sided proportion test vs 50%:

$\chi^2 = 39.63$, $df = 1$, $p = 3.07e-10$

95% CI for Blue WR: [0.523, 0.543]

-> Blue-side advantage is statistically significant.

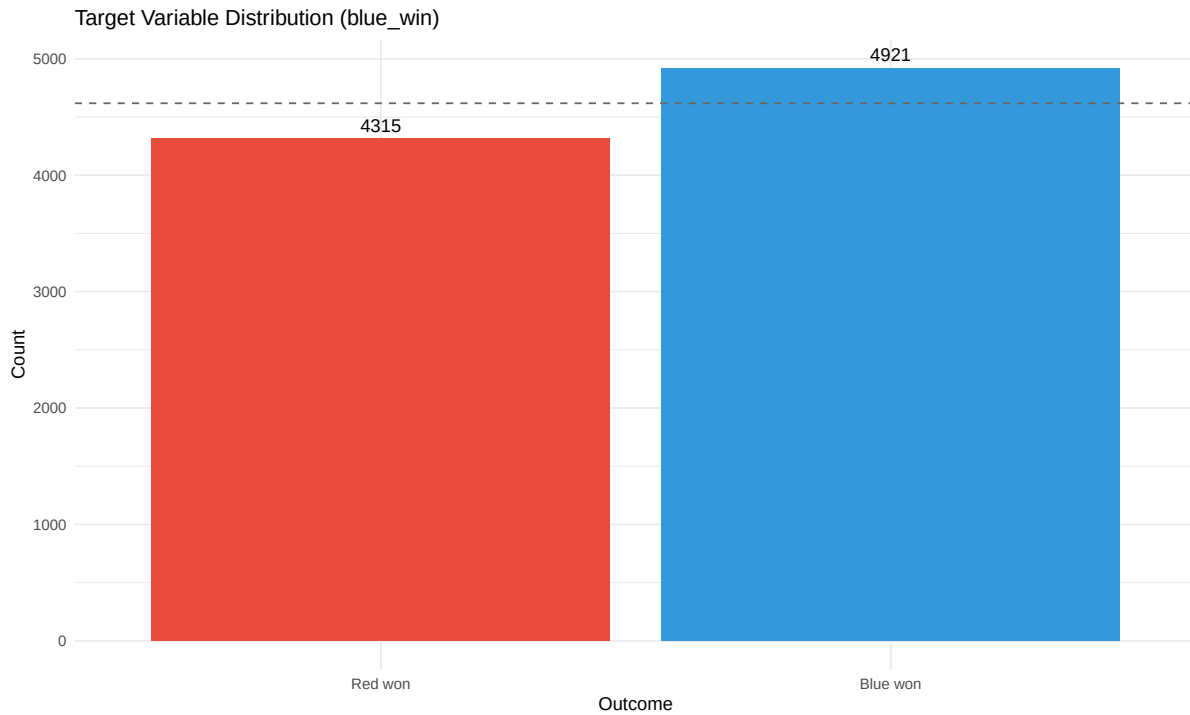
Blue side wins slightly more than half of games. The imbalance is mild enough that no resampling is needed and accuracy remains valid alongside AUC-ROC.

Exploratory Data Analysis

Everything below uses the **single-row-per-game** `games` dataset. The aim is to see what signal is available and catch any issues before we start modelling.

Target distribution

```
games %>%
  mutate(blue_win = factor(blue_win, levels = c(0, 1), labels = c("Red won", "Blue won")))
  ggplot(aes(x = blue_win, fill = blue_win)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.5) +
  scale_fill_manual(values = c("Red won" = "#e74c3c", "Blue won" = "#3498db")) +
  geom_hline(yintercept = nrow(games) / 2, linetype = "dashed", color = "gray40") +
  labs(
    title = "Target Variable Distribution (blue_win)",
    x = "Outcome", y = "Count"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



Blue wins slightly more than half of games ($p < 0.001$), but the imbalance is only roughly 3 percentage points above 50% - small enough to keep accuracy as a valid metric and skip resampling.

Descriptive statistics

We sort the early-game numerics by standard deviation. Features with much larger spread will dominate distance-based models like KNN unless they are scaled, which is why we centre and standardise later in the pipeline.

```
early_game_cols <- c(
  "blue_goldat10", "blue_xpat10", "blue_csat10",
  "blue_golddiffat10", "blue_xpdiffat10", "blue_csdiffat10",
  "blue_killsat10", "blue_deathsat10", "blue_assistsat10",
  "blue_goldat15", "blue_xpat15", "blue_csat15",
  "blue_golddiffat15", "blue_xpdiffat15", "blue_csdiffat15",
  "blue_killsat15", "blue_deathsat15", "blue_assistsat15",
  "blue_firstblood", "blue_firstdragon", "blue_firsttherald", "blue_firsttower",
  "blue_void_grubs", "red_void_grubs",
  "blue_turretplates", "red_turretplates",
  "blue_win"
)
```

```

games %>%
  select(all_of(setdiff(early_game_cols, "blue_win"))) %>%
  pivot_longer(everything(), names_to = "feature", values_to = "value") %>%
  drop_na() %>%
  group_by(feature) %>%
  summarise(
    n = n(),
    mean = mean(value),
    sd = sd(value),
    min = min(value),
    q25 = quantile(value, 0.25),
    median = median(value),
    q75 = quantile(value, 0.75),
    max = max(value),
    .groups = "drop"
  ) %>%
  mutate(across(c(mean, sd, min, q25, median, q75, max), ~ round(.x, 2))) %>%
  arrange(desc(sd)) %>%
  print(n = Inf)

```

A tibble: 26 x 9

	feature	n	mean	sd	min	q25	median	q75	max
	<chr>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	blue_golddiffat15	9236	243.	2560.	-11898	-1351	230	1761	13867
2	blue_xpdiffat15	9236	49.0	1934.	-9538	-1188	53	1265.	9395
3	blue_goldat15	9236	25099.	1642.	20419	23967	24892	26004.	35540
4	blue_xpat15	9236	30100.	1328.	24710	29244	30131	30981	35284
5	blue_golddiffat10	9236	113.	1299.	-7330	-691	115	924	7178
6	blue_xpdiffat10	9236	41	973.	-4863	-565	35	666.	5309
7	blue_goldat10	9236	16060.	874.	13352	15449	15964	16561	22123
8	blue_xpat10	9236	18891.	825.	14758	18374	18945	19468	21625
9	blue_csdiffat15	9236	1.55	37.8	-213	-22	1	25	223
10	blue_csat15	9236	520.	34.0	356	498	522	544	613
11	blue_csdiffat10	9236	1.62	25.0	-114	-15	1	18	135
12	blue_csat10	9236	330.	22.8	225	316	332	346	399
13	blue_assistsat15	9236	7.75	5.69	0	3	7	11	43
14	blue_assistsat10	9236	3.61	3.45	0	1	3	5	22
15	blue_killsat15	9236	4.56	3.02	0	2	4	6	24
16	blue_deathsat15	9236	4.38	2.95	0	2	4	6	27
17	blue_turretplates	9236	4.08	2.57	0	2	4	6	15
18	red_turretplates	9236	3.72	2.49	0	2	3	5	15

19	blue_deathsat10	9236	2.26	1.92	0	1	2	3	19
20	blue_killsat10	9236	2.34	1.92	0	1	2	3	18
21	blue_void_grubs	9236	2.5	1.76	0	1	3	3	6
22	red_void_grubs	9236	1.71	1.62	0	0	2	3	6
23	blue_firstblood	9236	0.51	0.5	0	0	1	1	1
24	blue_firsttower	9236	0.55	0.5	0	0	1	1	1
25	blue_firstdragon	9236	0.38	0.49	0	0	0	1	1
26	blue_firstherald	9236	0.61	0.49	0	0	1	1	1

Gold and XP have the widest spread, with standard deviations in the thousands; @15 totals cluster around 25k gold and 30k XP. CS columns are an order of magnitude smaller, kill/death/assist counts smaller still, and the 0/1 objectives sit at the bottom with sd around 0.5. The differentials (`blue_minus_red`) average close to zero (243 gold and 49 XP at 15), which is what we would expect across balanced matches - the small positive offset reflects the Blue-side advantage.

Early-game feature correlations with blue_win

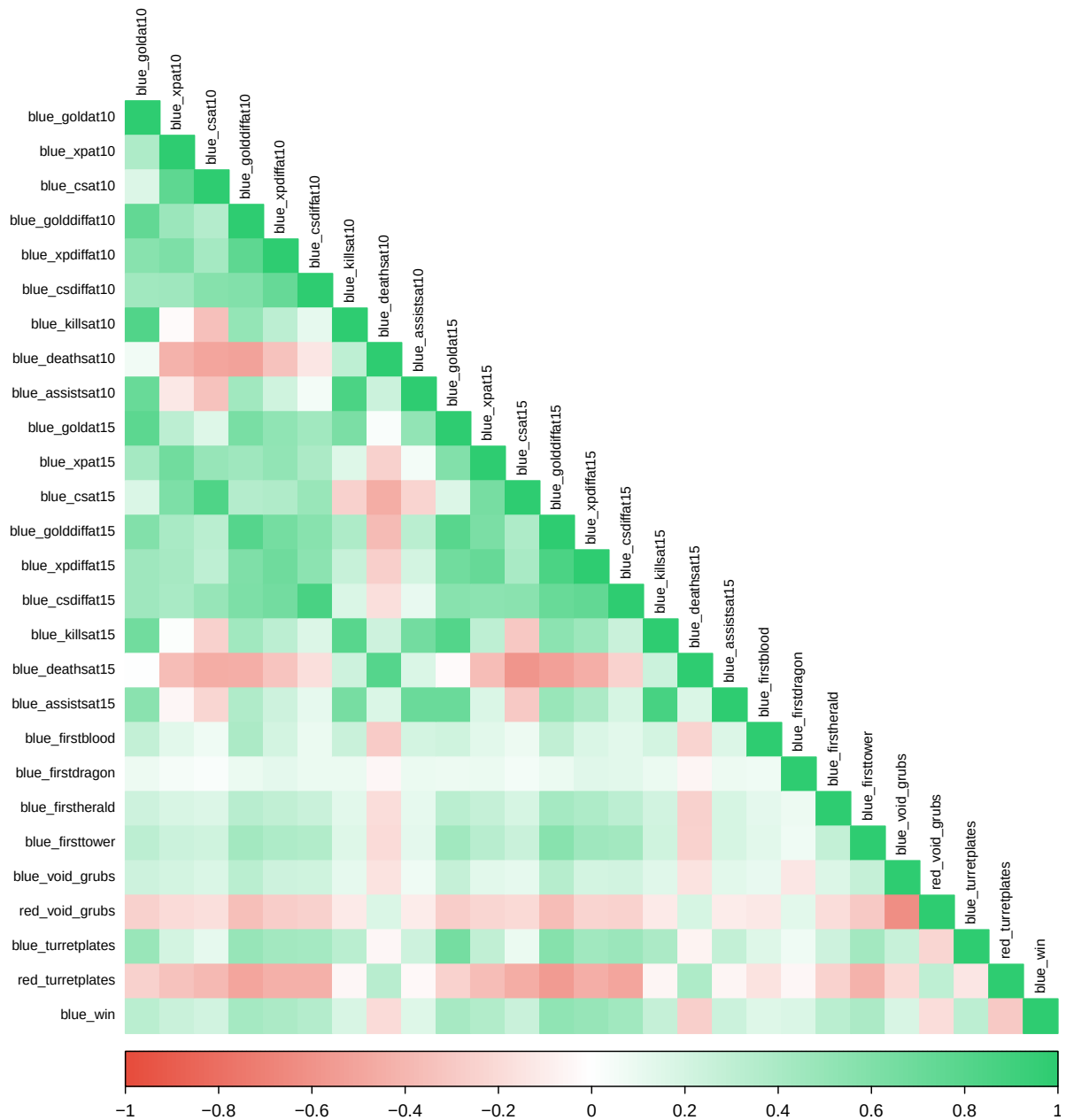
We correlate each early-game numeric feature (Blue's perspective) with the target.

```
cor_data <- games %>%
  select(all_of(early_game_cols)) %>%
  drop_na()

cor_matrix <- cor(cor_data)

corrplot(cor_matrix,
  method = "color", type = "lower",
  tl.cex = 0.7, tl.col = "black",
  col = colorRampPalette(c("#e74c3c", "white", "#2ecc71"))(200),
  col.lim = c(-1, 1),
  title = "Correlation Matrix - Early-Game Features (Blue's perspective)",
  mar = c(0, 0, 2, 0)
)
```

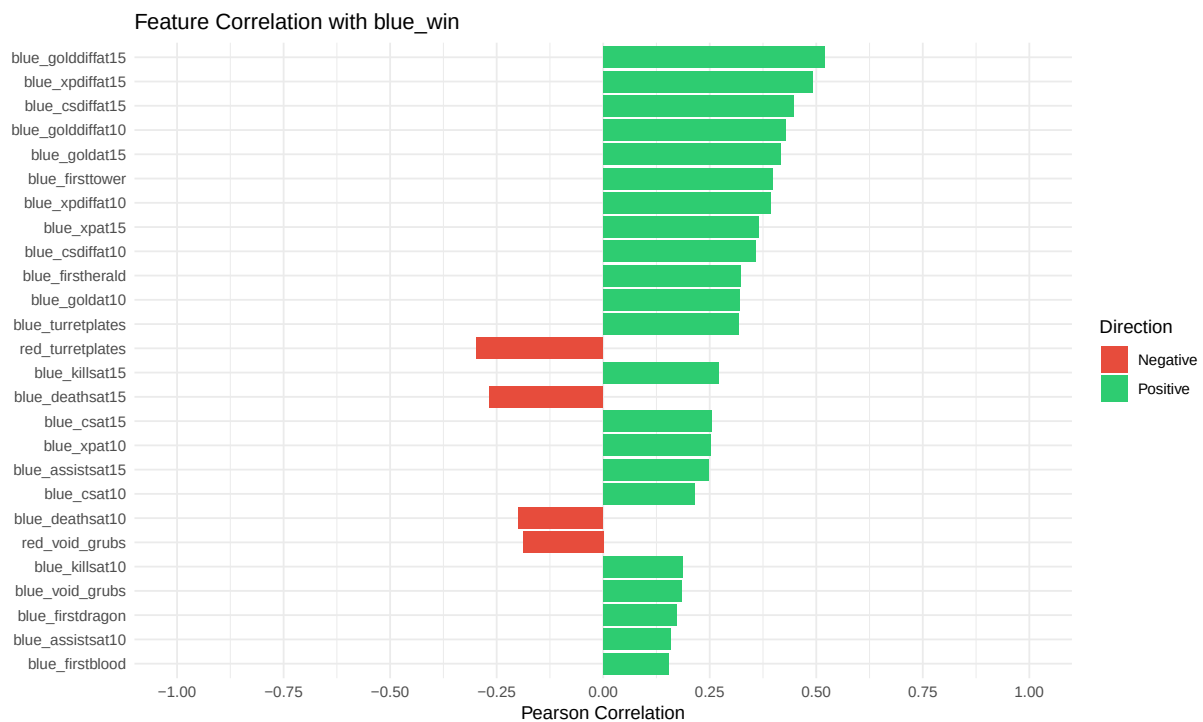
Correlation Matrix – Early-Game Features (Blue's perspective)



```
result_cors <- cor_data %>%
  select(-blue_win) %>%
  summarise(across(everything(), ~ cor(.x, cor_data$blue_win))) %>%
  pivot_longer(everything(), names_to = "feature", values_to = "correlation") %>%
  arrange(desc(abs(correlation)))
```



```
ggplot(result_cors, aes(
  x = reorder(feature, abs(correlation)),
  y = correlation,
  fill = correlation > 0
)) +
  geom_col() +
  coord_flip(ylim = c(-1, 1)) +
  scale_y_continuous(breaks = seq(-1, 1, 0.25)) +
  scale_fill_manual(
    values = c("TRUE" = "#2ecc71", "FALSE" = "#e74c3c"),
    labels = c("Negative", "Positive")
  ) +
  labs(
    title = "Feature Correlation with blue_win",
    x = NULL, y = "Pearson Correlation", fill = "Direction"
  ) +
  theme_minimal()
```



```
cat(sprintf(
  "Strongest positive: %s (%.3f)\nStrongest negative: %s (%.3f)",
  result_cors$feature[which.max(result_cors$correlation)],
  result_cors$correlation[which.max(result_cors$correlation)],
  result_cors$feature[which.min(result_cors$correlation)],
  result_cors$correlation[which.min(result_cors$correlation)]
))
```

```

    max(result_cors$correlation),
    result_cors$feature[which.min(result_cors$correlation)],
    min(result_cors$correlation)
  ))

```

Strongest positive: blue_golddiffat15 (0.520)

Strongest negative: red_turretplates (-0.297)

Differentials (blue_golddiffat15, blue_xpdiffat15, blue_csdiffat15) dominate - what matters is the *gap*, not the absolute totals. Among objectives, blue_firsttower and blue_firstherald stand out, while blue_firstblood and blue_firstdragon are weaker. The matrix also shows two multicollinearity patterns: a raw stat against its own differential (blue_goldat15 vs blue_golddiffat15), and Blue/Red shared-pool pairs - blue_void_grubs and red_void_grubs are negatively correlated because the 6-grub pit is shared between the two teams. Both patterns **motivate the LASSO and Elastic Net regularization used in Scenario 3.**

Per-outcome means

```

games %>%
  select(all_of(early_game_cols)) %>%
  drop_na() %>%
  mutate(outcome = if_else(blue_win == 1, "blue_win", "red_win")) %>%
  select(-blue_win) %>%
  pivot_longer(-outcome, names_to = "feature", values_to = "value") %>%
  group_by(feature, outcome) %>%
  summarise(mean = mean(value), .groups = "drop") %>%
  pivot_wider(names_from = outcome, values_from = mean) %>%
  mutate(diff = blue_win - red_win) %>%
  arrange(desc(abs(diff))) %>%
  mutate(across(c(blue_win, red_win, diff), ~ round(.x, 3))) %>%
  print(n = Inf)

```

A tibble: 26 x 4

	feature	blue_win	red_win	diff
	<chr>	<dbl>	<dbl>	<dbl>
1	blue_golddiffat15	1490.	-1180.	2670.
2	blue_xpdiffat15	939.	-966.	1905.
3	blue_goldat15	25741.	24367.	1374.

4	blue_goldldiffat10	634.	-480.	1114.
5	blue_xpat15	30554.	29583.	971.
6	blue_xpdiffat10	399.	-367.	766.
7	blue_goldat10	16323.	15761.	562.
8	blue_xpat10	19085.	18669.	416.
9	blue_csdiffat15	17.4	-16.5	33.9
10	blue_csdiffat10	10.0	-7.94	17.9
11	blue_csat15	528.	510.	17.3
12	blue_csat10	335.	325.	9.76
13	blue_assistsat15	9.07	6.24	2.83
14	blue_turretplates	4.85	3.21	1.64
15	blue_killsat15	5.32	3.69	1.64
16	blue_deathsat15	3.64	5.22	-1.58
17	red_turretplates	3.02	4.51	-1.48
18	blue_assistsat10	4.12	3.02	1.10
19	blue_deathsat10	1.90	2.66	-0.761
20	blue_killsat10	2.68	1.96	0.721
21	blue_void_grubs	2.80	2.15	0.647
22	red_void_grubs	1.42	2.03	-0.613
23	blue_firsttower	0.733	0.335	0.398
24	blue_firstherald	0.754	0.438	0.316
25	blue_firstdragon	0.46	0.292	0.168
26	blue_firstblood	0.585	0.43	0.155

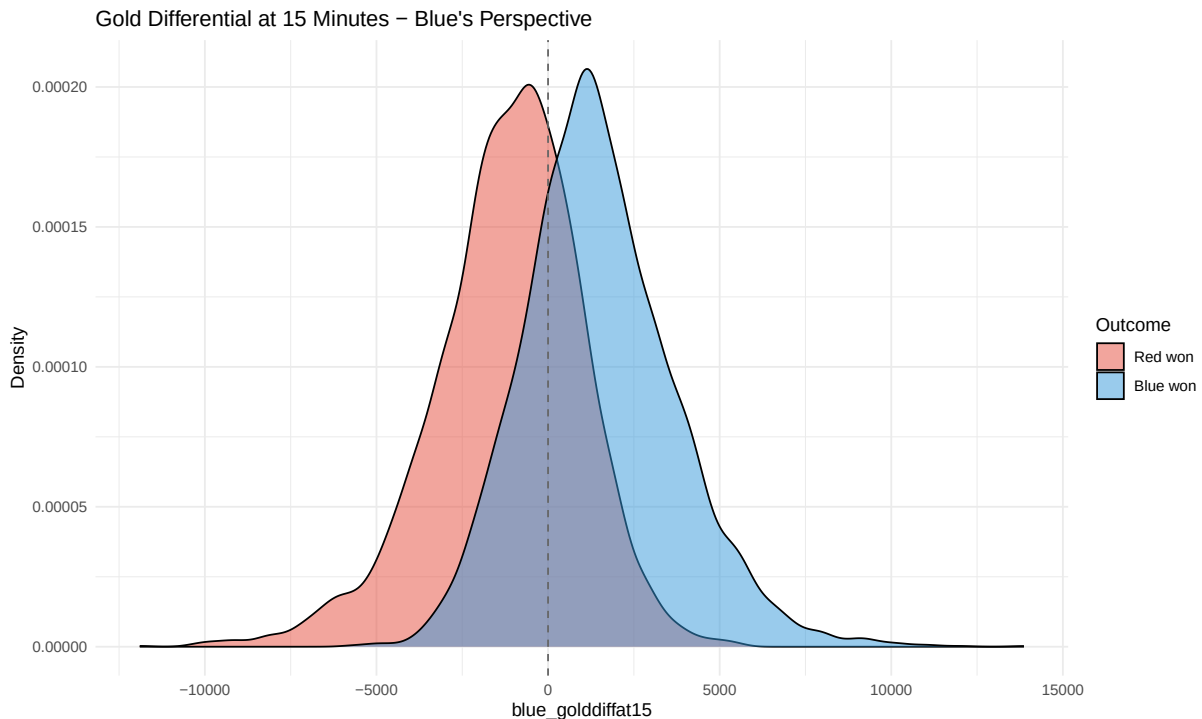
Gold and XP differentials at 15 minutes produce the largest gaps. When Blue wins, it leads by ~1500 gold and ~940 XP at 15 min; when it loses, it trails by roughly the same amount. Among the 0/1 features, `blue_firsttower` jumps from 0.33 in losses to 0.73 in wins and `blue_firstherald` from 0.44 to 0.75, while first blood and first dragon shift the means by less than 0.17. `blue_deathsat15` is the only count where the win-mean is *lower* than the loss-mean (3.6 vs 5.2), and `red_void_grubs` is negatively associated with `blue_win` because grubs taken by the enemy team hurt Blue.

Gold differential distribution

Gold differential at 15 minutes is the single strongest early-game predictor. The plots below show how cleanly it separates wins from losses and whether a simple threshold already captures most of the signal.

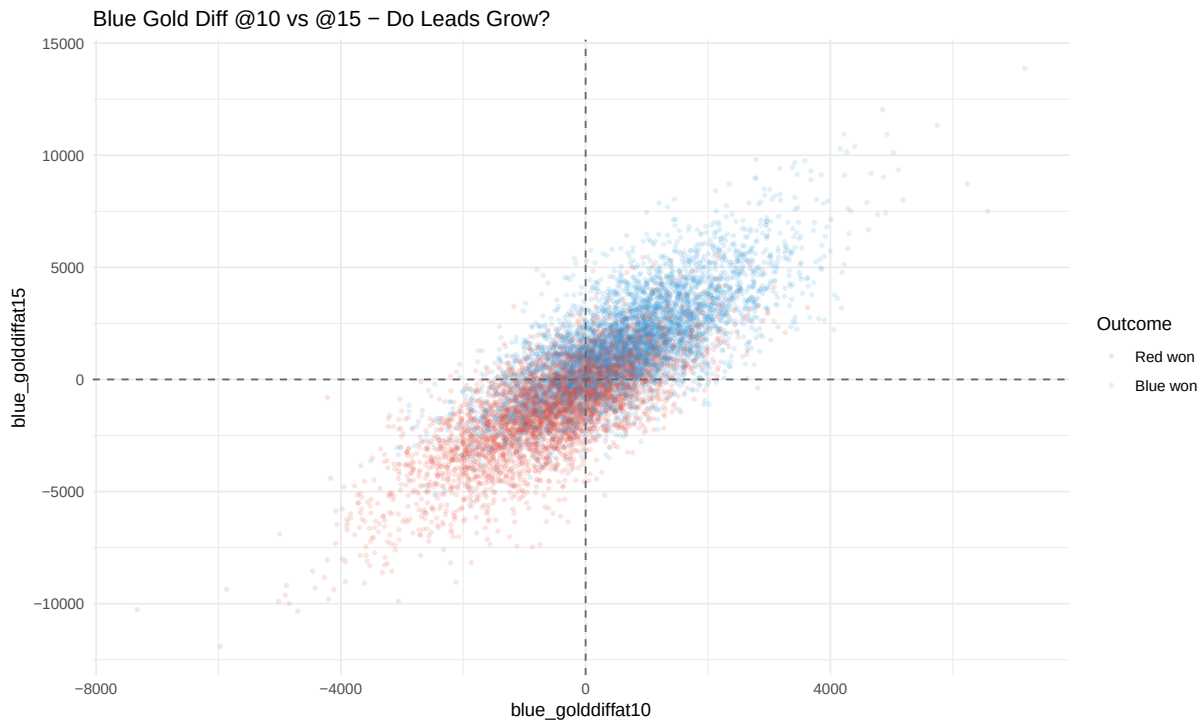
```
games %>%
  filter(!is.na(blue_goldldiffat15)) %>%
  mutate(blue_win = factor(blue_win, levels = c(0, 1), labels = c("Red won", "Blue won")))
```

```
ggplot(aes(x = blue_golddiffat15, fill = blue_win)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("Red won" = "#e74c3c", "Blue won" = "#3498db")) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray40") +
  labs(
    title = "Gold Differential at 15 Minutes - Blue's Perspective",
    x = "blue_golddiffat15", y = "Density", fill = "Outcome"
  ) +
  theme_minimal()
```



```
games %>%
  filter(!is.na(blue_golddiffat10), !is.na(blue_golddiffat15)) %>%
  mutate(blue_win = factor(blue_win, levels = c(0, 1), labels = c("Red won", "Blue won")))
ggplot(aes(x = blue_golddiffat10, y = blue_golddiffat15, color = blue_win)) +
  geom_point(alpha = 0.15, size = 0.8) +
  scale_color_manual(values = c("Red won" = "#e74c3c", "Blue won" = "#3498db")) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray40") +
  labs(
    title = "Blue Gold Diff @10 vs @15 - Do Leads Grow?",
    x = "blue_golddiffat10", y = "blue_golddiffat15", color = "Outcome"
  )
```

```
) +  
theme_minimal()
```



We score the naive rule “Blue ahead in gold @15 -> Blue wins” directly on `games`. The result defines the baseline any trained model has to beat.

```
gold_baseline_df <- games %>%  
  filter(!is.na(blue_golddiffat15)) %>%  
  transmute(  
    pred_blue_win = blue_golddiffat15 > 0,  
    actual         = blue_win == 1  
  )  
  
gold_baseline_acc <- mean(gold_baseline_df$pred_blue_win == gold_baseline_df$actual)  
gold_baseline_n <- nrow(gold_baseline_df)  
gold_baseline_ties <- sum(games$blue_golddiffat15 == 0, na.rm = TRUE)  
blue_base_rate <- mean(games$blue_win)  
  
cat(sprintf(  
  "Naive rule 'Blue ahead in gold @15 -> Blue wins':\n  N = %d games (%d games tied at exa  
  gold_baseline_n, gold_baseline_ties,
```

```

gold_baseline_acc * 100,
blue_base_rate * 100,
(gold_baseline_acc - blue_base_rate) * 100
))

```

Naive rule 'Blue ahead in gold @15 -> Blue wins':
 N = 9236 games (1 games tied at exactly 0 gold diff)
 Accuracy: 72.88%
 Always-predict-Blue baseline: 53.28%
 Rule beats always-Blue by: 19.60 pp

The two densities are clearly separated. The naive rule hits 72.9% accuracy - well above the 50% coin-flip and the 53.3% always-Blue baseline - and any model has to beat it to be worth keeping. The wide overlap around zero leaves room for non-linear models to find signal beyond gold alone.

First objective win rates

The four “first” objectives (first blood, first dragon, first herald, first tower) all happen in the first ~14 minutes; exactly one team secures each. For every objective we compute the securer’s win rate and a chi-square test of independence against outcome.

```

obj_cols <- c("firstblood", "firstdragon", "firstherald", "firsttower")

obj_winrates <- map_dfr(obj_cols, function(obj) {
  blue_col <- paste0("blue_", obj)
  df <- games %>%
    select(all_of(c(blue_col, "blue_win"))) %>%
    drop_na()

  n_blue <- sum(df[[blue_col]] == 1)
  n_red <- sum(df[[blue_col]] == 0)
  blue_wr <- mean(df$blue_win[df[[blue_col]] == 1]) * 100
  red_wr <- mean(df$blue_win[df[[blue_col]] == 0]) * 100 # P(Blue wins | Red secured)

  chi <- chisq.test(table(df[[blue_col]], df$blue_win))

  tibble(
    objective      = str_replace(obj, "first", "First ") %>% str_to_title(),
    secured_by     = c("Blue secured", "Red secured"),
    win_rate_blue = c(blue_wr, red_wr),

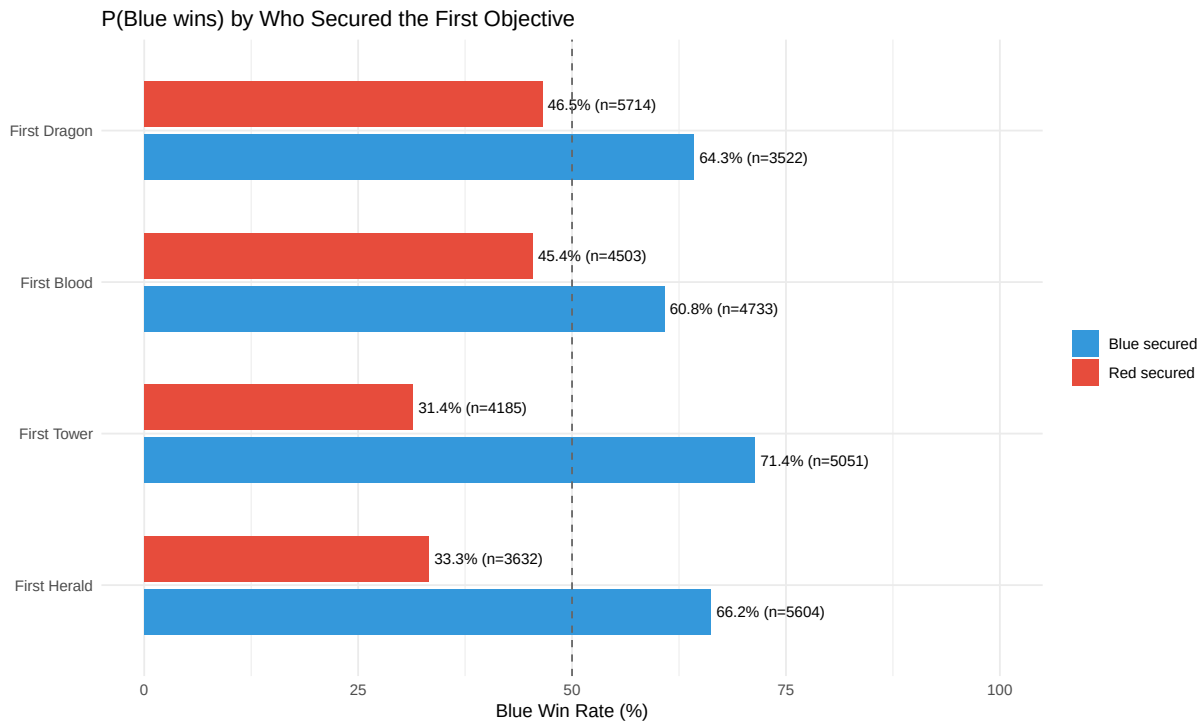
```

```

        n          = c(n_blue, n_red),
        chi2        = chi$statistic,
        p_value     = chi$p.value
    )
})

ggplot(
  obj_winrates,
  aes(x = reorder(objective, win_rate_blue), y = win_rate_blue, fill = secured_by)
) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_text(
    aes(label = sprintf("%.1f%% (n=%d)", win_rate_blue, n)),
    position = position_dodge(width = 0.7),
    hjust = -0.05, size = 3
  ) +
  coord_flip() +
  scale_fill_manual(values = c("Blue secured" = "#3498db", "Red secured" = "#e74c3c")) +
  scale_y_continuous(limits = c(0, 100)) +
  geom_hline(yintercept = 50, linetype = "dashed", color = "gray40") +
  labs(
    title = "P(Blue wins) by Who Secured the First Objective",
    x = NULL, y = "Blue Win Rate (%)", fill = NULL
  ) +
  theme_minimal()

```



```
obj_winrates %>%
  distinct(objective, chi2, p_value) %>%
  mutate(
    chi2 = round(chi2, 1),
    p_value = formatC(p_value, format = "e", digits = 2)
  ) %>%
  print()
```

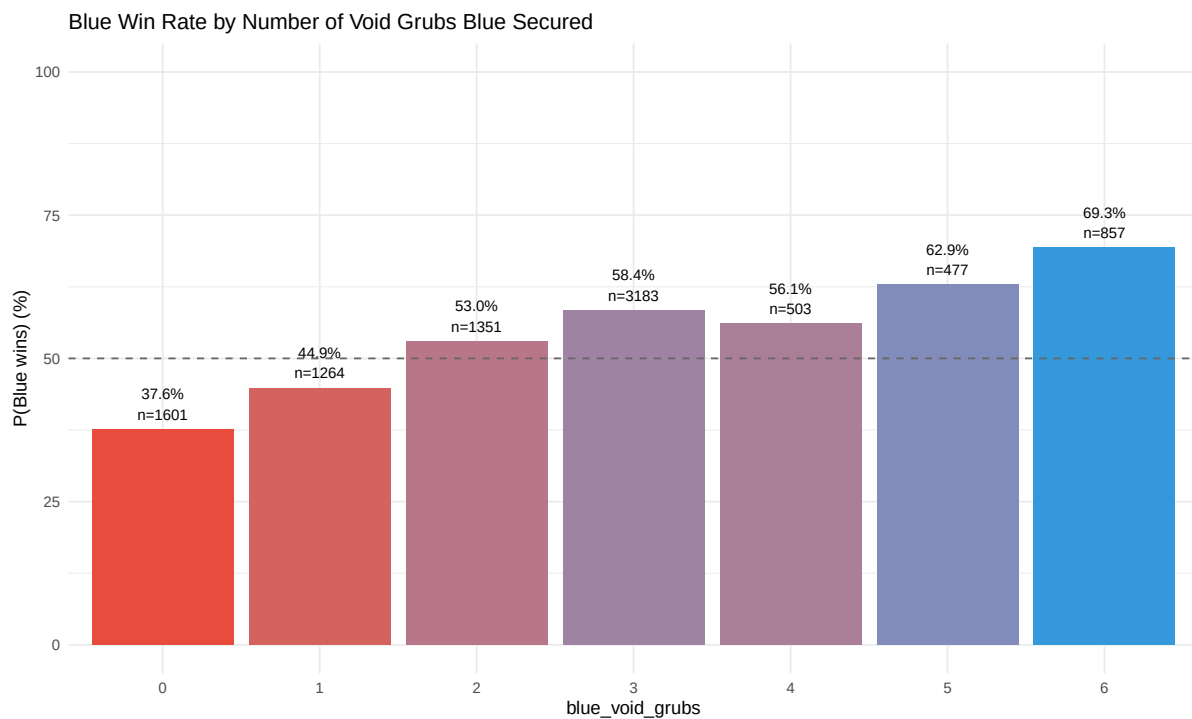
```
# A tibble: 4 x 3
  objective    chi2 p_value
  <chr>      <dbl> <chr>
1 First Blood  219. 1.45e-49
2 First Dragon 275. 1.11e-61
3 First Herald 960. 9.29e-211
4 First Tower 1467. 5.00e-321
```

When Blue secures first tower or first herald, Blue's win rate jumps well above 50%, and drops by a similar amount when Red gets them; first blood and first dragon move it much less. Every chi-square rejects independence at $p \ll 0.001$, so the direction is clear in all four cases - only the size of the effect differs. All four are worth keeping as features; we expect tower and herald to end up with the largest model coefficients in the LR fit and the highest importance scores in the tree-based models.

Void grubs

Void grubs are a 2024+ objective: they spawn at 5:00 and despawn at 13:45 on early-2025 patches (moved to 14:45 mid-season), so they are purely early-game. The pit holds up to 6 grubs split between the teams.

```
games %>%
  filter(!is.na(blue_void_grubs)) %>%
  group_by(blue_void_grubs) %>%
  summarise(
    win_rate = mean(blue_win) * 100,
    n        = n(),
    .groups  = "drop"
  ) %>%
  ggplot(aes(x = factor(blue_void_grubs), y = win_rate, fill = win_rate)) +
  geom_col() +
  geom_text(aes(label = sprintf("%.1f%%\nn=%d", win_rate, n)),
    vjust = -0.3, size = 3
  ) +
  scale_fill_gradient(low = "#e74c3c", high = "#3498db") +
  scale_y_continuous(limits = c(0, 100)) +
  geom_hline(yintercept = 50, linetype = "dashed", color = "gray40") +
  labs(
    title = "Blue Win Rate by Number of Void Grubs Blue Secured",
    x = "blue_void_grubs", y = "P(Blue wins) (%)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```



The relationship is essentially monotonic: 0 grubs sits well below 50% win rate, 6 grubs climbs to ~70%. That justifies the grub differential built later. Wobbles at intermediate counts reflect low sample sizes - split outcomes are rare because grub fights usually end with one side taking the remaining set.

Side advantage

Blue gets first pick in the draft and has slight map asymmetries. We report the share of Blue wins with a 95% CI and a chi-square test against the 50% null.

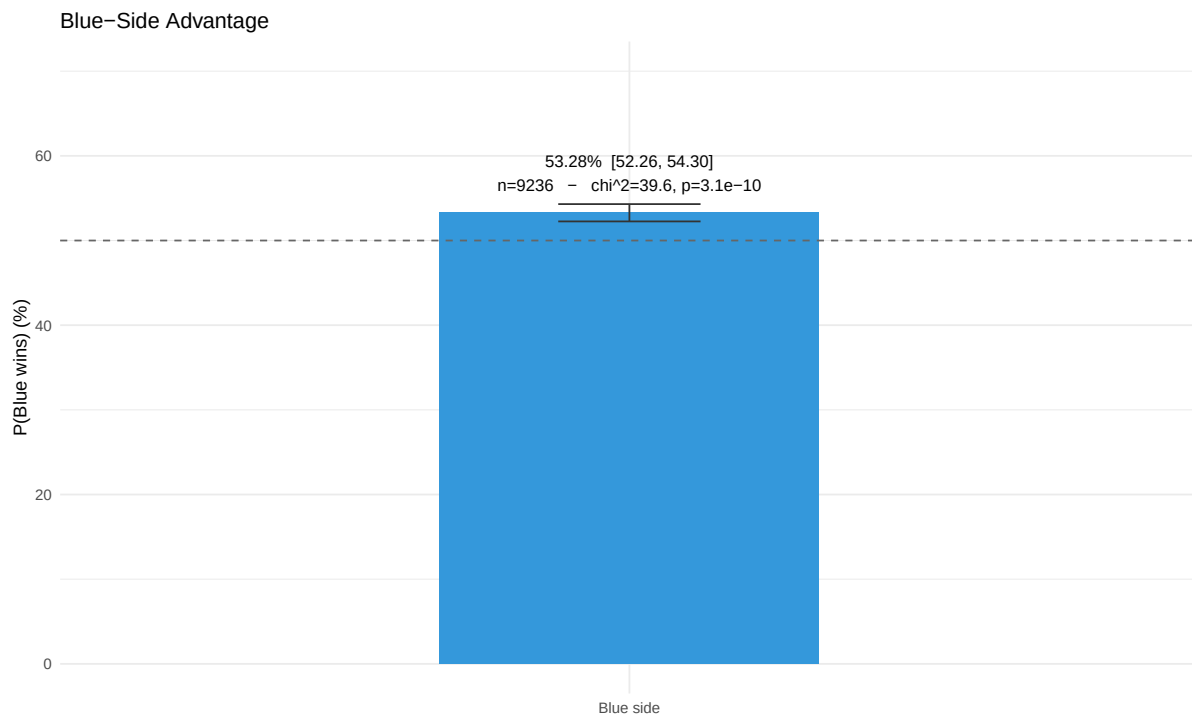
```
n_games <- nrow(games)
n_blue <- sum(games$blue_win)
side_pt <- prop.test(n_blue, n_games, p = 0.5)
blue_wr <- n_blue / n_games * 100
ci_low <- side_pt$conf.int[1] * 100
ci_high <- side_pt$conf.int[2] * 100

tibble(
  label = "Blue side",
  win_rate = blue_wr,
  ci_low = ci_low,
  ci_high = ci_high
)
```

```

) %>%
  ggplot(aes(x = label, y = win_rate)) +
  geom_col(width = 0.4, fill = "#3498db") +
  geom_errorbar(aes(ymin = ci_low, ymax = ci_high), width = 0.15, color = "gray20") +
  geom_text(
    aes(label = sprintf(
      "%.2f%%  [%.2f, %.2f]\nn=%d  -  chi^2=%.1f, p=%.2g",
      win_rate, ci_low, ci_high, n_games,
      side_pt$statistic, side_pt$p.value
    )),
    vjust = -0.6, size = 3.5
  ) +
  geom_hline(yintercept = 50, linetype = "dashed", color = "gray40") +
  scale_y_continuous(limits = c(0, 70)) +
  labs(
    title = "Blue-Side Advantage",
    x = NULL, y = "P(Blue wins) (%)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

```



Blue wins ~53% of games - a steady ~3 pp edge that is small but consistent across patches,

driven mainly by Blue picking first in champion select (which lets it lock the strongest contested champion of the patch before Red can) and by minor map and camera asymmetries that make objectives slightly easier to track and contest from Blue's side.

Role-specific gold differentials (H2 preview)

As a first look at H2, we correlate each Blue-side player's gold differential at 15 min with `blue_win`, broken out by role.

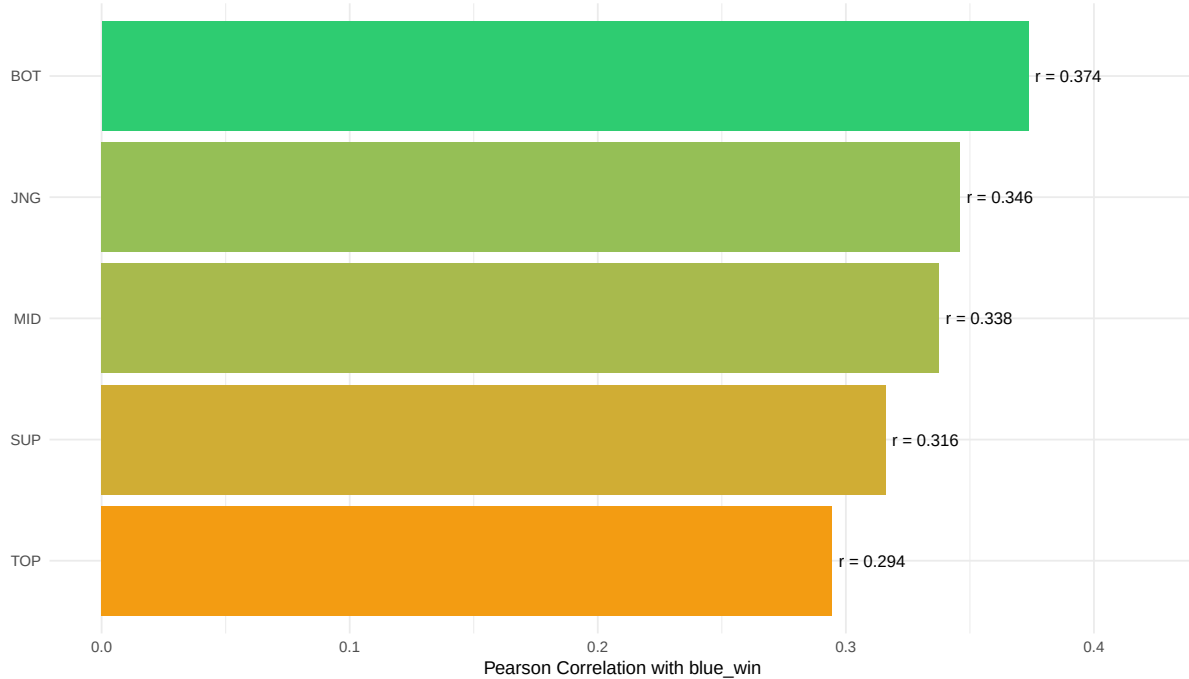
```
role_diffs_blue <- players %>%
  filter(
    position %in% c("top", "jng", "mid", "bot", "sup"),
    side == "Blue",
    !is.na(golddiffat15)
  ) %>%
  select(gameid, position, golddiffat15)

role_cors <- role_diffs_blue %>%
  inner_join(games %>% select(gameid, blue_win), by = "gameid") %>%
  group_by(position) %>%
  summarise(
    correlation      = cor(golddiffat15, blue_win),
    mean_diff_win    = mean(golddiffat15[blue_win == 1]),
    mean_diff_loss    = mean(golddiffat15[blue_win == 0]),
    .groups          = "drop"
  ) %>%
  mutate(position = toupper(position))

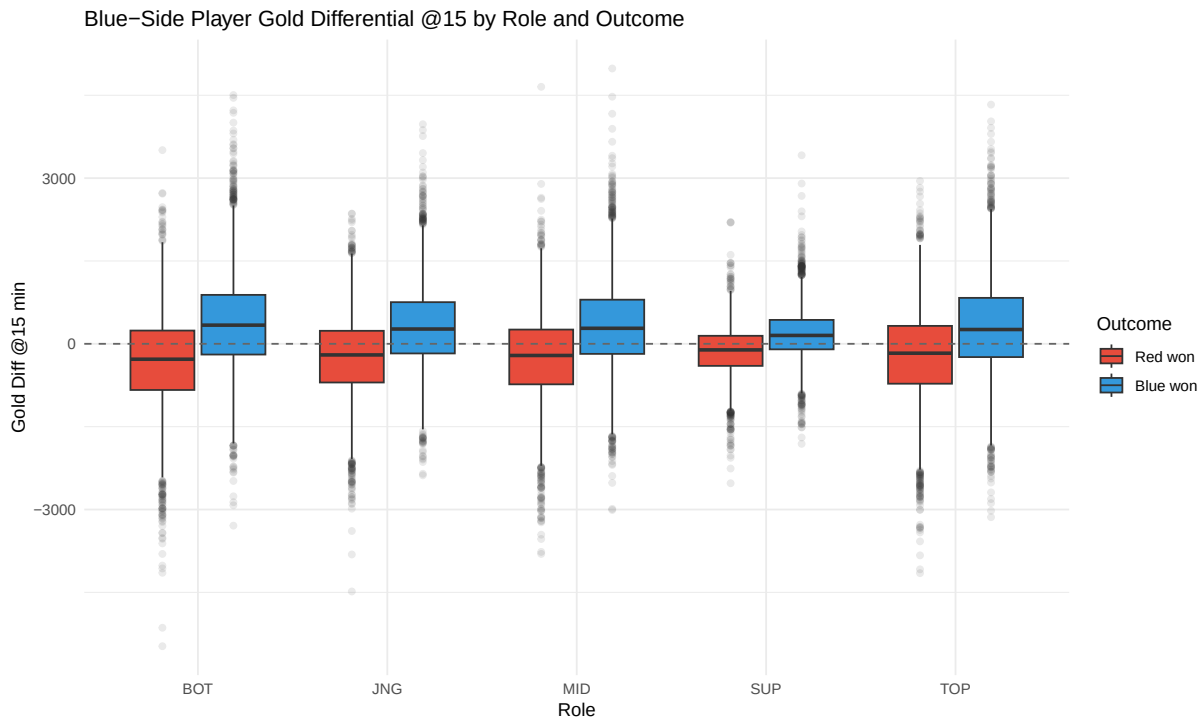
ggplot(role_cors, aes(
  x = reorder(position, correlation),
  y = correlation, fill = correlation
)) +
  geom_col() +
  geom_text(aes(label = sprintf("r = %.3f", correlation)),
    hjust = -0.1, size = 3.5
  ) +
  coord_flip() +
  scale_y_continuous(limits = c(0, 0.42)) +
  scale_fill_gradient(low = "#f39c12", high = "#2ecc71") +
  labs(
    title = "Correlation: Role Gold Diff @15 (Blue side) vs blue_win",
    x = NULL, y = "Pearson Correlation with blue_win"
```

```
) +
  theme_minimal() +
  theme(legend.position = "none")
```

Correlation: Role Gold Diff @15 (Blue side) vs blue_win



```
role_diffs_blue %>%
  inner_join(games %>% select(gameid, blue_win), by = "gameid") %>%
  mutate(
    blue_win = factor(blue_win, levels = c(0, 1), labels = c("Red won", "Blue won")),
    position = toupper(position)
  ) %>%
  ggplot(aes(x = position, y = golddiffat15, fill = blue_win)) +
  geom_boxplot(outlier.alpha = 0.1) +
  scale_fill_manual(values = c("Red won" = "#e74c3c", "Blue won" = "#3498db")) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  labs(
    title = "Blue-Side Player Gold Differential @15 by Role and Outcome",
    x = "Role", y = "Gold Diff @15 min", fill = "Outcome"
  ) +
  theme_minimal()
```



Bot lane (ADC) has the highest correlation with `blue_win` - early evidence for H2. Inter-role gaps are small, though, so the full H2 test decides whether the lead is real or noise. The boxplot shows clear win/loss separation for every role; bot lane simply has the widest gap.

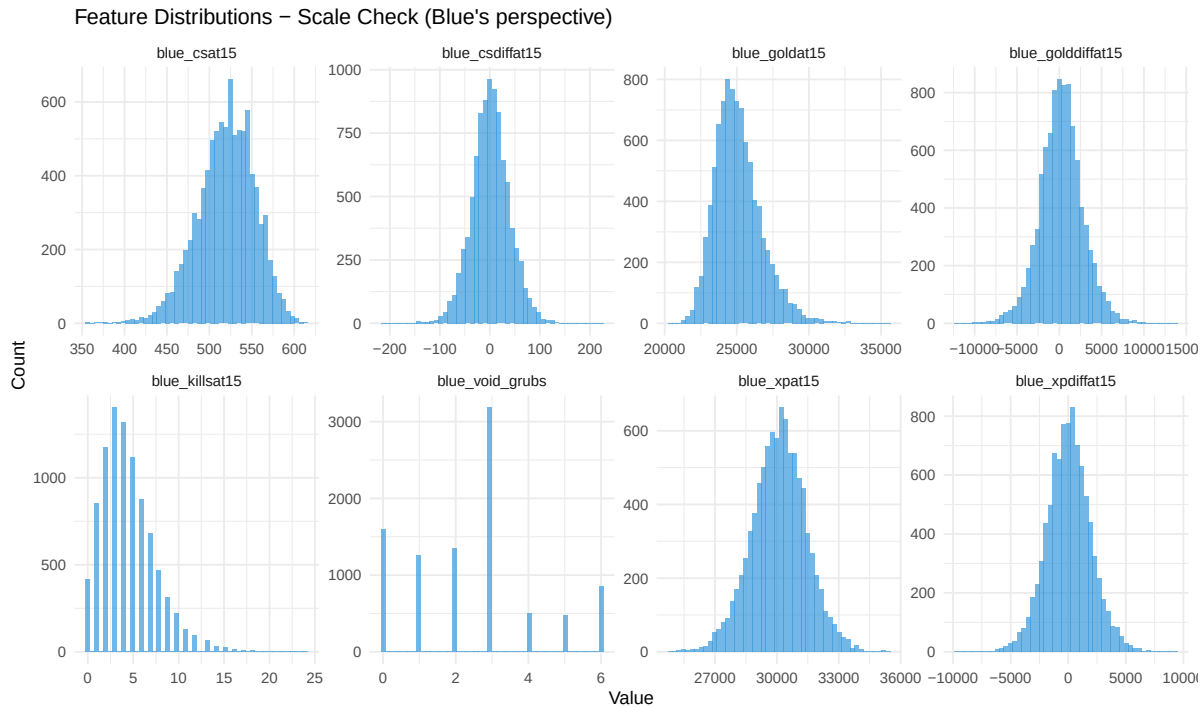
Feature distributions and scaling check

KNN and Logistic Regression are scale-sensitive; tree-based models and Naive Bayes are not. We check per-feature ranges below to decide whether standardisation is needed.

```
scale_check <- games %>%
  select(
    blue_goldat15, blue_xpat15, blue_csat15, blue_killsat15,
    blue_golddiffat15, blue_xpdiffat15, blue_csdiffat15,
    blue_void_grubs
  ) %>%
  drop_na() %>%
  pivot_longer(everything(), names_to = "feature", values_to = "value")

ggplot(scale_check, aes(x = value)) +
  geom_histogram(bins = 50, fill = "#3498db", alpha = 0.7) +
  facet_wrap(~feature, scales = "free", ncol = 4) +
```

```
labs(
  title = "Feature Distributions - Scale Check (Blue's perspective)",
  x = "Value", y = "Count"
) +
theme_minimal()
```



Scales differ wildly: `blue_goldat15` runs from ~20,000 to ~36,000, `blue_void_grubs` lives on 0-6, and `blue_killsat15` on 0-24. Without standardisation, KNN distances would be swamped by gold (1000x larger than the rest) and effectively ignore kills and grubs, and LR coefficients would not be comparable across features. We z-score standardise before fitting KNN and Logistic Regression; tree-based models (RF, CART) and Naive Bayes do not need it.

EDA Summary

1. **Target is mildly imbalanced** (~53/47 in Blue's favour). Significant but small - accuracy is still a valid metric and no resampling is needed.
2. **Gold diff @15 is the strongest single predictor.** The naive "gold-leader wins" rule already hits ~73% accuracy - the baseline to beat.
3. **Residual multicollinearity** remains between raw stats and their differentials and between Blue/Red paired counts. Linear models need regularisation even after dropping strict mirrors.

4. **First tower** and **first herald** swing Blue's win rate sharply (chi-square $p \ll 0.001$); they are the highest-impact binary objectives.
5. **First blood** and **first dragon** are statistically significant but carry weaker signal than tower or herald - both shift Blue's WR by ~10-15 pp when Blue secures, but only ~3-5 pp when Red does. The asymmetry comes from the herald-dragon trade pattern: Blue takes herald in 61% of games while Red free-takes dragon in 62%, so games where Red secured first dragon are biased toward games where Blue was already ahead.
6. **Void grubs** show a near-monotonic gradient in Blue's win rate from 0 to 6 grubs, which justifies engineering a grub differential.
7. **Blue-side advantage** is significant (~53% win rate, 95% CI excludes 50%) and absorbed into the base rate of `blue_win`.
8. **Feature scales differ by ~1000x**, so KNN and Logistic Regression need standardisation.
9. **Bot lane** has the highest role-specific correlation with `blue_win`, supporting the H2 analysis.
10. **Leads tend to grow** between @10 and @15.
11. **Turret plate differential** is a strong feature that is purely early-game, since plates fall by 14:00. It rarely appears in the published @15 literature.
12. **Total dragon/herald counts** are full-game statistics - using them would leak the outcome and they must not enter the feature set.

Feature Engineering

The correlation analysis indicates that *gaps* between teams carry the signal, so we retain signed differentials throughout. To control multicollinearity, we keep one element of each redundant triple (`goldat15`, `opp_goldat15`, `golddiffat15`) - the differential - and drop the other two. The raw @10 snapshots are likewise omitted: any signal they carry beyond the @15 snapshot is marginal, and combining them with the @15 differentials inflates VIF without raising held-out AUC. We supplement this core with two features that rarely appear in the published @15 literature: a **turret plate differential** (plates fall by 14:00, so the metric is purely early-game; $r = 0.409$ with `blue_win`) and a **rolling team win rate differential**, computed from prior games only.

We also evaluated several higher-order constructs - per-window momentum (`diff@15 - diff@10` for gold, XP, and CS) and per-team kill-pressure counts in the 10-15 minute window. Each was a linear combination of features already present in the matrix and, in pilot fits, added severe multicollinearity without contributing any independent predictive signal, so all of them were dropped before the modelling pipeline. We retained two non-linear ratio features that do *not* introduce the same redundancy: `gold_efficiency_15 = goldat15 / csat15` (gold-per-CS, a draft and macro proxy) and `kda_15 = (kills + assists) / max(deaths, 1)` (combat efficiency); both remain weakly correlated with the rest of the matrix.

Rolling team win rate

For each team, we compute a rolling win rate over all of its *prior* games, ordered by date, with first-appearance teams defaulted to 0.5. The per-side values are pivoted into one Blue and one Red column per game, and `winrate_diff = blue - red` captures pre-game asymmetry without leaking information from the current result

```
team_wr <- teams %>%
  arrange(date, gameid) %>%
  group_by(teamid) %>%
  mutate(
    games_before = row_number() - 1,
    wins_before = cumsum(result) - result,
    team_winrate = ifelse(games_before > 0, wins_before / games_before, 0.5)
  ) %>%
  ungroup() %>%
  mutate(side = tolower(side)) %>%
  select(gameid, side, team_winrate, games_before) %>%
  pivot_wider(
    id_cols = gameid,
    names_from = side,
    values_from = c(team_winrate, games_before),
    names_glue = "{side}_{.value}"
  )

games_wr <- games %>%
  left_join(team_wr, by = "gameid") %>%
  mutate(winrate_diff = blue_team_winrate - red_team_winrate)

cat(sprintf(
  "Median prior games: %.0f Blue / %.0f Red\nFirst-appearance teams (defaulted to 0.5): %d\n",
  median(games_wr$blue_games_before),
  median(games_wr$red_games_before),
  sum(games_wr$blue_games_before == 0),
  sum(games_wr$red_games_before == 0),
  cor(games_wr$winrate_diff, games_wr$blue_win, use = "complete.obs")
))
```

Median prior games: 31 Blue / 30 Red

First-appearance teams (defaulted to 0.5): 192 Blue, 190 Red

winrate_diff correlation with blue_win: 0.266

Select and engineer features

Every feature below is expressed from the Blue perspective: signed differentials in which a positive value indicates that Blue is ahead, together with four binary flags marking whether Blue secured each first objective. Since the perspective is fixed, the `blue_` prefix is implicit throughout the modelling stage and is omitted from column names. Each row represents one full match, so the structural Blue-side advantage is absorbed into the base rate of `blue_win` and serves as the intercept of the model.

```
games_model <- games_wr %>%
  transmute(
    gameid,
    blue_win,

    # Snapshot diffs at 15 (primary signal, Blue's perspective)
    golddiffat15 = blue_golddiffat15,
    xpdiffat15 = blue_xpdiffat15,
    csdiffat15 = blue_csdiffat15,

    # Combat diffs at 15 (death_diff omitted: near-perfect mirror of -kill_diff, see VIF)
    kill_diff_15 = blue_killsat15 - red_killsat15,
    assist_diff_15 = blue_assistsat15 - red_assistsat15,

    # Non-linear early-game ratios (low VIF against the diff core)
    gold_efficiency_15 = blue_goldat15 / pmax(blue_csat15, 1),
    kda_15 = (blue_killsat15 + blue_assistsat15) / pmax(blue_deathsat15, 1),

    # Objective differentials
    plate_diff = blue_turretplates - red_turretplates,
    grub_diff = blue_void_grubs - red_void_grubs,

    # Pre-game strength
    winrate_diff,

    # First-objective flags (1 if Blue secured)
    firstblood = blue_firstblood,
    firstdragon = blue_firstdragon,
    firsttherald = blue_firsttherald,
    firsttower = blue_firsttower
  ) %>%
  drop_na()

cat(sprintf(
```

```

    "Final feature set: %d features (excluding gameid and blue_win)\nFeatures: %s",
    ncol(games_model) - 2,
    paste(setdiff(names(games_model), c("gameid", "blue_win")), collapse = ", ")
  ))

```

Final feature set: 14 features (excluding gameid and blue_win)

Features: golddiffat15, xpdiffat15, csdiffat15, kill_diff_15, assist_diff_15, gold_efficiency

H2: Role-specific differentials

For H2 we add per-role early-game differentials. The signed `golddiffat15` of each Blue-side player is, by construction, `blue_player_gold - red_lane_opponent_gold`; retaining only the Blue rows and pivoting wide therefore yields a Blue-perspective signed differential per role and per metric (e.g., `bot_golddiffat15`, `mid_xpdiffat10`). The resulting table is inner-joined back to `games_model`.

```

role_diffs <- players %>%
  filter(
    position %in% c("top", "jng", "mid", "bot", "sup"),
    side == "Blue"
  ) %>%
  select(
    gameid, position,
    golddiffat10, golddiffat15,
    xpdiffat10, xpdiffat15,
    csdiffat10, csdiffat15
  ) %>%
  drop_na() %>%
  pivot_wider(
    id_cols = gameid,
    names_from = position,
    values_from = c(
      golddiffat10, golddiffat15,
      xpdiffat10, xpdiffat15,
      csdiffat10, csdiffat15
    ),
    names_glue = "{position}_{.value}"
  )

games_h2 <- games_model %>%
  inner_join(role_diffs, by = "gameid") %>%

```

```

drop_na()

cat(sprintf(
  "Role features added: %d\nH2 rows after join + drop_na: %d (removed %d from H1 set, %.1f",
  ncol(role_diffs) - 1,
  nrow(games_h2),
  nrow(games_model) - nrow(games_h2),
  (nrow(games_model) - nrow(games_h2)) / nrow(games_model) * 100,
  ncol(games_h2) - 2
))

```

Role features added: 30

H2 rows after join + drop_na: 9236 (removed 0 from H1 set, 0.0%)

H2 total features: 44

Train/Test Split

Each row in `games_model` represents one full match, so a plain 80/20 row split is sufficient. We sample game IDs once with `set.seed(42)` and reuse the same partition for H2, ensuring that both hypotheses are evaluated on the same held-out games.

```

set.seed(42)

train_ids <- sample(games_model$gameid, size = floor(0.8 * nrow(games_model)))

train_h1 <- games_model %>%
  filter(gameid %in% train_ids) %>%
  select(-gameid)
test_h1 <- games_model %>%
  filter(!gameid %in% train_ids) %>%
  select(-gameid)

train_h2 <- games_h2 %>%
  filter(gameid %in% train_ids) %>%
  select(-gameid)
test_h2 <- games_h2 %>%
  filter(!gameid %in% train_ids) %>%
  select(-gameid)

cat(sprintf(

```

```

"Games: %d train, %d test\nH1 - Train: %d rows, Test: %d rows\nH2 - Train: %d rows, Test: %d rows\n
length(train_ids), nrow(games_model) - length(train_ids),
nrow(train_h1), nrow(test_h1), nrow(train_h2), nrow(test_h2)
))

```

```

Games: 7388 train, 1848 test
H1 - Train: 7388 rows, Test: 1848 rows
H2 - Train: 7388 rows, Test: 1848 rows

```

Scenario 1: Model Comparison

Six classifiers on the same H1 feature set, judged on accuracy, AUC-ROC, precision, recall, and F1. Hyperparameters are tuned with shared 5-fold CV (`metric = "ROC"`); the test set is held out for the final report.

Category	Model	Role in the comparison
Statistical/ML	Logistic Regression	Interpretable linear baseline; standardised coefficients
Statistical/ML	Random Forest	Non-linear ensemble; agreement check on what matters
Statistical/ML	Naive Bayes	Bayes-rule baseline assuming feature independence
Partitioning	KNN	Distance-based; sensitive to scale and correlated features
Partitioning	Decision Tree (CART)	Single-tree analogue of RF for the consensus comparison
Statistical/ML	XGBoost	Gradient-boosted trees; benchmark used in modern LoL @15 prediction papers

The bar to beat is the EDA baseline: **72.9 %** from “Blue ahead in gold @15 wins”. Published @15 LoL studies report 73-78 %, so a 75 %-band result is consistent with the literature.

Preprocessing

KNN and Logistic Regression need standardised inputs. We fit the scaler on the training set and apply the same transformation to the test set. Tree-based models (RF, CART, XGBoost) and Naive Bayes do not need scaling, but reusing the scaled matrix is harmless.

```

X_train <- train_h1 %>% select(-blue_win)
y_train <- train_h1$blue_win
X_test <- test_h1 %>% select(-blue_win)
y_test <- test_h1$blue_win

preproc <- preProcess(X_train, method = c("center", "scale"))
X_train_scaled <- predict(preproc, X_train)
X_test_scaled <- predict(preproc, X_test)

cat(sprintf(
  "Training features: %d   Training rows: %d   Test rows: %d\n",
  ncol(X_train), nrow(X_train), nrow(X_test)
))

```

Training features: 14 Training rows: 7388 Test rows: 1848

Multicollinearity check (VIF)

VIF (Variance Inflation Factor) tells us how much each feature overlaps with the rest of the matrix. A high VIF means the feature can be largely reproduced from the others; values above 10 are conventionally treated as a sign of severe collinearity. We compute it on the training set only - VIF is a fit-time diagnostic and must not be informed by held-out data.

```

vif_df <- tibble(
  feature = names(X_train_scaled),
  VIF      = diag(solve(cor(X_train_scaled)))
) %>% arrange(desc(VIF))

knitr::kable(vif_df,
  digits = 2,
  caption = "VIF on H1 training features (diagnostic only - no features dropped in S1)."
)

```

Table 1: VIF on H1 training features (diagnostic only - no features dropped in S1).

feature	VIF
golddiffat15	22.90
kill_diff_15	12.57
assist_diff_15	5.24
xpdiffat15	5.17

feature	VIF
csdiffat15	4.58
plate_diff	3.54
kda_15	2.37
firsttower	1.63
grub_diff	1.32
firstherald	1.24
gold_efficiency_15	1.24
firstblood	1.22
winrate_diff	1.12
firstdragon	1.10

Two features exceed the threshold: `golddiffat15` (~ 23) and `kill_diff_15` (~ 13). Gold and kills are tightly coupled in League - every kill pays out gold and pushes the lane further ahead - so most of what `kill_diff_15` carries is already in `golddiffat15`. `xpdiffat15`, `csdiffat15`, and `assist_diff_15` sit in the 4-5 range, reflecting the same lane-economy signal at a weaker level. The binary objective flags, `winrate_diff`, and the two ratio features (`gold_efficiency_15`, `kda_15`) all stay below VIF 4 and are effectively independent.

Three of our six models are affected by this. Logistic Regression coefficients become unstable when predictors overlap, Naive Bayes assumes feature independence outright, and KNN distances double-count correlated dimensions. Random Forest, CART, and XGBoost split one feature at a time and are unaffected.

! Why we do not drop high-VIF features here

The standard rule of thumb is to drop features with $VIF > 10$. The two features above the threshold here are `golddiffat15` - the strongest single predictor in the problem - and `kill_diff_15`, its closest economic partner. Their VIF is high because the other features overlap with them, not because they themselves carry no signal: VIF measures redundancy, not predictive value. We keep every surviving feature in S1, treat VIF as a diagnostic only, and let the regularised methods in S3 (LASSO and Elastic Net) handle the residual redundancy.

CV setup

Shared `trainControl` so every model is tuned on the same folds.

```
ctrl <- trainControl(
  method = "cv",
```

```

    number = 5,
    classProbs = TRUE,
    summaryFunction = twoClassSummary,
    savePredictions = "final",
    verboseIter = FALSE
)

y_train_fct <- factor(y_train, levels = c(0, 1), labels = c("Loss", "Win"))
y_test_fct <- factor(y_test, levels = c(0, 1), labels = c("Loss", "Win"))

```

Logistic Regression

Linear baseline trained on the standardised feature matrix so coefficients are directly comparable. With Win as the last factor level, `glm` models log-odds of Win - a positive coefficient indicates the feature pushes towards Blue winning, a negative one pushes against.

```

t0 <- Sys.time()

lr_model <- train(
  x = X_train_scaled,
  y = y_train_fct,
  method = "glm",
  family = "binomial",
  trControl = ctrl,
  metric = "ROC"
)

lr_pred <- predict(lr_model, X_test_scaled)
lr_prob <- predict(lr_model, X_test_scaled, type = "prob")
lr_cm <- confusionMatrix(lr_pred, y_test_fct, positive = "Win")

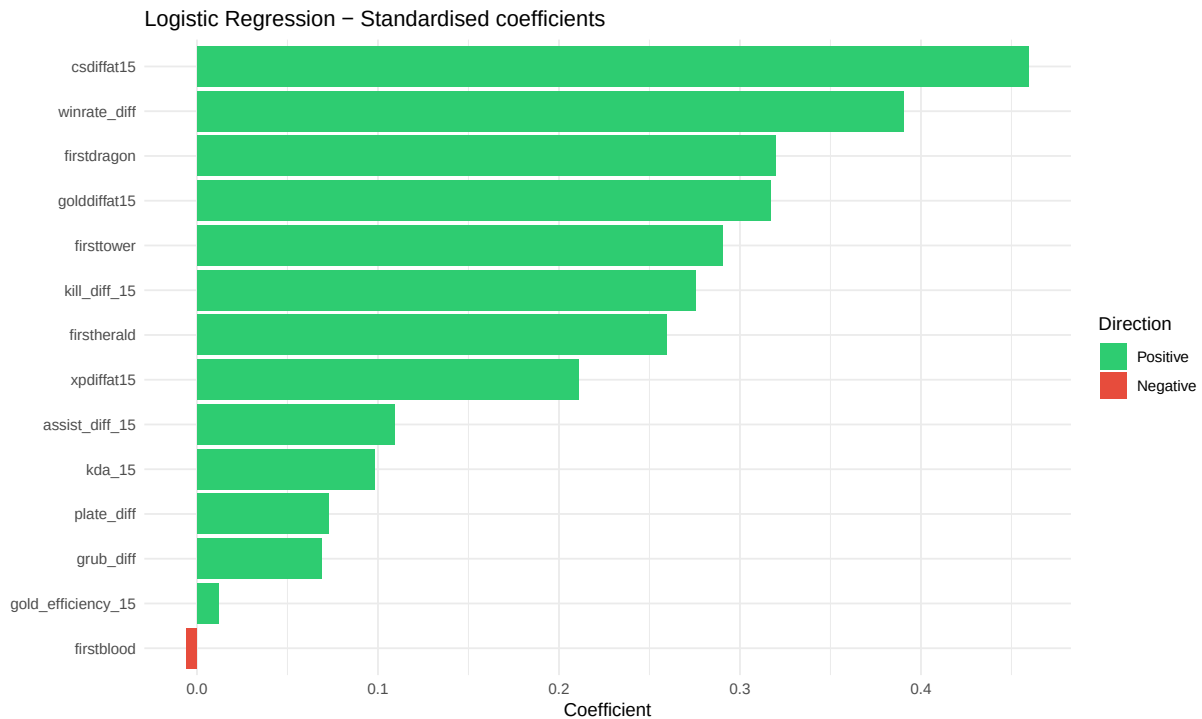
cat(sprintf(
  "\nLogistic Regression Results (trained in %.1fs):\nAccuracy: %.1f%\nSensitivity (Win r
  as.numeric(Sys.time() - t0, units = "secs"),
  lr_cm$overall["Accuracy"] * 100,
  lr_cm$byClass["Sensitivity"] * 100,
  lr_cm$byClass["Specificity"] * 100,
  max(lr_model$results$ROC)
))

```


Logistic Regression Results (trained in 1.8s):
Accuracy: 75.5%
Sensitivity (Win recall): 79.1%
Specificity (Loss recall): 71.2%
CV AUC-ROC: 0.843

```
lr_coefs <- coef(lr_model$finalModel) %>%
  as.data.frame() %>%
  rownames_to_column("feature") %>%
  rename(coefficient = 2) %>%
  filter(feature != "(Intercept)") %>%
  arrange(desc(abs(coefficient)))

ggplot(
  lr_coefs,
  aes(
    x = reorder(feature, abs(coefficient)),
    y = coefficient,
    fill = coefficient > 0
  )
) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(
    values = c("TRUE" = "#2ecc71", "FALSE" = "#e74c3c"),
    breaks = c("TRUE", "FALSE"),
    labels = c("Positive", "Negative")
  ) +
  labs(
    title = "Logistic Regression - Standardised coefficients",
    x = NULL, y = "Coefficient", fill = "Direction"
  ) +
  theme_minimal()
```



All non-trivial coefficients are positive: resource diffs at @15 (`golddiffat15`, `xpdiffat15`, `csdiffat15`), `kill_diff_15`, `winrate_diff`, and the first-objective flags all push toward Blue winning, exactly as the EDA predicted. `firstblood` is the only feature with a negative coefficient and its magnitude is essentially zero - consistent with the EDA showing it as the weakest of the four objective flags. There are no sign flips on the high-VIF pair, which is what we'd expect: the residual collinearity is moderate enough that LR can still attribute coefficients to roughly the right direction, even if SEs on `golddiffat15` and `kill_diff_15` are mildly inflated. S3's regularised methods will tighten this further by zeroing out the marginal contributors.

Random Forest

Non-linear ensemble; immune to the residual multicollinearity in the LR coefficient table. We tune `mtry` over {3, 5, 7, 10, 14} with 500 trees fixed.

```
t0 <- Sys.time()

rf_model <- train(
  x = X_train,
  y = y_train_fct,
  method = "rf",
```

```

    trControl = ctrl,
    metric = "ROC",
    tuneGrid = expand.grid(mtry = c(3, 5, 7, 10, 14)),
    ntree = 500
)

rf_pred <- predict(rf_model, X_test)
rf_prob <- predict(rf_model, X_test, type = "prob")
rf_cm <- confusionMatrix(rf_pred, y_test_fct, positive = "Win")

cat(sprintf(
  "\nRandom Forest Results (trained in %.1fs):\nAccuracy: %.1f%%\nSensitivity: %.1f%%\nSpe
as.numeric(Sys.time() - t0, units = "secs"),
  rf_cm$overall["Accuracy"] * 100,
  rf_cm$byClass["Sensitivity"] * 100,
  rf_cm$byClass["Specificity"] * 100,
  rf_model$bestTune$mtry,
  max(rf_model$results$ROC)
))

```

```

Random Forest Results (trained in 15.1s):
Accuracy: 74.1%
Sensitivity: 78.0%
Specificity: 69.3%
Best mtry: 3
CV AUC-ROC: 0.834

```

```

# Show CV results across mtry values
print(rf_model$results %>% select(mtry, ROC, Sens, Spec))

```

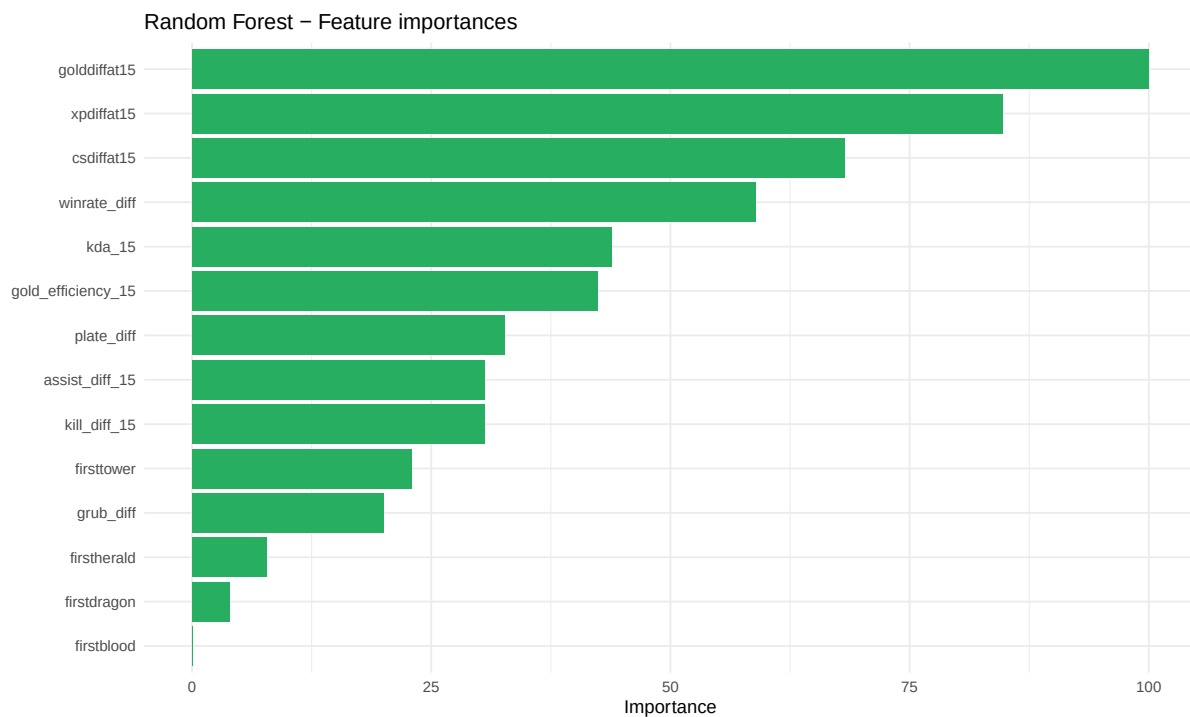
	mtry	ROC	Sens	Spec
1	3	0.8335167	0.7311169	0.7734762
2	5	0.8314114	0.7293936	0.7647730
3	7	0.8311798	0.7259429	0.7668213
4	10	0.8296019	0.7250833	0.7650265
5	14	0.8279736	0.7222105	0.7680991

```

rf_imp <- varImp(rf_model)$importance %>%
  rownames_to_column("feature") %>%
  arrange(desc(Overall))

```

```
ggplot(
  rf_imp,
  aes(x = reorder(feature, Overall), y = Overall)
) +
  geom_col(fill = "#27ae60") +
  coord_flip() +
  labs(
    title = "Random Forest - Feature importances",
    x = NULL, y = "Importance"
  ) +
  theme_minimal()
```



Naive Bayes

The independence assumption is directly violated by the VIF result above; we still fit it to see how much accuracy that costs. We tune `usekernel` (Gaussian vs KDE) and the KDE bandwidth multiplier `adjust`.

```
t0 <- Sys.time()
```

```

nb_grid <- expand.grid(
  usekernel = c(FALSE, TRUE),
  laplace    = 0,
  adjust     = c(0.5, 1.0, 1.5)
)

nb_model <- train(
  x = X_train,
  y = y_train_fct,
  method = "naive_bayes",
  trControl = ctrl,
  metric = "ROC",
  tuneGrid = nb_grid
)

nb_pred <- predict(nb_model, X_test)
nb_prob <- predict(nb_model, X_test, type = "prob")
nb_cm <- confusionMatrix(nb_pred, y_test_fct, positive = "Win")

cat(sprintf(
  "\nNaive Bayes Results (trained in %.1fs):\nAccuracy: %.1f%%\nSensitivity: %.1f%%\nSpeci:
  as.numeric(Sys.time() - t0, units = "secs"),
  nb_cm$overall["Accuracy"] * 100,
  nb_cm$byClass["Sensitivity"] * 100,
  nb_cm$byClass["Specificity"] * 100,
  as.character(nb_model$bestTune$usekernel),
  nb_model$bestTune$adjust,
  max(nb_model$results$ROC)
))

```

Naive Bayes Results (trained in 0.4s):
 Accuracy: 74.9%
 Sensitivity: 76.4%
 Specificity: 73.0%
 Best: usekernel=TRUE, adjust=1.5
 CV AUC-ROC: 0.831

```

print(nb_model$results %>%
  arrange(desc(ROC)) %>%
  select(usekernel, adjust, ROC, Sens, Spec))

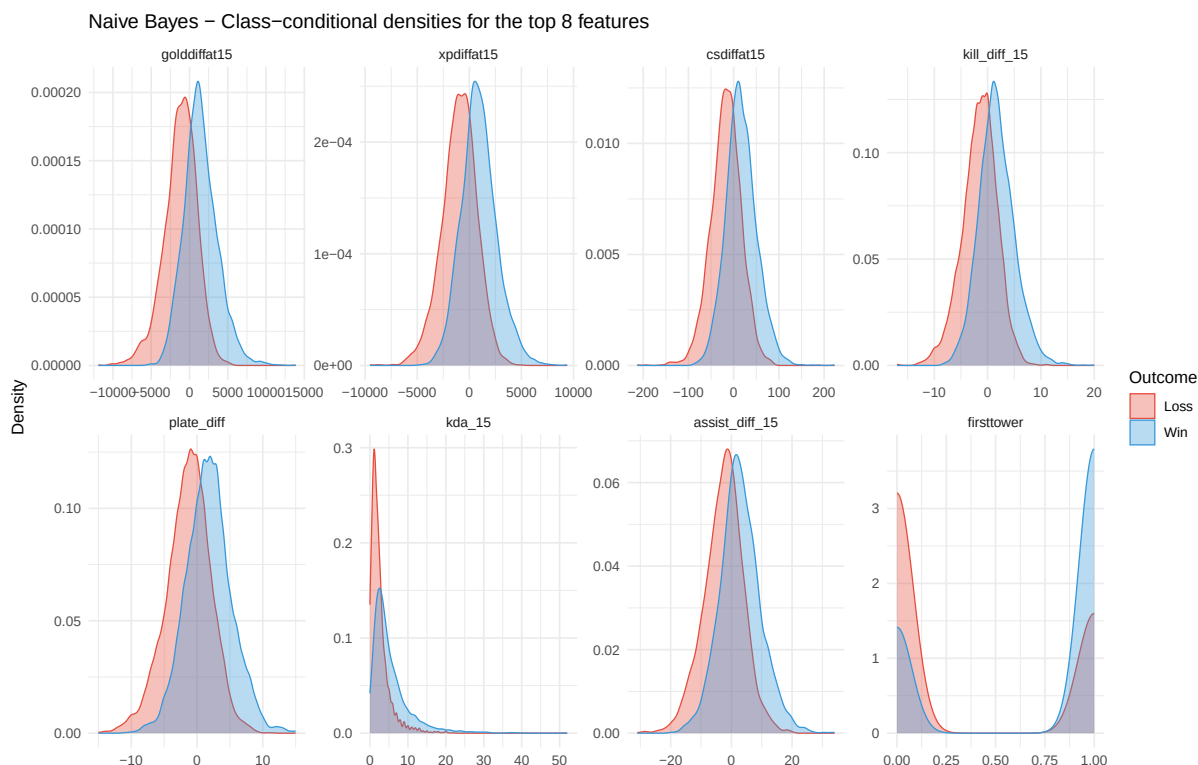
```

	usekernel	adjust	ROC	Sens	Spec
1	TRUE	1.5	0.8309412	0.7391551	0.7611993
2	TRUE	1.0	0.8307222	0.7391543	0.7606872
3	TRUE	0.5	0.8303214	0.7365705	0.7606855
4	FALSE	0.5	0.8293397	0.7759239	0.7343258
5	FALSE	1.0	0.8293397	0.7759239	0.7343258
6	FALSE	1.5	0.8293397	0.7759239	0.7343258

```
nb_imp <- tibble(
  feature = names(X_train),
  auc_raw = sapply(X_train, function(col) {
    as.numeric(pROC::auc(pROC::roc(y_train_fct, as.numeric(col),
      levels = c("Loss", "Win"), quiet = TRUE
    )))
  })
) %>%
  mutate(Overall = abs(auc_raw - 0.5) * 200) %>%
  select(feature, Overall) %>%
  arrange(desc(Overall))

# Class-conditional densities for the eight strongest features (NB internals).
top_nb_features <- nb_imp$feature[1:8]

X_train %>%
  select(all_of(top_nb_features)) %>%
  mutate(outcome = y_train_fct) %>%
  pivot_longer(-outcome, names_to = "feature", values_to = "value") %>%
  mutate(feature = factor(feature, levels = top_nb_features)) %>%
  ggplot(aes(x = value, fill = outcome, colour = outcome)) +
  geom_density(alpha = 0.35, linewidth = 0.4) +
  facet_wrap(~feature, scales = "free", ncol = 4) +
  scale_fill_manual(values = c("Loss" = "#e74c3c", "Win" = "#3498db")) +
  scale_colour_manual(values = c("Loss" = "#e74c3c", "Win" = "#3498db")) +
  labs(
    title = "Naive Bayes - Class-conditional densities for the top 8 features",
    x = NULL, y = "Density", fill = "Outcome", colour = "Outcome"
  ) +
  theme_minimal()
```



The four resource differentials carry the widest class gap (golddiffat15 1.22 pooled sd, xpdiffat15 1.12, csdiffat15 1.00, kill_diff_15 0.98) and are roughly Gaussian, so the default likelihood fits them well. kda_15 is right-skewed, plate_diff is integer-discretised, and firsttower is binary - none Gaussian, which is why CV picks `usekernel = TRUE`. Every Win density sits to the right of its Loss density with substantial overlap around zero, so NB needs to combine many features to reach a sharp posterior - the source of its well-known push-to-extremes calibration behaviour.

KNN

Distance-based; runs on the standardised matrix. Multicollinearity inflates correlated dimensions in the Euclidean metric, so we expect KNN to lose ground to RF/CART here.

```
t0 <- Sys.time()

# k grid log-spaced from 5 to ~4*sqrt(n), forced odd to avoid 50/50 ties.
n_train <- nrow(X_train_scaled)
k_theo <- round(sqrt(n_train))
k_grid <- unique(round(exp(seq(log(5), log(k_theo * 4), length.out = 12))))
k_grid <- ifelse(k_grid %% 2 == 0, k_grid + 1, k_grid)
```

```

knn_model <- train(
  x = X_train_scaled,
  y = y_train_fct,
  method = "knn",
  trControl = ctrl,
  metric = "ROC",
  tuneGrid = expand.grid(k = k_grid)
)

knn_pred <- predict(knn_model, X_test_scaled)
knn_prob <- predict(knn_model, X_test_scaled, type = "prob")
knn_cm <- confusionMatrix(knn_pred, y_test_fct, positive = "Win")

cat(sprintf(
  "\nKNN Results (%.1fs):\nAccuracy: %.1f%%\nSensitivity (Win recall): %.1f%%\nSpecificity
  as.numeric(Sys.time() - t0, units = "secs"),
  knn_cm$overall["Accuracy"] * 100,
  knn_cm$byClass["Sensitivity"] * 100,
  knn_cm$byClass["Specificity"] * 100,
  knn_model$bestTune$k,
  max(knn_model$results$ROC)
))

```

```

KNN Results (4.0s):
Accuracy: 75.2%
Sensitivity (Win recall): 79.8%
Specificity (Loss recall): 69.5%
Best k: 345
CV AUC-ROC: 0.838

```

Decision Tree CART

Single-tree analogue of RF; like RF, immune to multicollinearity. We tune the complexity parameter `cp` over `{1e-5, 5e-5, 1e-4, 5e-4, 1e-3}`.

```

t0 <- Sys.time()

# cp (complexity parameter) controls tree depth - tuned via 5-fold CV
cart_model <- train(

```



```

    x = X_train,
    y = y_train_fct,
    method = "rpart",
    trControl = ctrl,
    metric = "ROC",
    tuneGrid = expand.grid(cp = c(0.00001, 0.00005, 0.0001, 0.0005, 0.001))
)

cart_pred <- predict(cart_model, X_test)
cart_prob <- predict(cart_model, X_test, type = "prob")
cart_cm <- confusionMatrix(cart_pred, y_test_fct, positive = "Win")

cat(sprintf(
  "\n\nCART Results (%.1fs):\nAccuracy: %.1f%%\nSensitivity (Win recall): %.1f%%\nSpecificity (Loss recall): %.1f%%\n\n",
  as.numeric(Sys.time() - t0, units = "secs"),
  cart_cm$overall["Accuracy"] * 100,
  cart_cm$byClass["Sensitivity"] * 100,
  cart_cm$byClass["Specificity"] * 100,
  cart_model$bestTune$cp,
  max(cart_model$results$ROC)
))

```

```

CART Results (0.2s):
Accuracy: 72.9%
Sensitivity (Win recall): 76.6%
Specificity (Loss recall): 68.5%
Best cp: 0.0010
CV AUC-ROC: 0.793

```

```

cart_imp <- varImp(cart_model)$importance %>%
  rownames_to_column("feature") %>%
  arrange(desc(Overall))

```

XGBoost

This model is used as the strongest non-linear baseline in the recent LoL prediction literature. Like RF and CART it is immune to multicollinearity and feature scaling. We tune **nrounds**, **max_depth**, and **eta** over a small grid; the remaining XGBoost hyperparameters are held at sensible defaults to keep the search tractable.

```

t0 <- Sys.time()

xgb_grid <- expand.grid(
  nrounds      = c(100, 300),
  max_depth    = c(4, 6),
  eta          = c(0.05, 0.1),
  gamma        = 0,
  colsample_bytree = 0.8,
  min_child_weight = 1,
  subsample    = 0.8
)

xgb_model <- train(
  x = X_train,
  y = y_train_fct,
  method = "xgbTree",
  trControl = ctrl,
  metric = "ROC",
  tuneGrid = xgb_grid,
  verbosity = 0
)

xgb_pred <- predict(xgb_model, X_test)
xgb_prob <- predict(xgb_model, X_test, type = "prob")
xgb_cm <- confusionMatrix(xgb_pred, y_test_fct, positive = "Win")

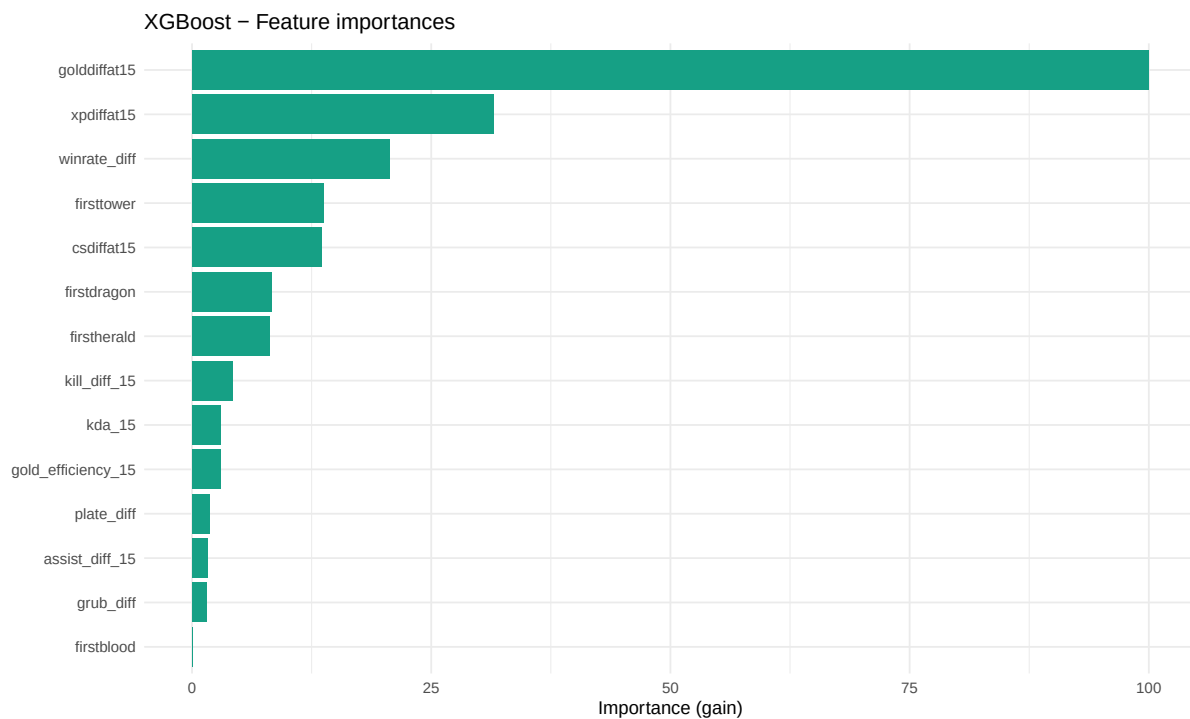
cat(sprintf(
  "\nXGBoost Results (trained in %.1fs):\nAccuracy: %.1f%%\nSensitivity (Win recall): %.1f%%\n",
  as.numeric(Sys.time() - t0, units = "secs"),
  xgb_cm$overall["Accuracy"] * 100,
  xgb_cm$byClass["Sensitivity"] * 100,
  xgb_cm$byClass["Specificity"] * 100,
  xgb_model$bestTune$nrounds,
  xgb_model$bestTune$max_depth,
  xgb_model$bestTune$eta,
  max(xgb_model$results$ROC)
))

```

XGBoost Results (trained in 2.8s):
 Accuracy: 75.2%
 Sensitivity (Win recall): 78.4%

Specificity (Loss recall): 71.2%
Best nrounds=100, max_depth=4, eta=0.05
CV AUC-ROC: 0.839

```
xgb_imp <- varImp(xgb_model)$importance %>%  
  rownames_to_column("feature") %>%  
  arrange(desc(Overall))  
  
ggplot(  
  xgb_imp,  
  aes(x = reorder(feature, Overall), y = Overall)  
) +  
  geom_col(fill = "#16a085") +  
  coord_flip() +  
  labs(  
    title = "XGBoost - Feature importances",  
    x = NULL, y = "Importance (gain)"  
  ) +  
  theme_minimal()
```



Model comparison

Side-by-side test-set metrics, confusion matrices, ROC curves. We check (a) which models beat the 72.9 % gold-rule baseline and (b) whether the non-linear RF gains anything over the linear/probabilistic baselines.

```
calc_metrics <- function(cm, prob, y_true, name) {
  roc_obj <- roc(y_true, prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
  tibble(
    Model = name,
    Accuracy = cm$overall["Accuracy"] * 100,
    AUC_ROC = auc(roc_obj) * 100,
    Precision = cm$byClass["Pos Pred Value"] * 100,
    Recall = cm$byClass["Sensitivity"] * 100,
    F1 = 2 * cm$byClass["Pos Pred Value"] * cm$byClass["Sensitivity"] /
      (cm$byClass["Pos Pred Value"] + cm$byClass["Sensitivity"]) * 100
  )
}

metrics <- bind_rows(
  calc_metrics(lr_cm, lr_prob, y_test_fct, "Logistic Regression"),
  calc_metrics(rf_cm, rf_prob, y_test_fct, "Random Forest"),
  calc_metrics(cart_cm, cart_prob, y_test_fct, "CART"),
  calc_metrics(nb_cm, nb_prob, y_test_fct, "Naive Bayes"),
  calc_metrics(knn_cm, knn_prob, y_test_fct, "KNN"),
  calc_metrics(xgb_cm, xgb_prob, y_test_fct, "XGBoost")
)

knitr::kable(metrics, digits = 1, caption = "Model Performance Comparison (H1)")
```

Table 2: Model Performance Comparison (H1)

Model	Accuracy	AUC_ROC	Precision	Recall	F1
Logistic Regression	75.5	84.2	77.0	79.1	78.0
Random Forest	74.1	82.8	75.5	78.0	76.8
CART	72.9	79.7	74.7	76.6	75.7
Naive Bayes	74.9	83.1	77.5	76.4	77.0
KNN	75.2	83.7	76.1	79.8	77.9
XGBoost	75.2	83.8	76.8	78.4	77.6

```

best_hyperparams <- tibble(
  Model = c(
    "Logistic Regression",
    "Random Forest",
    "CART",
    "Naive Bayes",
    "KNN",
    "XGBoost"
  ),
  `Tuned params` = c(
    "(none - glm has no hyperparameters)",
    sprintf("mtry = %d (ntree = 500 fixed)", rf_model$bestTune$mtry),
    sprintf("cp = %.5f", cart_model$bestTune$cp),
    sprintf(
      "usekernel = %s, adjust = %.2f, laplace = %.0f",
      nb_model$bestTune$usekernel,
      nb_model$bestTune$adjust,
      nb_model$bestTune$laplace
    ),
    sprintf("k = %d", knn_model$bestTune$k),
    sprintf(
      "nrounds = %d, max_depth = %d, eta = %.2f",
      xgb_model$bestTune$nrounds,
      xgb_model$bestTune$max_depth,
      xgb_model$bestTune$eta
    )
  ),
  `CV ROC` = c(
    max(lr_model$results$ROC),
    max(rf_model$results$ROC),
    max(cart_model$results$ROC),
    max(nb_model$results$ROC),
    max(knn_model$results$ROC),
    max(xgb_model$results$ROC)
  )
)

htmltools::div(
  style = "background-color: white; color: black; padding: 10px; border-radius: 4px;",
  htmltools::HTML(knitr::kable(
    best_hyperparams,
    digits = 3,

```

```

    caption = "Best hyperparameters per model (selected by 5-fold CV ROC)",
    format = "html"
  ))
)

```

Table 3: Best hyperparameters per model (selected by 5-fold CV ROC)

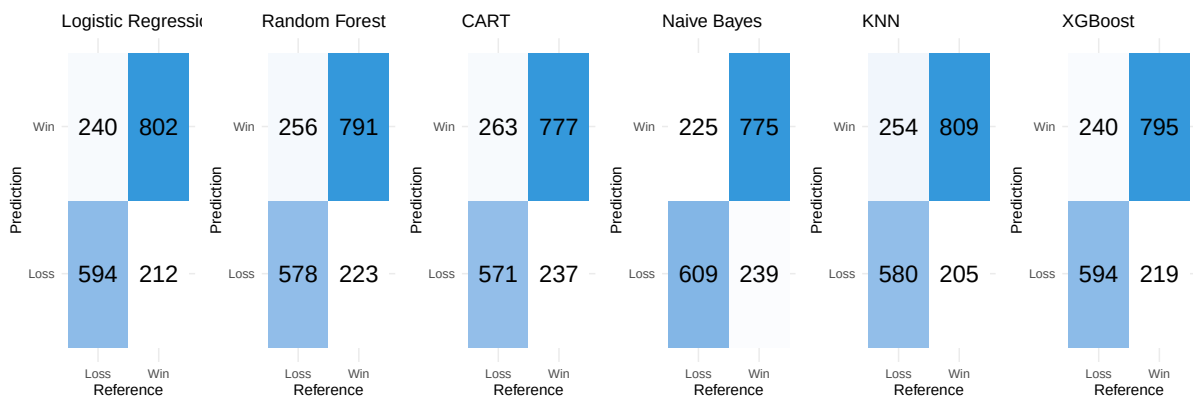
Model	Tuned params	CV ROC
Logistic Regression	(none - glm has no hyperparameters)	0.843
Random Forest	mtry = 3 (ntree = 500 fixed)	0.834
CART	cp = 0.00100	0.793
Naive Bayes	usekernel = TRUE, adjust = 1.50, laplace = 0	0.831
KNN	k = 345	0.838
XGBoost	nrounds = 100, max_depth = 4, eta = 0.05	0.839

```

plot_cm <- function(cm, title) {
  cm_df <- as.data.frame(cm$table)
  ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
    geom_tile() +
    geom_text(aes(label = Freq), size = 6) +
    scale_fill_gradient(low = "white", high = "#3498db") +
    labs(title = title) +
    theme_minimal() +
    theme(
      legend.position = "none",
      plot.background = element_rect(fill = "white", color = NA),
      panel.background = element_rect(fill = "white", color = NA)
    )
}

gridExtra::grid.arrange(
  plot_cm(lr_cm, "Logistic Regression"),
  plot_cm(rf_cm, "Random Forest"),
  plot_cm(cart_cm, "CART"),
  plot_cm(nb_cm, "Naive Bayes"),
  plot_cm(knn_cm, "KNN"),
  plot_cm(xgb_cm, "XGBoost"),
  ncol = 6
)

```



The test set inherits the underlying ~53/47 Blue-side base rate, so the ratio of TP to TN reflects how each model balances Win recall against Loss recall. Across all matrices, precision and recall stay within a few points of each other, which means F1 tracks accuracy closely and no model is systematically biased toward predicting Win or Loss.

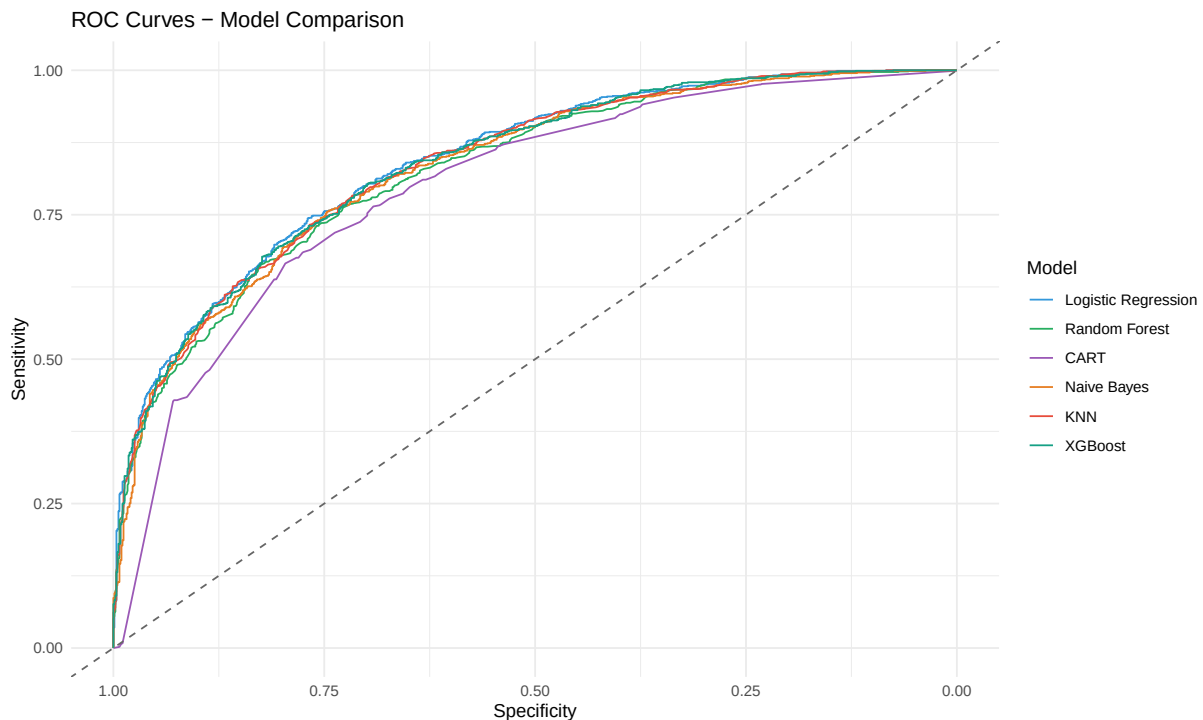
```
roc_lr <- roc(y_test_fct, lr_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_rf <- roc(y_test_fct, rf_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_cart <- roc(y_test_fct, cart_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_nb <- roc(y_test_fct, nb_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_knn <- roc(y_test_fct, knn_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_xgb <- roc(y_test_fct, xgb_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)

ggroc(list(
  "Logistic Regression" = roc_lr,
  "Random Forest" = roc_rf,
  "CART" = roc_cart,
  "Naive Bayes" = roc_nb,
  "KNN" = roc_knn,
  "XGBoost" = roc_xgb
)) +
  geom_abline(slope = 1, intercept = 1, linetype = "dashed", color = "gray40") +
  scale_color_manual(values = c(
    "Logistic Regression" = "#3498db",
    "Random Forest" = "#27ae60",
    "CART" = "#9b59b6",
    "Naive Bayes" = "#e67e22",
    "KNN" = "#e74c3c",
    "XGBoost" = "#16a085"
  )) +
  labs(
    title = "ROC Curves - Model Comparison",
```

```

    x = "Specificity", y = "Sensitivity", color = "Model"
  ) +
  theme_minimal()

```



All probabilistic AUC curves overlap closely; the discriminating differences are at the upper-left corner where the curves diverge by 1-2 pp.

Explainability comparison

Each model produces an importance score on a different axis (LR: coefficient magnitude, RF: mean Gini decrease, CART: total Gini reduction across splits, NB: filter-based per-feature AUC, XGBoost: split gain). KNN is excluded - no native importance signal. The interesting question is whether the five rankings agree on a small core of features.

```

# KNN excluded - no meaningful feature importance
top_features <- bind_rows(
  lr_coefs %>% mutate(rank = row_number(), model = "LR") %>%
    select(feature, rank, model),
  rf_imp %>% mutate(rank = row_number(), model = "RF") %>%
    select(feature, rank, model),
  cart_imp %>% mutate(rank = row_number(), model = "CART") %>%

```

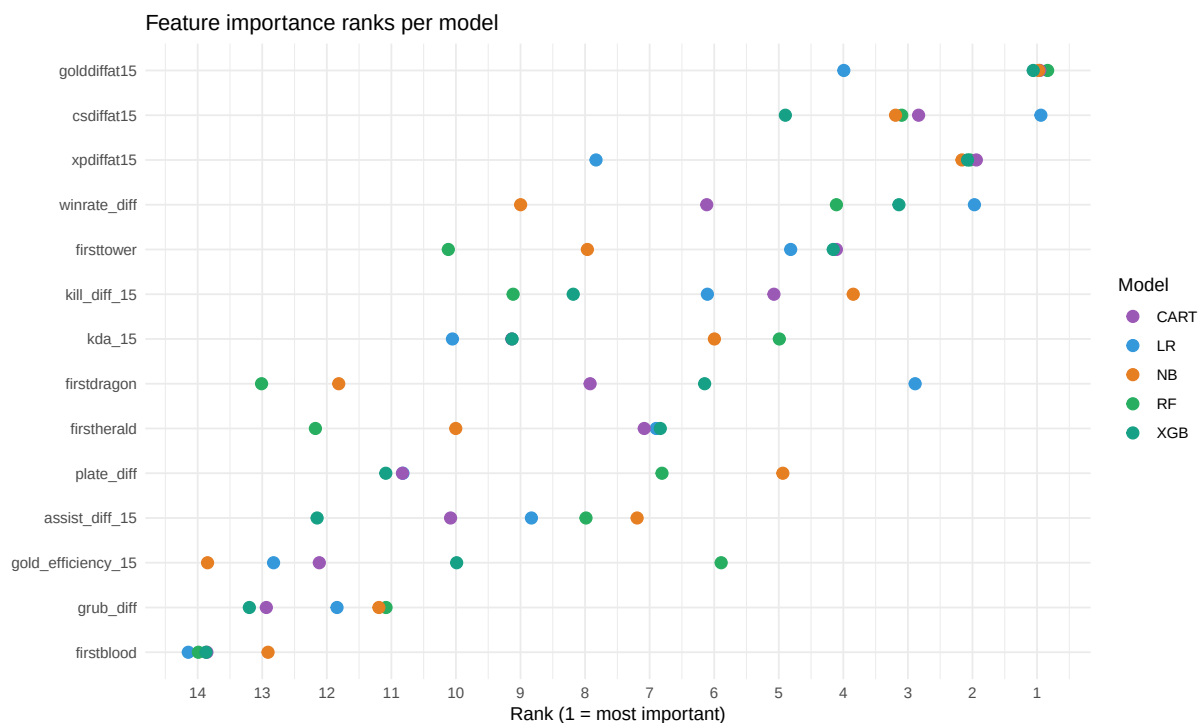


```

    select(feature, rank, model),
    nb_imp %>% mutate(rank = row_number(), model = "NB") %>%
    select(feature, rank, model),
    xgb_imp %>% mutate(rank = row_number(), model = "XGB") %>%
    select(feature, rank, model)
)

ggplot(top_features, aes(x = reorder(feature, -rank), y = rank, color = model)) +
  geom_jitter(size = 3, width = 0, height = 0.2) +
  coord_flip() +
  scale_y_reverse(breaks = 1:max(top_features$rank)) +
  scale_color_manual(values = c("LR" = "#3498db", "RF" = "#27ae60", "CART" = "#9b59b6", "NB" = "#f39c12", "XGB" = "#2ecc71")) +
  labs(
    title = "Feature importance ranks per model",
    x = NULL, y = "Rank (1 = most important)", color = "Model"
  ) +
  theme_minimal()

```



Each importance metric measures something different, so the raw scores cannot be averaged across models. Ranks, however, are comparable: **avg_rank** summarises which features every method agrees on, and **rank_spread** flags where they diverge.

```

full_ranks <- lr_coefs %>%
  mutate(rank_LR = row_number()) %>%
  select(feature, rank_LR) %>%
  full_join(rf_imp %>% mutate(rank_RF = row_number()) %>% select(feature, rank_RF),
    by = "feature"
  ) %>%
  full_join(cart_imp %>% mutate(rank_CART = row_number()) %>% select(feature, rank_CART),
    by = "feature"
  ) %>%
  full_join(nb_imp %>% mutate(rank_NB = row_number()) %>% select(feature, rank_NB),
    by = "feature"
  ) %>%
  full_join(xgb_imp %>% mutate(rank_XGB = row_number()) %>% select(feature, rank_XGB),
    by = "feature"
  ) %>%
  mutate(
    avg_rank = rowMeans(across(starts_with("rank_")), na.rm = TRUE),
    rank_spread = apply(
      across(starts_with("rank_")), 1,
      function(r) max(r, na.rm = TRUE) - min(r, na.rm = TRUE)
    )
  ) %>%
  arrange(avg_rank)

knitr::kable(
  full_ranks,
  digits = 2,
  caption = "Cross-model feature ranks. Lower avg_rank = consistently important across methods;
  higher rank_spread = methods disagree."
)

```

Table 4: Cross-model feature ranks. Lower avg_rank = consistently important across methods; higher rank_spread = methods disagree.

feature	rank_LR	rank_RF	rank_CART	rank_NB	rank_XGB	avg_rank	rank_spread
golddiffat15	4	1	1	1	1	1.6	3
csdiffat15	1	3	3	3	5	3.0	4
xpdiffat15	8	2	2	2	2	3.2	6
winrate_diff	2	4	6	9	3	4.8	7
firsttower	5	10	4	8	4	6.2	6
kill_diff_15	6	9	5	4	8	6.4	5
kda_15	10	5	9	6	9	7.8	5

feature	rank_LR	rank_RF	rank_CART	rank_NB	rank_XGB	avg_rank	rank_spread
firstdragon	3	13	8	12	6	8.4	10
firstherald	7	12	7	10	7	8.6	5
plate_diff	11	7	11	5	11	9.0	6
assist_diff_15	9	8	10	7	12	9.2	5
gold_efficiency_15	13	6	12	14	10	11.0	8
grub_diff	12	11	13	11	13	12.0	2
firstblood	14	14	14	13	14	13.8	1

The continuous early-game differentials - `golddiffat15`, `xpdiffat15`, `csdiffat15`, and `winrate_diff` - head every ranking with low `avg_rank` and small `rank_spread`, forming the consensus core that any defence of H1 should rely on. Disagreement concentrates on the binary objective flags `firstdragon` and `firstherald`, which rank high for Logistic Regression but low for the tree ensembles and the Naive Bayes filter. This is a metric-driven artefact: LR treats each flag as an additive log-odds shift, the tree models absorb the same signal into continuous splits, and the filter score reads the flags as weak standalone predictors. LASSO retains `firstdragon` with a non-zero coefficient in S3, which confirms that the disagreement reflects differences in measurement scale rather than a lack of substantive signal.

Same models, enriched data

H1 summarises the early game at the team level - one gold-diff per game, one xp-diff per game - but each match actually carries five values per metric, one per lane. Aggregating loses structure: a 1,000-gold team lead carried entirely by bot is a stronger signal than the same lead spread evenly across five lanes. We hold the S1 models and the train/test split fixed and refit them on an enriched matrix - H1 plus per-role @15 differentials for economy (gold, xp, cs) and combat (kills, assists, deaths) - to see whether the extra granularity beats the team aggregate.

```
# Per-role @15 differentials. Economy diffs come straight from the player row;
# combat diffs we compute as Blue-player-stat minus lane-opponent stat.
role_enriched <- players %>%
  filter(
    position %in% c("top", "jng", "mid", "bot", "sup"),
    side == "Blue"
  ) %>%
  mutate(
    killldiffat15 = killsat15 - opp_killsat15,
    assistdiffat15 = assistsat15 - opp_assistsat15,
    deathdiffat15 = deathsat15 - opp_deathsat15
```

```

) %>%
select(
  gameid, position,
  golddiffat15, xpdiffat15, csdiffat15,
  killdiffat15, assistdiffat15, deathdiffat15
) %>%
drop_na() %>%
pivot_wider(
  id_cols = gameid,
  names_from = position,
  values_from = c(
    golddiffat15, xpdiffat15, csdiffat15,
    killdiffat15, assistdiffat15, deathdiffat15
  ),
  names_glue = "{position}_{.value}"
)

games_enriched <- games_model %>%
  inner_join(role_enriched, by = "gameid") %>%
  drop_na()

cat(sprintf(
  "Per-role features added: %d (5 roles x 6 metrics @15)\nGames after join + drop_na: %d (",
  ncol(role_enriched) - 1,
  nrow(games_enriched),
  nrow(games_model) - nrow(games_enriched),
  (nrow(games_model) - nrow(games_enriched)) / nrow(games_model) * 100,
  ncol(games_enriched) - 2,
  ncol(games_model) - 2
))

```

Per-role features added: 30 (5 roles x 6 metrics @15)
 Games after join + drop_na: 9236 (lost 0 rows from H1, 0.0%)
 Feature count: 44 (was 14 on H1)

```

# Reuse train_ids so the only thing changing between runs is the feature matrix.
train_e <- games_enriched %>%
  filter(gameid %in% train_ids) %>%
  select(-gameid)
test_e <- games_enriched %>%
  filter(!gameid %in% train_ids) %>%
  select(-gameid)

```

```

X_train_e <- train_e %>% select(-blue_win)
X_test_e <- test_e %>% select(-blue_win)
y_train_e <- factor(train_e$blue_win, levels = c(0, 1), labels = c("Loss", "Win"))
y_test_e <- factor(test_e$blue_win, levels = c(0, 1), labels = c("Loss", "Win"))

preproc_e <- preProcess(X_train_e, method = c("center", "scale"))
X_train_e_scaled <- predict(preproc_e, X_train_e)
X_test_e_scaled <- predict(preproc_e, X_test_e)

```

The 30 per-role columns sum (per metric) to the team-level totals already in H1, so the enriched matrix is rank-deficient by construction (`top + jng + mid + bot + sup == team` for gold, XP, CS, kills, and assists; only the death-diff family escapes because S1 deliberately omitted `death_diff_15`). A naive `solve(cor(X))` therefore fails with a singular matrix. We first identify and drop the exact linear combinations with `caret::findLinearCombos()`, then iteratively remove the feature with the largest VIF until every remaining VIF is at or below 10. Working on the trimmed matrix removes the redundancy that would otherwise dominate the comparison with H1 and lets each model fit on a feature set whose coefficients (LR) and split candidates (RF/XGB) are no longer aliased.

```

# Step 1: drop the 5 columns that are exact linear combinations of others
lc_e <- caret::findLinearCombos(as.matrix(X_train_e_scaled))
removed_perfect <- names(X_train_e_scaled)[lc_e$remove]
keep1 <- setdiff(seq_len(ncol(X_train_e_scaled)), lc_e$remove)
X_iter <- X_train_e_scaled[, keep1]

# Step 2: iteratively drop the max-VIF feature while max(VIF) > 10
threshold_e <- 10
removed_vif <- character()
repeat {
  vif <- diag(solve(cor(X_iter)))
  if (max(vif) <= threshold_e) break
  drop <- names(vif)[which.max(vif)]
  removed_vif <- c(removed_vif, drop)
  X_iter <- X_iter[, setdiff(names(X_iter), drop)]
}
surviving_features_e <- names(X_iter)

cat(sprintf(
  "Started: %d features\nDropped (perfect collinearity): %d -> %s\nDropped (iterative VIF :
  ncol(X_train_e_scaled),
  length(removed_perfect), paste(removed_perfect, collapse = ", "),

```

```

    threshold_e,
    length(removed_vif),    paste(removed_vif, collapse = ", "),
    length(surviving_features_e)
  ))

```

Started: 44 features

Dropped (perfect collinearity): 5 -> sup_golddiffat15, sup_xpdiffat15, sup_csdiffat15, sup_k

Dropped (iterative VIF > 10): 8 -> kill_diff_15, golddiffat15, assist_diff_15, csdiffat15, xp

Surviving feature count: 31

```

# Subset matrices to the surviving features and reuse the H1 variable names
X_train_e <- X_train_e[, surviving_features_e]
X_test_e <- X_test_e[, surviving_features_e]
X_train_e_scaled <- X_train_e_scaled[, surviving_features_e]
X_test_e_scaled <- X_test_e_scaled[, surviving_features_e]

final_vif_e <- tibble(
  feature = surviving_features_e,
  VIF      = diag(solve(cor(X_train_e_scaled)))
) %>% arrange(desc(VIF))

knitr::kable(final_vif_e,
  digits = 2,
  caption = sprintf(
    "Final VIF on trimmed enriched matrix (%d features, all VIF <= %d).",
    length(surviving_features_e), threshold_e
  )
)

```

Table 5: Final VIF on trimmed enriched matrix (31 features, all VIF <= 10).

feature	VIF
bot_golddiffat15	9.15
bot_killdiffat15	8.91
jng_xpdiffat15	6.43
jng_csdiffat15	5.65
jng_killdiffat15	5.60
top_deathdiffat15	5.04
mid_killdiffat15	4.90
mid_deathdiffat15	4.67

feature	VIF
jng_deathdiffat15	4.24
sup_deathdiffat15	4.04
bot_deathdiffat15	3.94
top_killdiffat15	3.82
bot_csdiffat15	3.67
top_xpdiffat15	3.53
mid_xpdiffat15	3.37
jng_assistdiffat15	3.05
plate_diff	2.64
top_csdiffat15	2.50
mid_csdiffat15	2.50
bot_xpdiffat15	2.43
kda_15	2.34
bot_assistdiffat15	2.12
mid_assistdiffat15	1.95
firsttower	1.57
top_assistdiffat15	1.42
grub_diff	1.36
firstherald	1.25
firstblood	1.22
gold_efficiency_15	1.22
firstdragon	1.14
winrate_diff	1.13

The iterative pass drops `golddiffat15`, `csdiffat15`, `xpdiffat15`, and three of the five per-role gold columns; only `bot_golddiffat15` survives among the gold-diff family, which is exactly the H2 question - does the bot-lane carry signal that the team-level totals do not? The trimmed matrix is the cleanest setting in which to ask it.

```
lr_e <- train(
  x = X_train_e_scaled, y = y_train_e,
  method = "glm", family = "binomial",
  trControl = ctrl, metric = "ROC"
)

# RF mtry grid scaled to the new feature count.
n_feat_e <- ncol(X_train_e)
rf_grid_e <- unique(round(c(
  sqrt(n_feat_e), sqrt(n_feat_e) * 1.5,
  sqrt(n_feat_e) * 2, sqrt(n_feat_e) * 3
```

```

)))
rf_e <- train(
  x = X_train_e, y = y_train_e,
  method = "rf", trControl = ctrl, metric = "ROC",
  tuneGrid = expand.grid(mtry = rf_grid_e), ntree = 500
)

nb_e <- train(
  x = X_train_e, y = y_train_e,
  method = "naive_bayes", trControl = ctrl, metric = "ROC",
  tuneGrid = nb_grid
)

n_train_e <- nrow(X_train_e_scaled)
k_theo_e <- round(sqrt(n_train_e))
k_grid_e <- unique(round(exp(seq(log(5), log(k_theo_e * 4), length.out = 12))))
k_grid_e <- ifelse(k_grid_e %% 2 == 0, k_grid_e + 1, k_grid_e)
knn_e <- train(
  x = X_train_e_scaled, y = y_train_e,
  method = "knn", trControl = ctrl, metric = "ROC",
  tuneGrid = expand.grid(k = k_grid_e)
)

cart_e <- train(
  x = X_train_e, y = y_train_e,
  method = "rpart", trControl = ctrl, metric = "ROC",
  tuneGrid = expand.grid(cp = c(0.00001, 0.00005, 0.0001, 0.0005, 0.001))
)

xgb_e <- train(
  x = X_train_e, y = y_train_e,
  method = "xgbTree", trControl = ctrl, metric = "ROC",
  tuneGrid = xgb_grid, verbosity = 0
)

score_e <- function(model, X_eval, name) {
  cls <- predict(model, X_eval)
  prob <- predict(model, X_eval, type = "prob")
  cm <- confusionMatrix(cls, y_test_e, positive = "Win")
  calc_metrics(cm, prob, y_test_e, name)
}

```



```

metrics_enriched <- bind_rows(
  score_e(lr_e, X_test_e_scaled, "Logistic Regression"),
  score_e(rf_e, X_test_e, "Random Forest"),
  score_e(cart_e, X_test_e, "CART"),
  score_e(nb_e, X_test_e, "Naive Bayes"),
  score_e(knn_e, X_test_e_scaled, "KNN"),
  score_e(xgb_e, X_test_e, "XGBoost")
)

knitr::kable(metrics_enriched,
  digits = 1,
  caption = "Enriched feature set: same six models, H1 + 30 per-role @15 features"
)

```

Table 6: Enriched feature set: same six models, H1 + 30 per-role @15 features

Model	Accuracy	AUC_ROC	Precision	Recall	F1
Logistic Regression	75.4	84.4	76.5	79.8	78.1
Random Forest	74.5	83.3	76.2	77.8	77.0
CART	73.6	78.5	75.4	77.0	76.2
Naive Bayes	74.1	82.5	78.4	73.0	75.6
KNN	75.5	83.7	75.7	81.6	78.5
XGBoost	75.5	83.7	76.4	80.2	78.2

```

enrich_delta <- metrics %>%
  select(Model, H1_Acc = Accuracy, H1_AUC = AUC_ROC, H1_F1 = F1) %>%
  inner_join(
    metrics_enriched %>% select(Model,
      Enr_Acc = Accuracy,
      Enr_AUC = AUC_ROC,
      Enr_F1 = F1
    ),
    by = "Model"
  ) %>%
  mutate(
    Acc_delta = Enr_Acc - H1_Acc,
    AUC_delta = Enr_AUC - H1_AUC,
    F1_delta = Enr_F1 - H1_F1
  )

knitr::kable(enrich_delta,

```

```

digits = 2,
caption = "Same models, enriched data: deltas vs H1 (positive = enrichment helped)"
)

```

Table 7: Same models, enriched data: deltas vs H1 (positive = enrichment helped)

Model	H1_Acc	H1_AUC	H1_F1	Enr_Acc	Enr_AUC	Enr_F1	Acc_delta	AUC_delta	F1_delta
Logistic Regression	75.54	84.21	78.02	75.43	84.35	78.09	-0.11	0.14	0.07
Random Forest	74.08	82.81	76.76	74.51	83.33	77.01	0.43	0.52	0.25
CART	72.94	79.72	75.66	73.59	78.46	76.20	0.65	-1.26	0.54
Naive Bayes	74.89	83.14	76.96	74.13	82.52	75.59	-0.76	-0.62	-1.37
KNN	75.16	83.65	77.90	75.49	83.75	78.50	0.32	0.09	0.60
XGBoost	75.16	83.79	77.60	75.54	83.75	78.25	0.38	-0.04	0.65

```

roc_lr_e <- roc(y_test_e, predict(lr_e, X_test_e_scaled, type = "prob"), levels = c("Loss", "Win"), "Win")
roc_rf_e <- roc(y_test_e, predict(rf_e, X_test_e, type = "prob"), levels = c("Loss", "Win"), "Win")
roc_cart_e <- roc(y_test_e, predict(cart_e, X_test_e, type = "prob"), levels = c("Loss", "Win"), "Win")
roc_nb_e <- roc(y_test_e, predict(nb_e, X_test_e, type = "prob"), levels = c("Loss", "Win"), "Win")
roc_knn_e <- roc(y_test_e, predict(knn_e, X_test_e_scaled, type = "prob"), levels = c("Loss", "Win"), "Win")
roc_xgb_e <- roc(y_test_e, predict(xgb_e, X_test_e, type = "prob"), levels = c("Loss", "Win"), "Win")

ggroc(list(
  "LR (H1)"      = roc_lr,    "LR (enriched)"  = roc_lr_e,
  "RF (H1)"      = roc_rf,    "RF (enriched)"  = roc_rf_e,
  "CART (H1)"    = roc_cart,  "CART (enriched)" = roc_cart_e,
  "NB (H1)"      = roc_nb,    "NB (enriched)"  = roc_nb_e,
  "KNN (H1)"     = roc_knn,   "KNN (enriched)" = roc_knn_e,
  "XGB (H1)"     = roc_xgb,   "XGB (enriched)" = roc_xgb_e
)) +
  geom_abline(slope = 1, intercept = 1, linetype = "dashed", color = "gray40") +
  labs(
    title = "ROC: H1 baseline vs enriched (per-role) feature set",
    x = "Specificity", y = "Sensitivity", color = "Model"
  ) +
  theme_minimal()

```



```
mean_auc <- mean(enrich_delta$AUC_delta)
mean_acc <- mean(enrich_delta$Acc_delta)
n_better <- sum(enrich_delta$AUC_delta > 0)
n_models <- nrow(enrich_delta)

cat(sprintf(
  "Mean AUC change across %d models: %+.2f pp\nMean accuracy change: %+.2f pp\nModels improved by enrichment: %d / %d\n",
  n_models, mean_auc, mean_acc, n_better, n_models
))
```

Mean AUC change across 6 models: -0.20 pp
 Mean accuracy change: +0.15 pp
 Models improved by enrichment (AUC delta > 0): 3 / 6

```
get_delta <- function(model_name, col) {
  enrich_delta[[col]][enrich_delta$Model == model_name]
}

lr_auc_d <- get_delta("Logistic Regression", "AUC_delta")
rf_auc_d <- get_delta("Random Forest", "AUC_delta")
cart_auc_d <- get_delta("CART", "AUC_delta")
cart_acc_d <- get_delta("CART", "Acc_delta")
```

```

nb_auc_d <- get_delta("Naive Bayes", "AUC_delta")
knn_auc_d <- get_delta("KNN", "AUC_delta")
xgb_auc_d <- get_delta("XGBoost", "AUC_delta")

n_surviving <- length(surviving_features_e)
n_h1 <- ncol(X_train)
n_dropped <- length(removed_perfect) + length(removed_vif)
knn_k_enriched <- knn_e$bestTune$k
rf_mtry_h1 <- rf_model$bestTune$mtry
rf_mtry_e <- rf_e$bestTune$mtry

```

With multicollinearity removed, the trimmed enriched matrix matches the H1 baseline. Mean AUC -0.20 pp, mean accuracy +0.15 pp, 3 of 6 models improve. The trim is what makes the comparison fair - on the raw 44-feature enrichment every model paid a penalty for the redundant gold/XP/CS aliases, with NB hit hardest by the `top + jng + mid + bot + sup == team` identities.

- **LR:** +0.14 AUC - coefficients no longer split across the aliased gold/XP duplicates; surviving per-role columns add only marginal signal.
- **RF:** +0.52 AUC - tuned `mtry` rises from 3 to 6, but the ceiling is unchanged.
- **NB:** -0.62 AUC - independence assumption is now satisfied ($VIF \leq 10$); the residual loss is the per-role columns adding noise rather than signal.
- **KNN:** +0.09 AUC - 31 standardised dimensions stay Euclidean-comparable; `k = 345` smooths over local noise.
- **CART:** -1.26 AUC, +0.65 pp accuracy - single-tree variance is sensitive to the wider input regardless of VIF cleanup.

Team-level totals already saturate the @15 signal, so H1 stays as the S1 reference set (S3 selection, Shiny export). H2 below asks the sharper question: does bot lane specifically carry signal beyond the team aggregates? Of the five per-role gold-diff columns, only `bot_golddiffat15` survives the VIF filter, so a non-trivial bot-lane importance is not an alias artefact.

Role-feature importance (H2 test)

H2 asks whether bot lane carries the strongest role-specific early-game signal. The AUC half of the test is the enriched-vs-H1 RF comparison in the delta table above. The role-ranking half is sharper after the VIF trim: of the original 30 per-role columns, 22 survive (the others were either perfect linear combinations of the team totals or had $VIF > 10$ against the rest of the matrix). H2 is supported if and only if the top-ranked `bot_*` feature in the enriched RF outranks the best surviving feature of every other role.

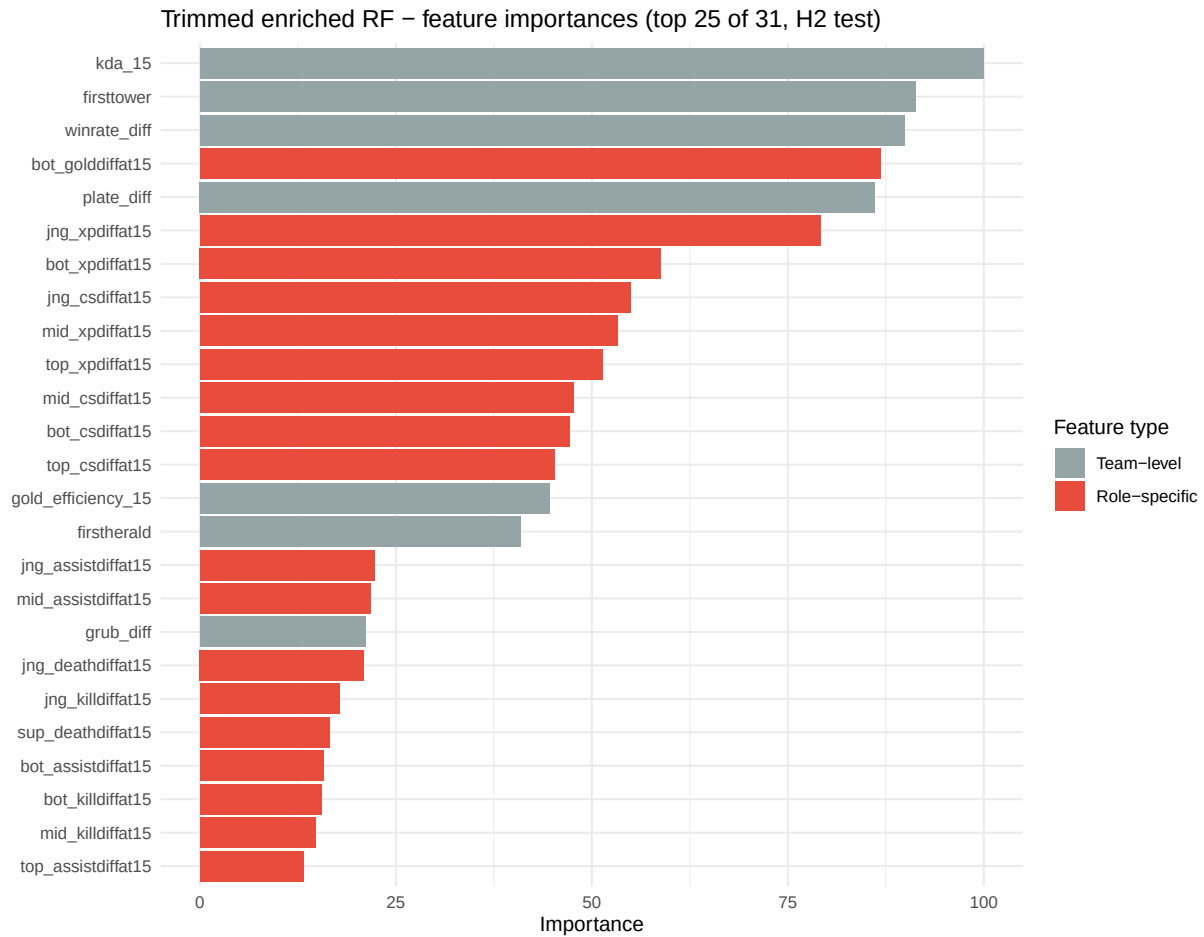
```

enr_imp <- varImp(rf_e)$importance %>%
  rownames_to_column("feature") %>%
  arrange(desc(Overall))

role_features <- enr_imp %>%
  filter(str_detect(feature, "^(top|jng|mid|bot|sup)_"))

ggplot(
  enr_imp %>% head(25),
  aes(
    x = reorder(feature, Overall), y = Overall,
    fill = str_detect(feature, "^(top|jng|mid|bot|sup)_")
  )
) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(
    values = c("FALSE" = "#95a5a6", "TRUE" = "#e74c3c"),
    labels = c("Team-level", "Role-specific")
  ) +
  labs(
    title = sprintf(
      "Trimmed enriched RF - feature importances (top %d of %d, H2 test)",
      min(25, nrow(enr_imp)), nrow(enr_imp)
    ),
    x = NULL, y = "Importance", fill = "Feature type"
  ) +
  theme_minimal()

```



```
bot_features <- role_features %>%
  filter(str_detect(feature, "^bot_")) %>%
  mutate(overall_rank = match(feature, enr_imp$feature))

other_role_features <- role_features %>%
  filter(!str_detect(feature, "^bot_")) %>%
  group_by(role = str_extract(feature, "[a-z]+")) %>%
  summarise(
    best_rank = min(match(feature, enr_imp$feature)),
    avg_importance = mean(Overall),
    .groups = "drop"
  )

cat("Bot lane feature rankings in enriched (H2) RF:\n")
```

Bot lane feature rankings in enriched (H2) RF:

```
print(bot_features)
```

	feature	Overall	overall_rank
1	bot_golddiffat15	86.83691	4
2	bot_xpdiffat15	58.81688	7
3	bot_csdiffat15	47.16377	12
4	bot_assistdiffat15	15.78876	22
5	bot_killdiffat15	15.51681	23
6	bot_deathdiffat15	11.67553	29

```
cat("\nBest rank per role:\n")
```

Best rank per role:

```
print(other_role_features)
```

```
# A tibble: 4 x 3
  role best_rank avg_importance
  <chr>    <int>         <dbl>
1 jng         6          39.0
2 mid         9          29.9
3 sup        21          16.6
4 top        10          26.9
```

The two prints above, read together with the enriched-vs-H1 AUC deltas earlier in this section, give the H2 evidence: bot lane is supported if and only if at least one **bot_*** feature outranks the best surviving feature of every other role. The VIF trim makes this an honest comparison - **bot_golddiffat15** is the only per-role gold-diff column to clear the threshold, so a bot-lane top rank cannot be driven by an aliased team-level total.

Scenario 3: Feature Selection

Scenario 3 reduces the 14 candidate early-game predictors to a more interpretable subset. The S1 VIF analysis flagged residual collinearity on **golddiffat15** and **kill_diff_15**, and the LR coefficient table left several features carrying near-zero weight - both are signs that some of the 14 predictors duplicate signal already captured by the dominant resource diffs. We compare one algorithmic method (forward stepwise via AIC) with two embedded regularisation methods (LASSO, Elastic Net) to identify which predictors are consistently retained and whether a compact subset can match the full model's predictive performance.

Forward Stepwise (AIC)

Forward stepwise selection starts from an intercept-only model and admits one feature at a time, each time choosing the feature whose addition lowers AIC by the most, and stopping when no remaining candidate can lower AIC further. We use it on the 14 H1 features and track per-step Wald p-values to see when individual evidence weakens as correlated predictors enter.

```
t0 <- Sys.time()

fwd_train_df <- as.data.frame(X_train_scaled)
fwd_train_df$result <- y_train_fct
fwd_test_df <- as.data.frame(X_test_scaled)

null_model <- glm(result ~ 1, data = fwd_train_df, family = "binomial")
full_formula <- as.formula(
  paste("result ~", paste(names(X_train_scaled), collapse = " + "))
)

set.seed(42)
fwd_model <- MASS::stepAIC(
  null_model,
  scope      = list(lower = null_model, upper = full_formula),
  direction = "forward",
  trace      = FALSE
)

fwd_selected <- names(coef(fwd_model))
fwd_selected <- fwd_selected[fwd_selected != "(Intercept)"]
fwd_n <- length(fwd_selected)
fwd_eliminated <- setdiff(colnames(X_train_scaled), fwd_selected)

# Reconstruct addition order from stepAIC's anova trace
fwd_anova <- fwd_model$anova
fwd_addition_order <- gsub("^\\+ ", "", as.character(fwd_anova$Step))
fwd_addition_order <- fwd_addition_order[fwd_addition_order != ""]

fwd_pred_prob_raw <- predict(fwd_model, newdata = fwd_test_df, type = "response")
fwd_pred_class <- factor(ifelse(fwd_pred_prob_raw > 0.5, "Win", "Loss"),
  levels = c("Loss", "Win")
)

fwd_cm <- confusionMatrix(fwd_pred_class, y_test_fct, positive = "Win")
fwd_roc <- roc(y_test_fct, fwd_pred_prob_raw,
```



```

    levels = c("Loss", "Win"), quiet = TRUE
)

cat(sprintf(
  "\nForward Stepwise Results (%.1fs):\nFeatures selected: %d / %d\nFinal AIC: %.1f\nAccuracy: %.1f\n",
  as.numeric(Sys.time() - t0, units = "secs"),
  fwd_n, ncol(X_train_scaled),
  AIC(fwd_model),
  fwd_cm$overall["Accuracy"] * 100,
  fwd_cm$byClass["Sensitivity"] * 100,
  fwd_cm$byClass["Specificity"] * 100,
  auc(fwd_roc)
))

```

Forward Stepwise Results (1.0s):
 Features selected: 10 / 14
 Final AIC: 7158.8
 Accuracy: 75.6%
 Sensitivity (Win recall): 79.5%
 Specificity (Loss recall): 71.0%
 AUC-ROC (test): 0.842

```
cat("\nSelected features (in order of addition):\n")
```

Selected features (in order of addition):

```
cat(paste(fwd_addition_order, collapse = ", "), "\n")
```

golddiffat15, winrate_diff, firstdragon, firsttherald, firsttower, csdiffat15, kill_diff_15, ...

```
cat("\nEliminated features:\n")
```

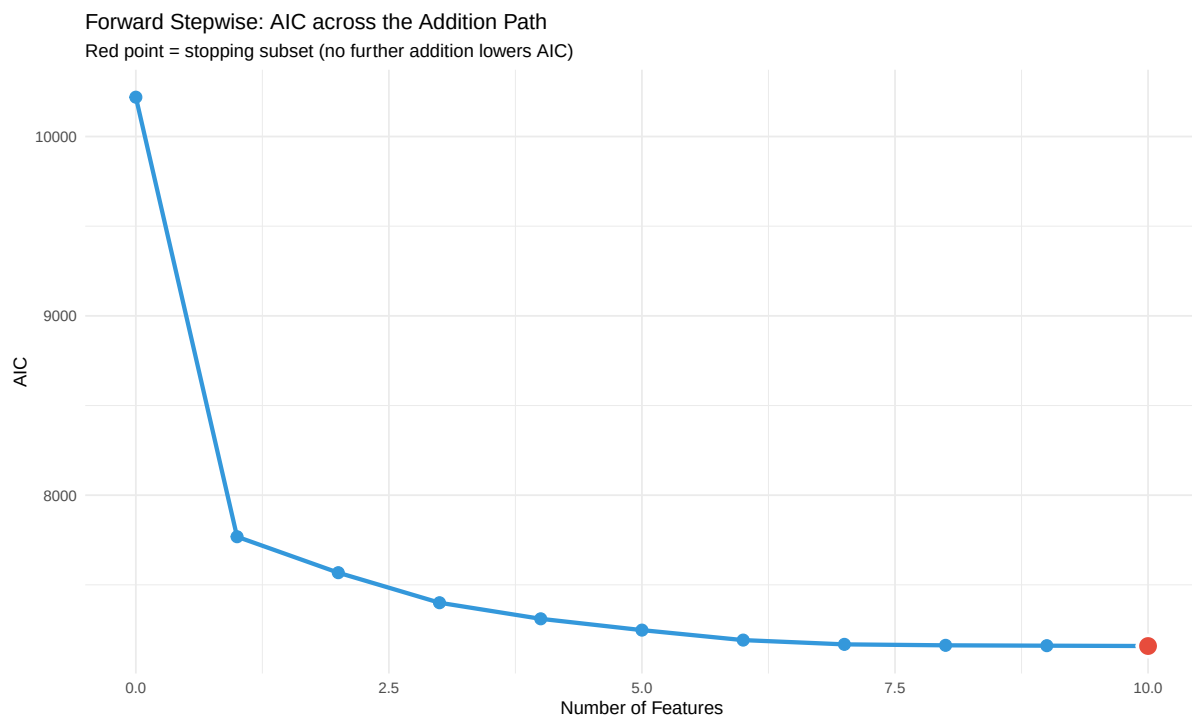
Eliminated features:

```
cat(paste(fwd_eliminated, collapse = ", "), "\n")
```

```
assist_diff_15, gold_efficiency_15, plate_diff, firstblood
```

```
fwd_aic_df <- data.frame(
  step = seq_along(fwd_anova$AIC) - 1L,
  AIC = fwd_anova$AIC,
  added = c(NA_character_, fwd_addition_order)
)

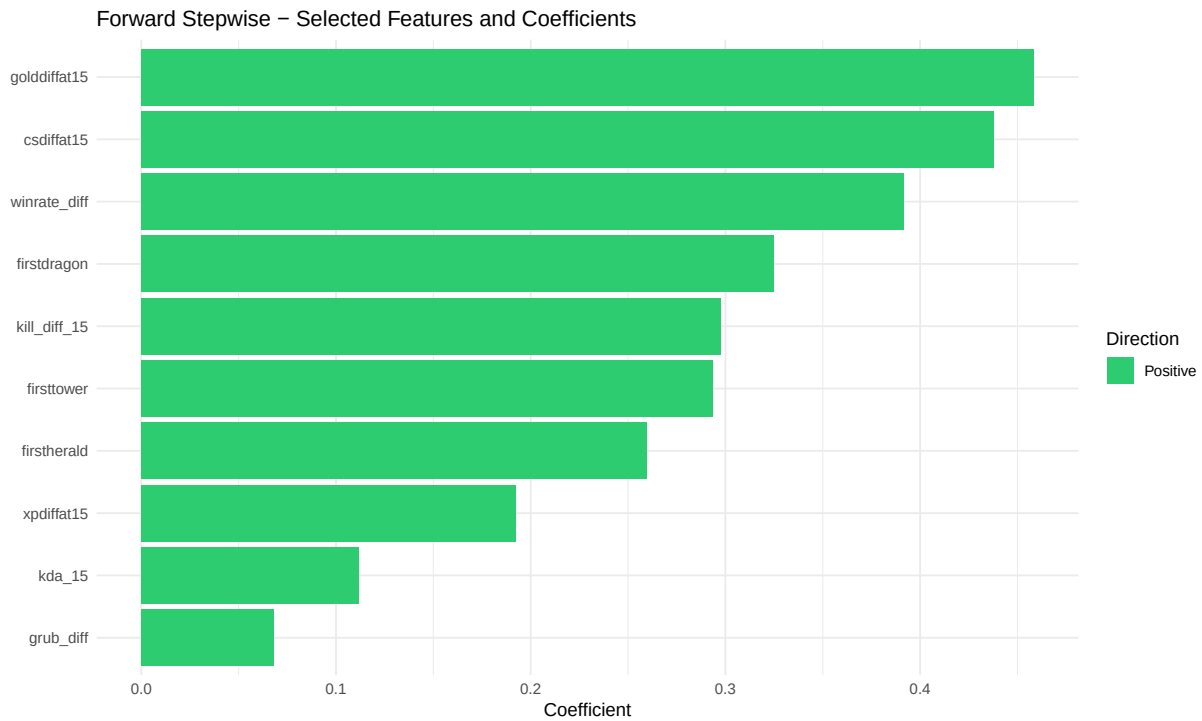
ggplot(fwd_aic_df, aes(x = step, y = AIC)) +
  geom_line(color = "#3498db", linewidth = 1.2) +
  geom_point(size = 3, color = "#3498db") +
  geom_point(
    data = filter(fwd_aic_df, step == max(step)),
    size = 5, shape = 21, fill = "#e74c3c", color = "white", stroke = 1.5
  ) +
  labs(
    title = "Forward Stepwise: AIC across the Addition Path",
    subtitle = "Red point = stopping subset (no further addition lowers AIC)",
    x = "Number of Features", y = "AIC"
  ) +
  theme_minimal()
```



```
fwd_coef_raw <- coef(fwd_model)
fwd_coef_raw <- fwd_coef_raw[names(fwd_coef_raw) != "(Intercept)"]
fwd_coef_df <- data.frame(
  feature      = names(fwd_coef_raw),
  coefficient   = as.numeric(fwd_coef_raw)
) %>%
  arrange(desc(abs(coefficient)))

ggplot(
  fwd_coef_df,
  aes(x = reorder(feature, abs(coefficient)), y = coefficient, fill = coefficient > 0)
) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(
    values = c("TRUE" = "#2ecc71", "FALSE" = "#e74c3c"),
    breaks = c("TRUE", "FALSE"),
    labels = c("Positive", "Negative")
  ) +
  labs(
    title = "Forward Stepwise - Selected Features and Coefficients",
    x = NULL, y = "Coefficient", fill = "Direction"
  )
```

```
) +  
theme_minimal()
```



```
fwd_vs_full <- tibble(  
  Model = c("LR - full model (S1)", "Forward Stepwise - selected subset (S3)"),  
  Features = c(ncol(X_train_scaled), fwd_n),  
  Accuracy = c(  
    round(lr_cm$overall["Accuracy"] * 100, 1),  
    round(fwd_cm$overall["Accuracy"] * 100, 1)  
  ),  
  `AUC-ROC` = c(  
    round(auc(roc_lr), 3),  
    round(auc(fwd_roc), 3)  
  ),  
  Sensitivity = c(  
    round(lr_cm$byClass["Sensitivity"] * 100, 1),  
    round(fwd_cm$byClass["Sensitivity"] * 100, 1)  
  ),  
  Specificity = c(  
    round(lr_cm$byClass["Specificity"] * 100, 1),  
    round(fwd_cm$byClass["Specificity"] * 100, 1)  
  )  
)
```

```

    )
  )

htmltools::div(
  style = "background-color: white; color: black; padding: 10px; border-radius: 4px;",
  htmltools::HTML(knitr::kable(
    fwd_vs_full,
    caption = "Forward Stepwise selected subset vs full model (LR)",
    format = "html"
  ))
)

```

Table 8: Forward Stepwise selected subset vs full model (LR)

Model	Features	Accuracy	AUC-ROC	Sensitivity	Specificity
LR - full model (S1)	14	75.5	0.842	79.1	71.2
Forward Stepwise - selected subset (S3)	10	75.6	0.842	79.5	71.0

Forward stepwise keeps 10 of the 14 features and matches the full-model LR baseline (test AUC 0.842 vs 0.842, accuracy 75.6% vs 75.5%). The four dropped features - `assist_diff_15`, `firstblood`, `gold_efficiency_15`, `plate_diff` - are collectively redundant with the rest of the matrix.

`golddiffat15` enters first and drops AIC by ~2,400 on its own - more than every subsequent admission combined. The curve plateaus by $k = 7$; the last three admissions clear AIC by deviance drops under four units each - the boundary of the criterion, and the band where the p-value evidence is weakest.

Significance evolution along the addition path

```

# At each subset size k, refit glm on the first k features (in addition order)
# and capture coefficient + SE + p-value for every feature in the model.
fwd_signif_path <- map_dfr(seq_along(fwd_addition_order), function(k) {
  feats <- fwd_addition_order[seq_len(k)]
  df_k <- fwd_train_df[, c(feats, "result"), drop = FALSE]
  m_k <- glm(result ~ ., data = df_k, family = "binomial")
  s_k <- summary(m_k)$coefficients
  s_k <- s_k[rownames(s_k) != "(Intercept)", , drop = FALSE]
  tibble(

```

```

      step      = k,
      feature = rownames(s_k),
      coef      = s_k[, "Estimate"],
      se        = s_k[, "Std. Error"],
      p_value   = s_k[, "Pr(>|z|)"]
    )
  })

fwd_pvalue_table <- fwd_signif_path %>%
  select(step, feature, p_value) %>%
  pivot_wider(names_from = step, values_from = p_value, names_prefix = "k=") %>%
  mutate(feature = factor(feature, levels = fwd_addition_order)) %>%
  arrange(feature)

fwd_pvalue_display <- fwd_pvalue_table %>%
  mutate(across(
    starts_with("k="),
    ~ ifelse(is.na(.x), "-",
             ifelse(.x < 0.001, "<0.001", sprintf("%.3f", .x))
    )
  ))

knitr::kable(
  fwd_pvalue_display,
  caption = sprintf(
    "P-value evolution across the addition path (k = subset size, %d steps total). Each r
    fwd_n
  ),
  align = c("l", rep("r", fwd_n))
)

```

Table 9: P-value evolution across the addition path (k = subset size, 10 steps total). Each row is a retained feature in order of admission; each column is the model with that many predictors. Cells where a feature has not yet entered are blank.

feature	k=1	k=2	k=3	k=4	k=5	k=6	k=7	k=8	k=9	k=10
golddif-	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
fat15										
win-	-	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
rate_diff										
firstdragon	-	-	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

feature	k=1	k=2	k=3	k=4	k=5	k=6	k=7	k=8	k=9	k=10
firstherald	-	-	-	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
firsttower	-	-	-	-	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
csdiffat15	-	-	-	-	-	<0.001	<0.001	<0.001	<0.001	<0.001
kill_diff_15	-	-	-	-	-	-	<0.001	<0.001	<0.001	0.001
xpdiffat15	-	-	-	-	-	-	-	0.007	0.003	0.003
grub_diff	-	-	-	-	-	-	-	-	0.046	0.041
kda_15	-	-	-	-	-	-	-	-	-	0.051

```
fwd_signif_summary <- fwd_signif_path %>%
  group_by(feature) %>%
  summarise(
    added_at = min(step),
    max_p = max(p_value),
    final_p = p_value[step == max(step)],
    ever_lost = any(p_value > 0.05),
    .groups = "drop"
  ) %>%
  mutate(feature = factor(feature, levels = fwd_addition_order)) %>%
  arrange(feature)

fwd_signif_display <- fwd_signif_summary %>%
  transmute(
    feature,
    added_at,
    max_p_value = ifelse(max_p < 0.001, "<0.001", sprintf("%.3f", max_p)),
    final_p_value = ifelse(final_p < 0.001, "<0.001", sprintf("%.3f", final_p)),
    ever_above_0.05 = ifelse(ever_lost, "yes", "no")
  )

knitr::kable(
  fwd_signif_display,
  caption = "Per-feature significance summary along the forward-stepwise addition path",
  align = c("l", "r", "r", "r", "r")
)
```

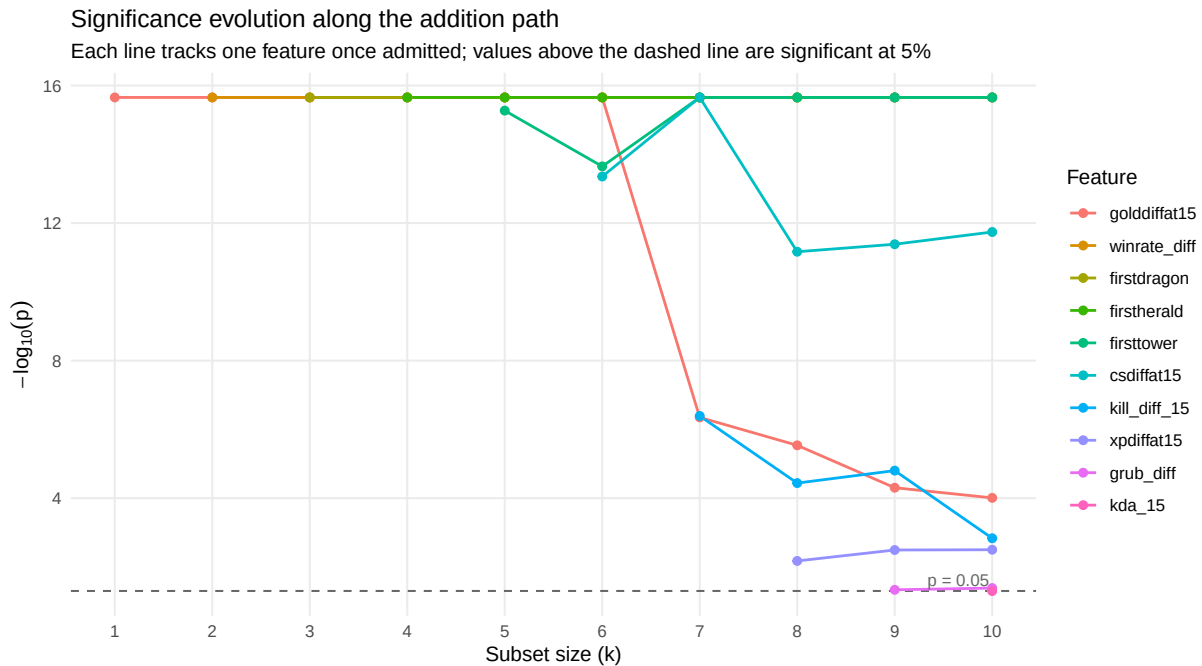
Table 10: Per-feature significance summary along the forward-stepwise addition path

feature	added_at	max_p_value	final_p_value	ever_above_0.05
golddiffat15	1	<0.001	<0.001	no

feature	added_at	max_p_value	final_p_value	ever_above_0.05
winrate_diff	2	<0.001	<0.001	no
firstdragon	3	<0.001	<0.001	no
firstherald	4	<0.001	<0.001	no
firsttower	5	<0.001	<0.001	no
csdiffat15	6	<0.001	<0.001	no
kill_diff_15	7	0.001	0.001	no
xpdiffat15	8	0.007	0.003	no
grub_diff	9	0.046	0.041	no
kda_15	10	0.051	0.051	yes

```
fwd_signif_plot_df <- fwd_signif_path %>%
  mutate(
    feature = factor(feature, levels = fwd_addition_order),
    neg_log_p = -log10(pmax(p_value, .Machine$double.eps))
  )

ggplot(fwd_signif_plot_df, aes(x = step, y = neg_log_p, color = feature, group = feature)) +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed", color = "grey40") +
  geom_line(linewidth = 0.7) +
  geom_point(size = 1.8) +
  annotate("text",
    x = max(fwd_signif_plot_df$step), y = -log10(0.05),
    label = "p = 0.05", hjust = 1.05, vjust = -0.4, color = "grey40", size = 3.2
  ) +
  scale_x_continuous(breaks = seq_len(fwd_n)) +
  labs(
    title = "Significance evolution along the addition path",
    subtitle = "Each line tracks one feature once admitted; values above the dashed line",
    x = "Subset size (k)",
    y = expression(-log[10](p)),
    color = "Feature"
  ) +
  theme_minimal() +
  theme(
    legend.position = "right",
    panel.grid.minor = element_blank()
  )
```

Most features enter well above the 0.05 line and stay there. Two show clear decay once their collinear partner enters: `golddiffat15` drops at $k = 6$ (joined by `csdiffat15`), and `kill_diff_15` drops at $k = 8$ (joined by `xpdiffat15`) - both still finish significant. Only `kda_15`, admitted last, ends below the threshold ($p = 0.051$): AIC accepts it for joint fit, but its standalone evidence is borderline.

LASSO (L1)

LASSO adds an L1 penalty to logistic regression, shrinking each coefficient toward zero proportionally to λ and driving weak ones to exactly zero - selection and estimation happen in the same step. It is directly motivated by the VIF findings from S1: features like `assist_diff_15` and `kill_diff_15` share most of their signal with `golddiffat15`, and L1 is the standard way to resolve that redundancy.

`lambda` is tuned with `cv.glmnet` (5-fold CV over ~ 100 values). We select `lambda.1se` - the largest λ within one standard error of the CV minimum - because it gives a sparser model statistically indistinguishable from the best-fitting one.

```
t0 <- Sys.time()

set.seed(42)
lasso_cv <- cv.glmnet(
  x      = as.matrix(X_train_scaled),
```

```

    y          = y_train,
    alpha      = 1,
    family     = "binomial",
    nfolds     = 5,
    type.measure = "auc"
)

# lambda.1se: the largest lambda within 1 SE of the minimum - gives a sparser model
# that is nearly as accurate as lambda.min but keeps fewer features
lasso_coefs <- coef(lasso_cv, s = "lambda.1se")

lasso_coef_df <- data.frame(
  feature = rownames(lasso_coefs),
  coefficient = as.numeric(lasso_coefs)
) %>%
  filter(feature != "(Intercept)") %>%
  arrange(desc(abs(coefficient)))

lasso_selected <- lasso_coef_df %>% filter(coefficient != 0)
lasso_eliminated <- lasso_coef_df %>% filter(coefficient == 0)

lasso_pred_prob <- predict(lasso_cv,
  newx = as.matrix(X_test_scaled),
  s = "lambda.1se", type = "response"
)
lasso_pred_class <- factor(ifelse(lasso_pred_prob > 0.5, "Win", "Loss"),
  levels = c("Loss", "Win")
)
lasso_cm <- confusionMatrix(lasso_pred_class, y_test_fct, positive = "Win")
lasso_roc <- roc(y_test_fct, as.numeric(lasso_pred_prob),
  levels = c("Loss", "Win"), quiet = TRUE
)

cat(sprintf(
  "\nLASSO Results (%.1fs):\nLambda (1se): %.6f\nFeatures kept: %d / %d (eliminated %d)\nAUC: %.4f\n",
  as.numeric(Sys.time() - t0, units = "secs"),
  lasso_cv$lambda.1se,
  nrow(lasso_selected), nrow(lasso_coef_df), nrow(lasso_eliminated),
  lasso_cm$overall["Accuracy"] * 100,
  lasso_cm$byClass["Sensitivity"] * 100,
  lasso_cm$byClass["Specificity"] * 100,
  auc(lasso_roc)
))

```

```
))
```

```
LASSO Results (0.8s):  
Lambda (1se): 0.040387  
Features kept: 7 / 14 (eliminated 7)  
Accuracy: 75.0%  
Sensitivity (Win recall): 79.5%  
Specificity (Loss recall): 69.5%  
AUC-ROC: 0.839
```

```
cat("\nKept features:\n")
```

Kept features:

```
cat(paste(lasso_selected$feature, collapse = ", "), "\n")
```

golddiffat15, xpdiffat15, csdiffat15, firsttower, winrate_diff, firsttherald, firstdragon

```
cat("\nEliminated features:\n")
```

Eliminated features:

```
cat(paste(lasso_eliminated$feature, collapse = ", "), "\n")
```

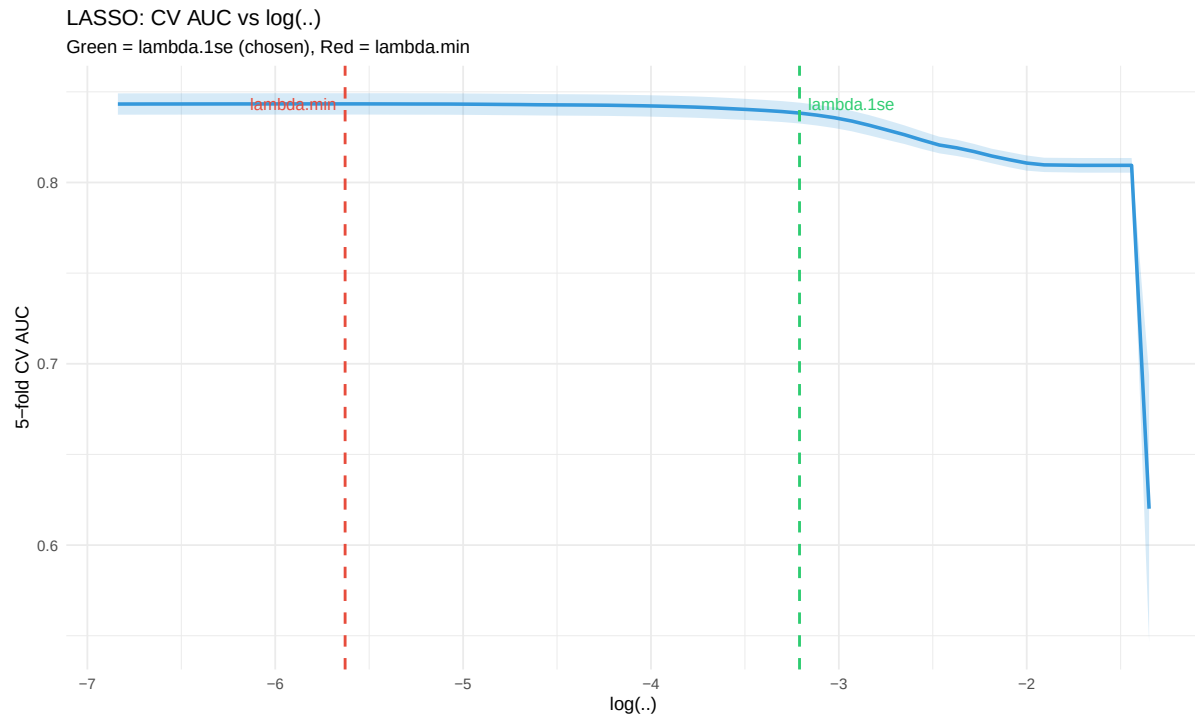
kill_diff_15, assist_diff_15, gold_efficiency_15, kda_15, plate_diff, grub_diff, firstblood

```
cv_df <- data.frame(  
  log_lambda = log(lasso_cv$lambda),  
  cvm        = lasso_cv$cvm,  
  cvsd       = lasso_cv$cvsd  
)  
  
ggplot(cv_df, aes(x = log_lambda, y = cvm)) +  
  geom_ribbon(aes(ymin = cvm - cvsd, ymax = cvm + cvsd),  
    fill = "#3498db", alpha = 0.2
```

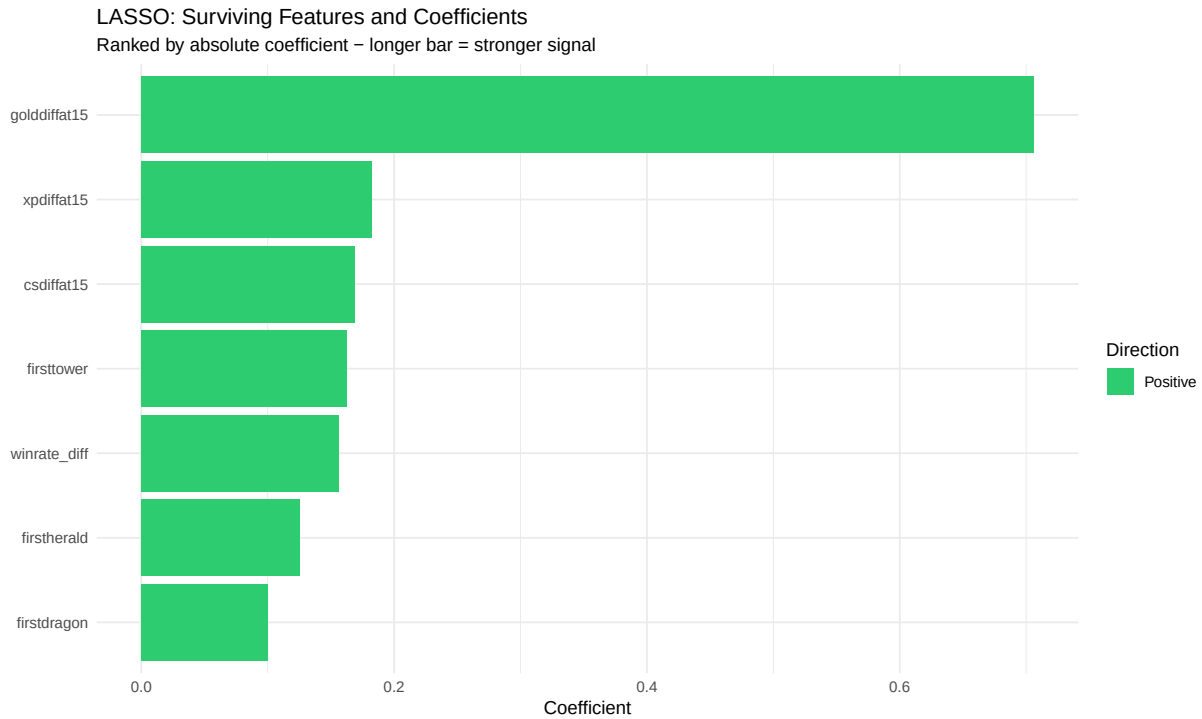
```

) +
geom_line(color = "#3498db", linewidth = 1) +
geom_vline(
  xintercept = log(lasso_cv$lambda.min),
  linetype = "dashed", color = "#e74c3c", linewidth = 0.8
) +
geom_vline(
  xintercept = log(lasso_cv$lambda.1se),
  linetype = "dashed", color = "#2ecc71", linewidth = 0.8
) +
annotate("text",
  x = log(lasso_cv$lambda.min), y = max(cv_df$cvm),
  label = "lambda.min", hjust = 1.1, color = "#e74c3c", size = 3.5
) +
annotate("text",
  x = log(lasso_cv$lambda.1se), y = max(cv_df$cvm),
  label = "lambda.1se", hjust = -0.1, color = "#2ecc71", size = 3.5
) +
labs(
  title = "LASSO: CV AUC vs log()",
  subtitle = "Green = lambda.1se (chosen), Red = lambda.min",
  x = "log()", y = "5-fold CV AUC"
) +
theme_minimal()

```



```
lasso_selected %>%
  mutate(feature = reorder(feature, abs(coefficient))) %>%
  ggplot(aes(x = feature, y = coefficient, fill = coefficient > 0)) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(
    values = c("TRUE" = "#2ecc71", "FALSE" = "#e74c3c"),
    breaks = c("TRUE", "FALSE"),
    labels = c("Positive", "Negative")
  ) +
  labs(
    title = "LASSO: Surviving Features and Coefficients",
    subtitle = "Ranked by absolute coefficient - longer bar = stronger signal",
    x = NULL, y = "Coefficient", fill = "Direction"
  ) +
  theme_minimal()
```



LASSO retained 7 of the 14 features at $\text{lambda.1se} = 0.040$ and achieved AUC-ROC of 0.839 with 75.0% accuracy - within 0.3 pp of the full-model baseline while halving the feature count. We chose lambda.1se because it sits at the point where the curve begins to rise - the last position where a sparser model is statistically indistinguishable from the best-fitting one.

The combat features `kill_diff_15` and `assist_diff_15` were zeroed out, consistent with the VIF analysis: kills and assists translate directly into team gold, so their signal is already captured by `golddiffat15` once L1 pressure is applied. The two ratio features were also eliminated - `gold_efficiency_15 = goldat15 / csat15` overlaps with `golddiffat15` and `csdiffat15`, and `kda_15` overlaps with the combat differentials. `plate_diff` and `grub_diff` are absorbed by the dominant economic core; their univariate signal is real but redundant. `firstblood` was also eliminated - consistent with the EDA, securing the first kill adds no predictive value once the full 15-minute economic picture is present.

The 7 retained features form a coherent minimal set: the three resource differentials (`golddiffat15`, `xpdiffat15`, `csdiffat15`) capture the economic picture at 15 minutes, `winrate_diff` adds pre-game team quality, and the three first-objective flags (`firsttower`, `firstdragon`, `firstherald`) contribute a separate structural layer. All surviving coefficients are positive: being ahead in any of these dimensions increases the Blue team's win probability.

Elastic Net (L1 + L2)

We apply Elastic Net as the second embedded method because our predictors include groups of correlated variables that may still carry complementary information. LASSO can be unstable under strong collinearity - it may arbitrarily keep one variable from a correlated group and zero the rest - which is a concern given the resource differential cluster in our data.

Elastic Net combines L1 and L2 regularisation, allowing it to perform feature selection while also handling correlated predictors more smoothly than pure LASSO. This makes it useful for checking whether features removed by LASSO are truly redundant, or whether they still contribute secondary signal when correlated variables are allowed to share predictive weight.

The blend between L1 and L2 is controlled by `alpha` (0 = pure Ridge, 1 = pure LASSO). We search the full alpha range in 0.05 steps, using the same fixed cross-validation folds across all alphas so the resulting CV AUC values are directly comparable. The alpha whose `lambda.1se` model maximises 5-fold CV AUC is selected as the final model; if the AUC curve is flat across alphas, the data cannot distinguish LASSO from Ridge at this sample size and the L1/L2 mix is effectively underdetermined.

```
t0 <- Sys.time()

alpha_grid <- seq(0, 1, by = 0.05)

# Fix fold assignments across all alphas so cv_auc values are directly comparable
set.seed(42)
fold_ids <- sample(rep(seq(5), length.out = length(y_train)))

enet_cv_results <- map_dfr(alpha_grid, function(a) {
  cv_fit <- cv.glmnet(
    x      = as.matrix(X_train_scaled),
    y      = y_train,
    alpha  = a,
    family = "binomial",
    foldid = fold_ids,
    type.measure = "auc"
  )
  idx_1se <- cv_fit$index["1se", 1]
  n_kept <- sum(coef(cv_fit, s = "lambda.1se") != 0) - 1
  tibble(
    alpha      = a,
    lambda_1se = cv_fit$lambda.1se,
    cv_auc     = cv_fit$cvm[idx_1se],
    cv_auc_se  = cv_fit$cvstd[idx_1se],
```

```

      n_features = n_kept
    )
  })

best_row <- enet_cv_results %>%
  arrange(desc(cv_auc)) %>%
  dplyr::slice(1)
best_alpha <- best_row$alpha
best_auc <- best_row$cv_auc
best_se <- best_row$cv_auc_se
within_1se <- enet_cv_results %>% filter(cv_auc >= best_auc - best_se)

cat(sprintf(
  "\nAlpha grid diagnostic: %d of %d alphas sit within 1 SE of the best CV AUC (0.3f >= 0.3f).",
  nrow(within_1se), nrow(enet_cv_results), best_auc, best_auc - best_se
))

```

Alpha grid diagnostic: 21 of 21 alphas sit within 1 SE of the best CV AUC (0.839 >= 0.833).

```

cat(sprintf(
  "Alphas within 1 SE: %s\n",
  paste(sprintf("%.2f", within_1se$alpha), collapse = ", ")
))

```

Alphas within 1 SE: 0.00, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95, 1.00

```

if (nrow(within_1se) >= nrow(enet_cv_results) - 2) {
  cat("-> CV AUC curve is effectively flat across alpha; the L1/L2 mix is underdetermined by this data")
}

```

-> CV AUC curve is effectively flat across alpha; the L1/L2 mix is underdetermined by this data

```

cat(sprintf("Best alpha: %.2f (CV AUC = %.4f)\n", best_alpha, best_auc))

```

Best alpha: 0.90 (CV AUC = 0.8386)


```

enet_cv_final <- cv.glmnet(
  x      = as.matrix(X_train_scaled),
  y      = y_train,
  alpha  = best_alpha,
  family = "binomial",
  foldid = fold_ids,
  type.measure = "auc"
)

enet_coef_raw <- coef(enet_cv_final, s = "lambda.1se")
enet_coef_df <- data.frame(
  feature      = rownames(enet_coef_raw),
  coefficient   = as.numeric(enet_coef_raw)
) %>%
  filter(feature != "(Intercept)") %>%
  arrange(desc(abs(coefficient)))

enet_selected_df <- enet_coef_df %>% filter(coefficient != 0)
enet_selected <- enet_selected_df$feature

enet_pred_prob <- predict(enet_cv_final,
  newx = as.matrix(X_test_scaled),
  s = "lambda.1se", type = "response"
)
enet_pred_class <- factor(ifelse(enet_pred_prob > 0.5, "Win", "Loss"),
  levels = c("Loss", "Win")
)
enet_cm <- confusionMatrix(enet_pred_class, y_test_fct, positive = "Win")
enet_roc <- roc(y_test_fct, as.numeric(enet_pred_prob), levels = c("Loss", "Win"), quiet = TRUE)

cat(sprintf(
  "\nElastic Net Results (%.1fs):\nBest alpha: %.2f (0=Ridge, 1=LASSO)\nLambda (1se): %.6f\n",
  as.numeric(Sys.time() - t0, units = "secs"),
  best_alpha,
  enet_cv_final$lambda.1se,
  nrow(enet_selected_df), nrow(enet_coef_df), nrow(enet_coef_df) - nrow(enet_selected_df),
  enet_cm$overall["Accuracy"] * 100,
  enet_cm$byClass["Sensitivity"] * 100,
  enet_cm$byClass["Specificity"] * 100,
  auc(enet_roc)
))

```

Elastic Net Results (17.5s):
 Best alpha: 0.90 (0=Ridge, 1=LASSO)
 Lambda (1se): 0.044874
 Features kept: 8 / 14 (eliminated 6)
 Accuracy: 75.2%
 Sensitivity (Win recall): 79.8%
 Specificity (Loss recall): 69.5%
 AUC-ROC: 0.840

```
cat("\nKept features:\n")
```

Kept features:

```
cat(paste(enet_selected, collapse = ", "), "\n")
```

golddiffat15, xpdiffat15, csdiffat15, firsttower, winrate_diff, firstherald, firstdragon, ki

```
cat("\nEliminated features:\n")
```

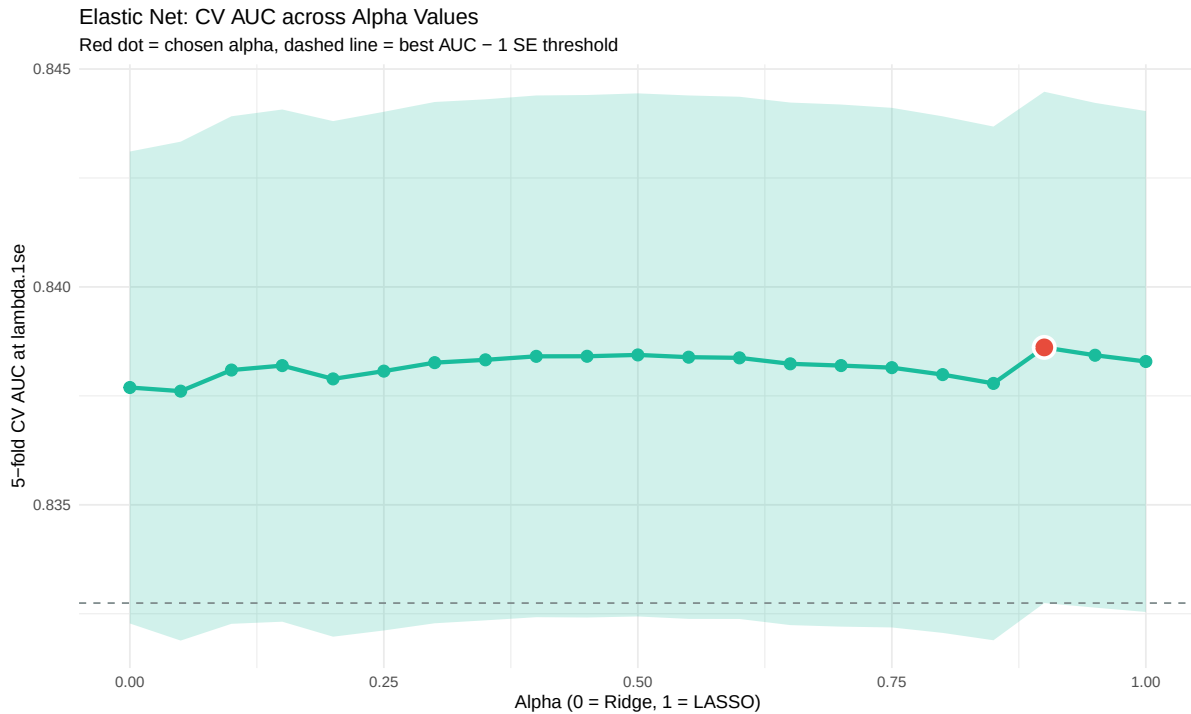
Eliminated features:

```
cat(paste(enet_coef_df %>% filter(coefficient == 0) %>% pull(feature), collapse = ", "), "\n")
```

assist_diff_15, gold_efficiency_15, kda_15, plate_diff, grub_diff, firstblood

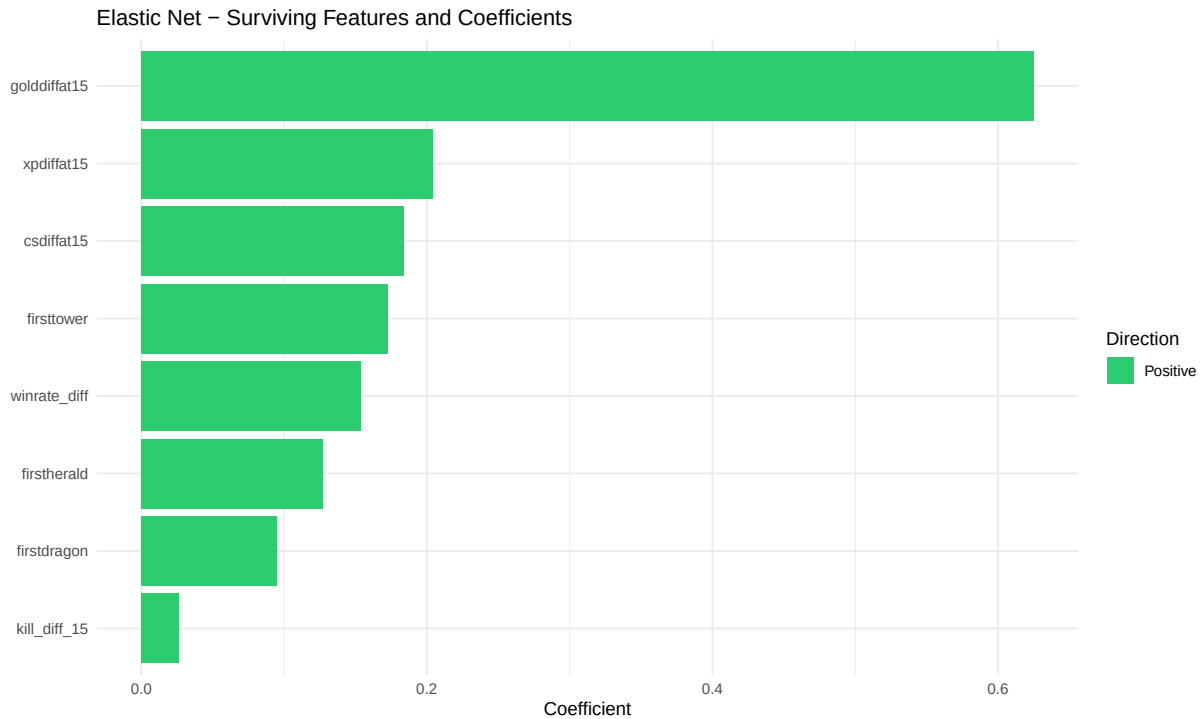
```
ggplot(enet_cv_results, aes(x = alpha, y = cv_auc)) +
  geom_ribbon(aes(ymin = cv_auc - cv_auc_se, ymax = cv_auc + cv_auc_se),
    fill = "#1abc9c", alpha = 0.2
  ) +
  geom_hline(yintercept = best_auc - best_se, linetype = "dashed", color = "#7f8c8d") +
  geom_line(color = "#1abc9c", linewidth = 1.2) +
  geom_point(size = 3, color = "#1abc9c") +
  geom_point(
    data = filter(enet_cv_results, alpha == best_alpha),
    size = 5, shape = 21, fill = "#e74c3c", color = "white", stroke = 1.5
  ) +
```

```
labs(
  title = "Elastic Net: CV AUC across Alpha Values",
  subtitle = "Red dot = chosen alpha, dashed line = best AUC - 1 SE threshold",
  x = "Alpha (0 = Ridge, 1 = LASSO)", y = "5-fold CV AUC at lambda.1se"
) +
theme_minimal()
```



```
enet_selected_df %>%
  mutate(feature = reorder(feature, abs(coefficient))) %>%
  ggplot(aes(x = feature, y = coefficient, fill = coefficient > 0)) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(
    values = c("TRUE" = "#2ecc71", "FALSE" = "#e74c3c"),
    breaks = c("TRUE", "FALSE"),
    labels = c("Positive", "Negative")
  ) +
  labs(
    title = "Elastic Net - Surviving Features and Coefficients",
    x = NULL, y = "Coefficient", fill = "Direction"
  ) +
```

```
theme_minimal()
```



The CV AUC curve is almost flat across the entire alpha grid - all 21 tested values sit within 1 SE of the best AUC (0.839), and the wide confidence band suggests that differences between alphas are noise rather than signal. The dashed line marks the best AUC - 1 SE threshold; every alpha clears it comfortably, meaning no mixing ratio offers a statistically meaningful advantage over any other.

$\alpha = 0.90$ (near-LASSO) achieves the highest point estimate, but the margin over other alphas is negligible. The correlated resource differentials do not cause the instability Elastic Net is designed to address - LASSO handles them as well as any Ridge blend at this sample size. Elastic Net keeps one feature beyond the LASSO core (`kill_diff_15`, the partner of the high-VIF `golddiffat15` pair), reflecting the slightly weaker L1 pressure at $\alpha = 0.90$. The downstream LR fits on the LASSO and Elastic Net subsets land within 0.05 pp AUC of each other, confirming that this 8th feature carries no predictive value beyond the consensus 7.

```
s3_best_hyperparams <- tibble(  
  Method = c("Forward Stepwise", "LASSO", "Elastic Net"),  
  Category = c("Algorithmic", "Embedded", "Embedded"),  
  `Hyperparameters` = c(  
    sprintf("AIC criterion, final size = %d features", fwd_n),  
    sprintf("lambda.1se = %.5f", lasso_cv$lambda.1se),  
  )  
)
```

```

    sprintf("alpha = %.2f, lambda.1se = %.5f", best_alpha, enet_cv_final$lambda.1se)
  ),
  `Features kept` = c(
    fwd_n,
    length(lasso_selected$feature),
    length(enet_selected)
  ),
  `AUC-ROC` = c(
    sprintf("%.3f", auc(fwd_roc)),
    sprintf("%.3f", auc(lasso_roc)),
    sprintf("%.3f", auc(enet_roc))
  )
)

knitr::kable(s3_best_hyperparams,
  format      = "html",
  caption      = "Best hyperparameters per S3 selection method",
  table.attr = 'style="border-collapse: collapse; width: 100%;"'
) |>
  gsub(pattern = "<th>", replacement = '<th style="padding: 8px 16px; background:#2c3e50; color:white;">')
  gsub(pattern = "<td>", replacement = '<td style="padding: 6px 16px; border-bottom: 1px solid black;">')
  htmltools::HTML() |>
  htmltools::div()

```

Table 11: Best hyperparameters per S3 selection method

Method	Category	Hyperparameters	Features kept	AUC-ROC
Forward Stepwise	Algorithmic	AIC criterion, final size = 10 features	10	0.842
LASSO	Embedded	lambda.1se = 0.04039	7	0.839
Elastic Net	Embedded	alpha = 0.90, lambda.1se = 0.04487	8	0.840

Feature selection comparison

```

selection_results <- list(
  Forward      = fwd_selected,
  LASSO        = lasso_selected$feature,
  ElasticNet   = enet_selected
)

```

```

all_features <- colnames(X_train_scaled)

overlap_df <- map_dfr(all_features, function(f) {
  sel <- map_lgl(selection_results, ~ f %in% .x)
  tibble(
    feature      = f,
    Forward      = sel[["Forward"]],
    LASSO        = sel[["LASSO"]],
    ElasticNet   = sel[["ElasticNet"]],
    n_methods    = sum(sel)
  )
}) %>%
  arrange(desc(n_methods), feature)

consensus_3 <- overlap_df %>% filter(n_methods == 3)
consensus_2 <- overlap_df %>% filter(n_methods == 2)
never_sel <- overlap_df %>% filter(n_methods == 0)

cat("Feature selection summary across all 3 methods:\n")

```

Feature selection summary across all 3 methods:

```
cat(sprintf("  Total features:                %d\n", length(all_features)))
```

```
Total features:                14
```

```
cat(sprintf("  Selected by all 3 (consensus):  %d\n", nrow(consensus_3)))
```

```
Selected by all 3 (consensus):  7
```

```
cat(sprintf("  Selected by 2 methods:                %d\n", nrow(consensus_2)))
```

```
Selected by 2 methods:                1
```

```
cat(sprintf("  Never selected:                %d\n\n", nrow(never_sel)))
```

```
Never selected:                4
```

```
cat("Consensus features (all 3 methods agree):\n")
```

Consensus features (all 3 methods agree):

```
if (nrow(consensus_3) > 0) {  
  cat(paste(consensus_3$feature, collapse = ", "), "\n")  
} else {  
  cat("None\n")  
}
```

csdiffat15, firstdragon, firstherald, firsttower, golddiffat15, winrate_diff, xpdiffat15

```
cat("\nFeatures selected by 2 methods:\n")
```

Features selected by 2 methods:

```
if (nrow(consensus_2) > 0) {  
  cat(paste(consensus_2$feature, collapse = ", "), "\n")  
} else {  
  cat("None\n")  
}
```

kill_diff_15

```
cat("\nFeatures never selected by any method:\n")
```

Features never selected by any method:

```
if (nrow(never_sel) > 0) {  
  cat(paste(never_sel$feature, collapse = ", "), "\n")  
} else {  
  cat("None - every feature was selected by at least one method\n")  
}
```

assist_diff_15, firstblood, gold_efficiency_15, plate_diff

```

perf_summary <- tibble(
  Method = c("Forward", "LASSO", "ElasticNet", "LR_all"),
  n_features = c(
    fwd_n,
    nrow(lasso_selected),
    nrow(enet_selected_df),
    ncol(X_train_scaled)
  ),
  Accuracy = c(
    fwd_cm$overall["Accuracy"] * 100,
    lasso_cm$overall["Accuracy"] * 100,
    enet_cm$overall["Accuracy"] * 100,
    lr_cm$overall["Accuracy"] * 100
  ),
  AUC_ROC = c(
    auc(fwd_roc),
    auc(lasso_roc),
    auc(enet_roc),
    auc(roc_lr)
  ) * 100
)

cat("\nPerformance comparison (sorted by AUC-ROC):\n")

```

Performance comparison (sorted by AUC-ROC):

```

print(perf_summary %>% arrange(desc(AUC_ROC)), n = Inf)

```

```

# A tibble: 4 x 4
  Method      n_features Accuracy AUC_ROC
  <chr>         <int>     <dbl>   <dbl>
1 Forward         10      75.6    84.2
2 LR_all          14      75.5    84.2
3 ElasticNet       8      75.2    84.0
4 LASSO           7       75     83.9

```

```

overlap_long <- overlap_df %>%
  filter(n_methods > 0) %>%
  pivot_longer(

```

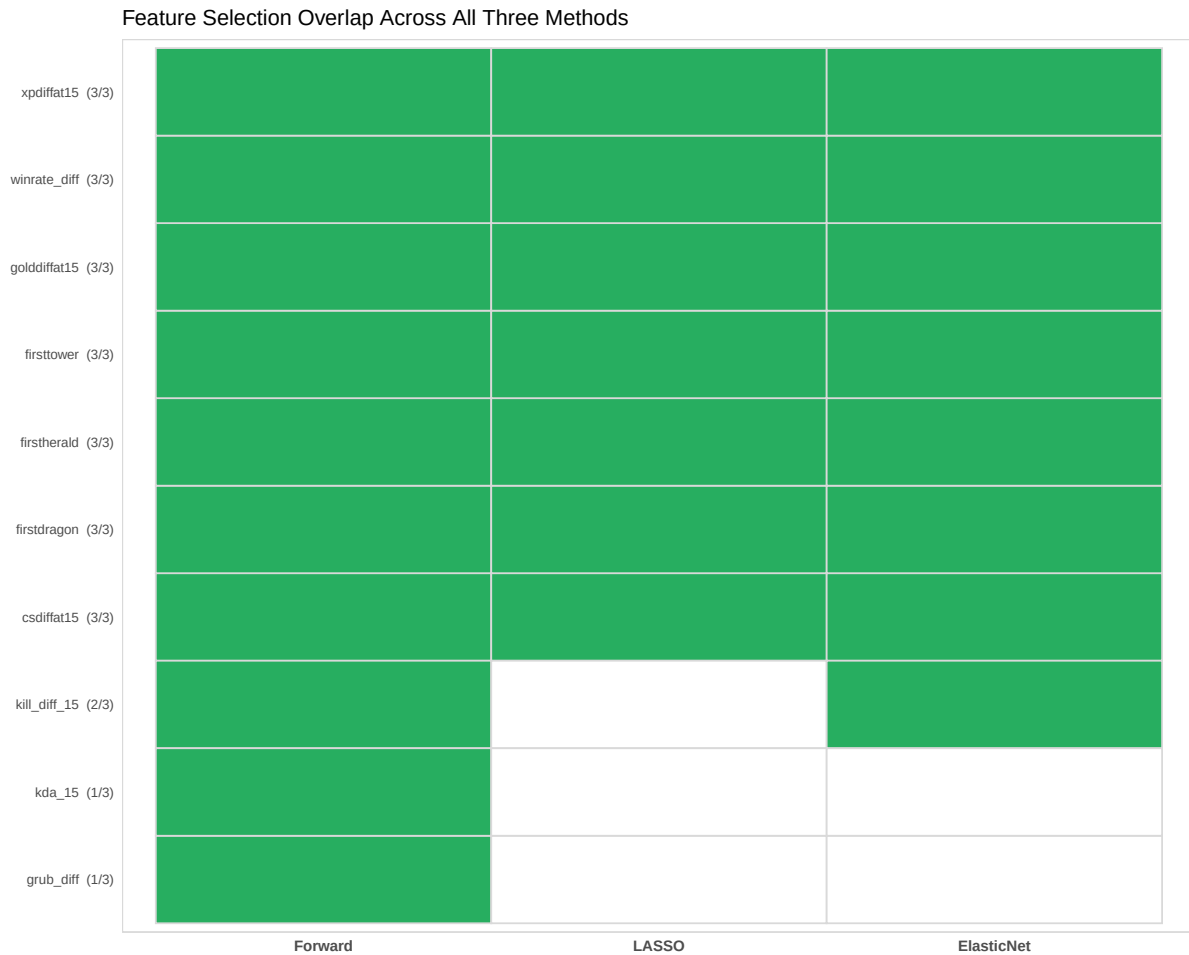


```

      cols      = c(Forward, LASSO, ElasticNet),
      names_to  = "method",
      values_to = "selected"
    ) %>%
    mutate(
      feature_label = paste0(feature, "  (", n_methods, "/3)",
      feature_label = reorder(feature_label, n_methods),
      method        = factor(method, levels = c("Forward", "LASSO", "ElasticNet"))
    )

ggplot(overlap_long, aes(x = method, y = feature_label, fill = selected)) +
  geom_tile(color = "grey85", linewidth = 0.5) +
  scale_fill_manual(
    values = c("TRUE" = "#27ae60", "FALSE" = "white"),
    labels = c("Not selected", "Selected"),
    guide  = "none"
  ) +
  labs(
    title = "Feature Selection Overlap Across All Three Methods",
    x = NULL, y = NULL
  ) +
  theme_minimal() +
  theme(
    axis.text.y      = element_text(size = 8),
    axis.text.x      = element_text(size = 9, face = "bold"),
    panel.grid       = element_blank(),
    panel.border     = element_rect(color = "grey85", fill = NA)
  )

```



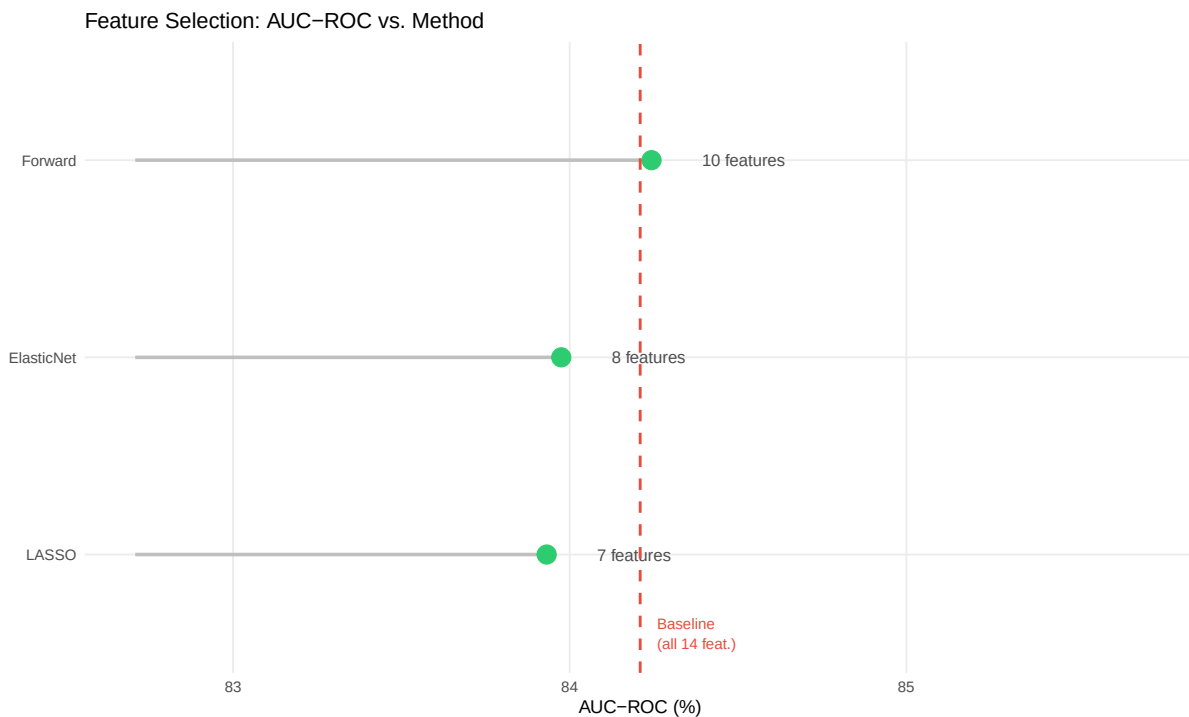
```
baseline_auc <- perf_summary$AUC_ROC[perf_summary$Method == "LR_all"]

perf_summary %>%
  filter(Method != "LR_all") %>%
  mutate(Method = reorder(Method, AUC_ROC)) %>%
  ggplot(aes(x = AUC_ROC, y = Method)) +
  geom_segment(aes(x = baseline_auc - 1.5, xend = AUC_ROC, yend = Method),
    color = "grey75", linewidth = 1
  ) +
  geom_point(size = 5, color = "#2ecc71") +
  geom_text(aes(label = sprintf("%d features", n_features)),
    nudge_x = 0.15, hjust = 0, size = 3.5, color = "grey30"
  ) +
  geom_vline(
    xintercept = baseline_auc, linetype = "dashed",
```

```

    color = "#e74c3c", linewidth = 0.8
) +
annotate("text",
  x = baseline_auc + 0.05, y = 0.6,
  label = sprintf("Baseline\n(all %d feat.)", ncol(X_train_scaled)),
  color = "#e74c3c", size = 3, hjust = 0
) +
scale_x_continuous(limits = c(baseline_auc - 1.5, baseline_auc + 1.5)) +
labs(
  title = "Feature Selection: AUC-ROC vs. Method",
  x = "AUC-ROC (%)", y = NULL
) +
theme_minimal() +
theme(panel.grid.minor = element_blank())

```



All three methods land within 0.3 pp AUC of the 14-feature LR baseline (84.2%) while using between 7 and 10 features: Forward 10 features at AUC 84.2%, ElasticNet 8 features at AUC 84.0%, LASSO 7 features at AUC 83.9%. The leanest model loses 0.3 pp AUC for half the feature count, so the four-to-seven features that the more aggressive selectors drop add at most 0.3 pp of joint signal.

S3 findings

Seven features are picked by all three methods, the consensus core: `golddiffat15`, `xpdiffat15`, `csdiffat15`, `firsttower`, `firstdragon`, `firsttherald`, `winrate_diff`. This matches the EDA correlation ranking and the S1 cross-model importance consensus exactly. The three resource differentials at 15 minutes carry the largest share of signal - who is ahead in gold, XP, and CS is the dominant predictor. The three first-objective flags add a separate structural layer; first tower and first herald showed the strongest binary win-rate correlations in the EDA, and all three flags survive every selection criterion here. `winrate_diff` adds pre-game team quality independent of the in-game snapshot.

One feature falls in the “exactly two methods” band. `kill_diff_15` is kept by Forward and Elastic Net but dropped by pure LASSO. It is the partner of the high-VIF cluster identified in S1: its standalone signal is real (Wald $p < 0.001$ in the forward fit), but most of it is captured by `golddiffat15` once a strict L1 penalty is applied. Elastic Net’s small Ridge blend at $\lambda = 0.90$ spares it; pure LASSO does not. Forward keeps it because admitting it lowers AIC by ~26 units - a clear improvement, even though the same predictive content can be cheaply absorbed by `golddiffat15` under regularisation.

Two features are kept only by Forward: `grub_diff` and `kda_15`. Both are admitted at the very end of the addition path, with deviance drops just below four units each. The p-value evolution table shows that `grub_diff` retains marginal significance (final $p = 0.041$) while `kda_15` slips just over the threshold (final $p = 0.051$) - both sit at the boundary of the AIC criterion and would not survive an L1 penalty.

Four features are never selected by any method: `assist_diff_15`, `firstblood`, `gold_efficiency_15`, `plate_diff`. Their coefficients in the unpenalised LR fit are small, and they offer no improvement strong enough to clear AIC, L1, or the mixed L1/L2 criterion. `firstblood` is the strongest null result: despite its prominence in esports commentary, its 400 gold value is negligible relative to a 15-minute lead, and the EDA already showed it produced the weakest binary win-rate shift of any objective flag. `assist_diff_15` overlaps with the combat-and-economy core already covered by `golddiffat15` and `kill_diff_15`; `gold_efficiency_15` is a ratio of two features already in the model; `plate_diff` is an early-game objective signal already absorbed by the resource lead it helped produce.

Validation: best parametric S1 model on the S3-selected feature subsets

We refit Logistic Regression - the top S1 parametric learner (test AUC 0.842) - on each S3-selected subset to check whether the choice of selection method matters downstream. Same `caret` setup as S1 (5-fold CV, `metric = "ROC"`, scaled inputs); only the feature matrix changes.

```

t0 <- Sys.time()

fit_lr_subset <- function(features) {
  X_tr <- X_train_scaled[, features, drop = FALSE]
  X_te <- X_test_scaled[, features, drop = FALSE]
  set.seed(42)
  fit <- train(
    x      = X_tr,
    y      = y_train_fct,
    method = "glm",
    family  = "binomial",
    trControl = ctrl,
    metric   = "ROC"
  )
  pred <- predict(fit, X_te)
  prob <- predict(fit, X_te, type = "prob")
  cm <- confusionMatrix(pred, y_test_fct, positive = "Win")
  roc_obj <- roc(y_test_fct, prob[, "Win"],
    levels = c("Loss", "Win"), quiet = TRUE
  )
  list(fit = fit, cm = cm, roc = roc_obj, n = length(features))
}

lr_fwd <- fit_lr_subset(fwd_selected)
lr_lasso <- fit_lr_subset(lasso_selected$feature)
lr_enet <- fit_lr_subset(enet_selected)

cat(sprintf(
  "\nFitted three reduced-feature LR variants in %.1fs.\n",
  as.numeric(Sys.time() - t0, units = "secs")
))

```

Fitted three reduced-feature LR variants in 0.4s.

```

validation_compare <- tibble(
  Variant = c(
    sprintf("LR (full %d features)", ncol(X_train)),
    sprintf("LR (Forward %d features)", lr_fwd$n),
    sprintf("LR (LASSO %d features)", lr_lasso$n),
    sprintf("LR (Elastic Net %d features)", lr_enet$n)
  )
)

```

```

),
Features = c(ncol(X_train), lr_fwd$n, lr_lasso$n, lr_enet$n),
Accuracy = c(lr_cm$overall["Accuracy"], lr_fwd$cm$overall["Accuracy"], lr_lasso$cm$overall["Accuracy"], lr_enet$cm$overall["Accuracy"]),
Sensitivity = c(lr_cm$byClass["Sensitivity"], lr_fwd$cm$byClass["Sensitivity"], lr_lasso$cm$byClass["Sensitivity"], lr_enet$cm$byClass["Sensitivity"]),
Specificity = c(lr_cm$byClass["Specificity"], lr_fwd$cm$byClass["Specificity"], lr_lasso$cm$byClass["Specificity"], lr_enet$cm$byClass["Specificity"]),
AUC_ROC = c(auc(roc_lr), auc(lr_fwd$roc), auc(lr_lasso$roc), auc(lr_enet$roc))
)

knitr::kable(validation_compare,
  digits = c(0, 0, 2, 2, 2, 3),
  caption = "LR on the full 14-feature set vs LR on each S3-selected subset (same train/test split)"
)

```

Table 12: LR on the full 14-feature set vs LR on each S3-selected subset (same train/test split)

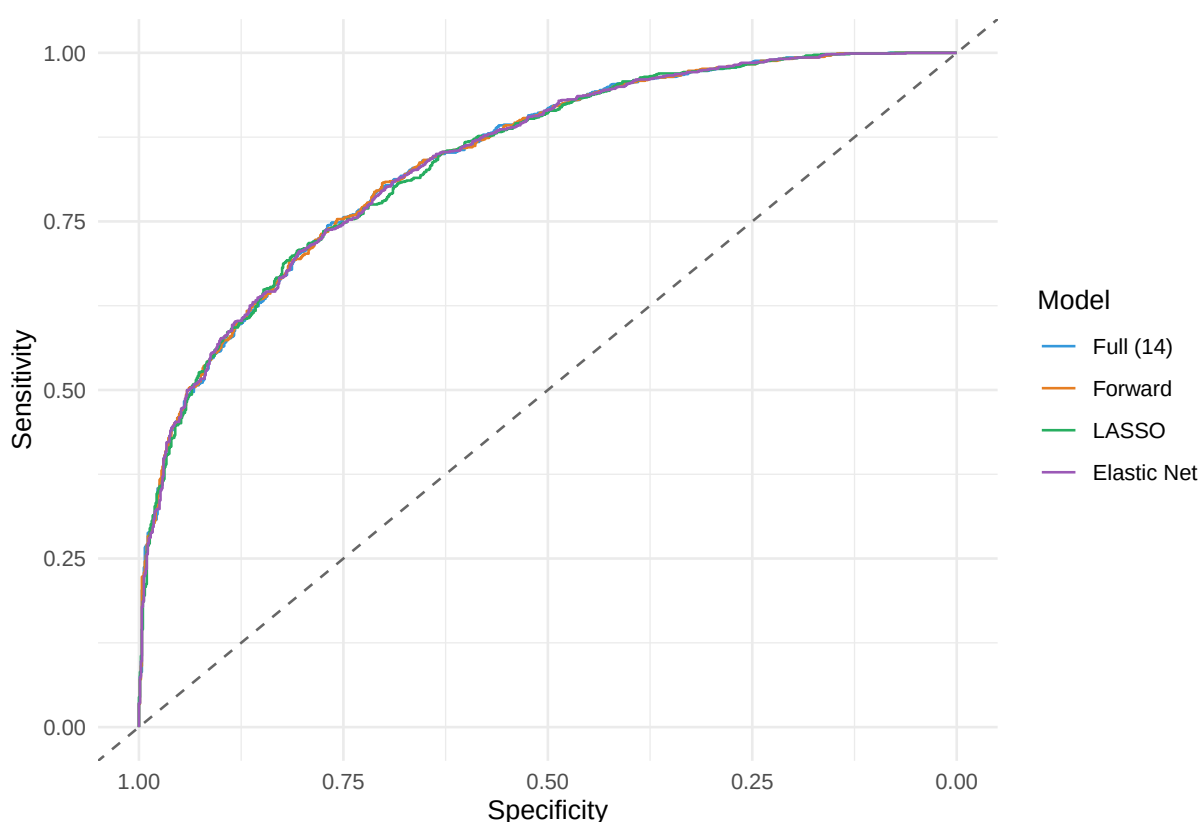
Variant	Features	Accuracy	Sensitivity	Specificity	AUC_ROC
LR (full 14 features)	14	75.54	79.09	71.22	0.842
LR (Forward 10 features)	10	75.65	79.49	70.98	0.842
LR (LASSO 7 features)	7	74.35	78.11	69.78	0.841
LR (Elastic Net 8 features)	8	75.27	79.29	70.38	0.842

```

ggroc(list(
  "Full (14)" = roc_lr,
  "Forward" = lr_fwd$roc,
  "LASSO" = lr_lasso$roc,
  "Elastic Net" = lr_enet$roc
)) +
  geom_abline(slope = 1, intercept = 1, linetype = "dashed", color = "gray40") +
  scale_color_manual(values = c(
    "Full (14)" = "#3498db",
    "Forward" = "#e67e22",
    "LASSO" = "#27ae60",
    "Elastic Net" = "#9b59b6"
  )) +
  labs(
    title = "ROC: full LR vs LR on each S3-selected subset",
    x = "Specificity", y = "Sensitivity", color = "Model"
  ) +
  theme_minimal()

```

ROC: full LR vs LR on each S3-selected subset



```
auc_delta_fwd <- (auc(lr_fwd$roc) - auc(roc_lr)) * 100
auc_delta_lasso <- (auc(lr_lasso$roc) - auc(roc_lr)) * 100
auc_delta_enet <- (auc(lr_enet$roc) - auc(roc_lr)) * 100

acc_delta_fwd <- (lr_fwd$cm$overall["Accuracy"] - lr_cm$overall["Accuracy"]) * 100
acc_delta_lasso <- (lr_lasso$cm$overall["Accuracy"] - lr_cm$overall["Accuracy"]) * 100
acc_delta_enet <- (lr_enet$cm$overall["Accuracy"] - lr_cm$overall["Accuracy"]) * 100

worst_auc_drop <- max(abs(c(auc_delta_fwd, auc_delta_lasso, auc_delta_enet)))
worst_acc_drop <- max(abs(c(acc_delta_fwd, acc_delta_lasso, acc_delta_enet)))

lasso_enet_identical <- identical(sort(lasso_selected$feature), sort(enet_selected))
```

All three reduced-feature LRs land within 0.3 pp AUC of the 14-feature baseline (0.842): Forward 0.842 (10 features), Elastic Net 0.840 (8), LASSO 0.839 (7). The four ROC curves overlap almost exactly, so any practical decision threshold transfers between subsets. The 7-feature LASSO model is the cheapest hit on the AUC ceiling, and the 0.3 pp gap to the full

model is small enough that parsimony - coefficient interpretability and drift-robustness - is the better trade.

Cross-validation grid: every S1 model on every S3 selection

We extend the LR-specific validation to all six S1 learners by refitting each on every S3 subset (plus the full 14-feature baseline) under the S1 tuning grids. Naive Bayes is the model of interest here: its independence assumption is the one most directly violated by the correlated 15-minute differentials, so reduction should help it more than any other learner.

```
t0 <- Sys.time()

fit_model_subset <- function(model_name, features) {
  X_tr <- X_train[, features, drop = FALSE]
  X_te <- X_test[, features, drop = FALSE]
  X_tr_s <- X_train_scaled[, features, drop = FALSE]
  X_te_s <- X_test_scaled[, features, drop = FALSE]
  n_feat <- length(features)

  if (model_name == "LR") {
    set.seed(42)
    fit <- train(
      x = X_tr_s, y = y_train_fct, method = "glm", family = "binomial",
      trControl = ctrl, metric = "ROC"
    )
    prob <- predict(fit, X_te_s, type = "prob")
    pred <- predict(fit, X_te_s)
  } else if (model_name == "RF") {
    mtry_grid <- unique(pmin(c(2, 5, 7, 10, 15, 19), n_feat))
    set.seed(42)
    fit <- train(
      x = X_tr, y = y_train_fct, method = "rf",
      trControl = ctrl, metric = "ROC",
      tuneGrid = expand.grid(mtry = mtry_grid), ntree = 500
    )
    prob <- predict(fit, X_te, type = "prob")
    pred <- predict(fit, X_te)
  } else if (model_name == "NB") {
    set.seed(42)
    fit <- train(
      x = X_tr, y = y_train_fct, method = "naive_bayes",
```



```

        trControl = ctrl, metric = "ROC",
        tuneGrid = expand.grid(
            usekernel = c(FALSE, TRUE), laplace = 0,
            adjust = c(0.5, 1.0, 1.5)
        )
    )
    prob <- predict(fit, X_te, type = "prob")
    pred <- predict(fit, X_te)
} else if (model_name == "KNN") {
    k_theo <- round(sqrt(nrow(X_tr_s)))
    k_grid <- unique(round(exp(seq(log(5), log(k_theo * 4), length.out = 12))))
    k_grid <- ifelse(k_grid %% 2 == 0, k_grid + 1, k_grid)
    set.seed(42)
    fit <- train(
        x = X_tr_s, y = y_train_fct, method = "knn",
        trControl = ctrl, metric = "ROC",
        tuneGrid = expand.grid(k = k_grid)
    )
    prob <- predict(fit, X_te_s, type = "prob")
    pred <- predict(fit, X_te_s)
} else if (model_name == "CART") {
    set.seed(42)
    fit <- train(
        x = X_tr, y = y_train_fct, method = "rpart",
        trControl = ctrl, metric = "ROC",
        tuneGrid = expand.grid(cp = c(1e-5, 5e-5, 1e-4, 5e-4, 1e-3))
    )
    prob <- predict(fit, X_te, type = "prob")
    pred <- predict(fit, X_te)
} else if (model_name == "XGBoost") {
    set.seed(42)
    fit <- train(
        x = X_tr, y = y_train_fct, method = "xgbTree",
        trControl = ctrl, metric = "ROC", verbosity = 0,
        tuneGrid = expand.grid(
            nrounds = c(100, 300), max_depth = c(4, 6),
            eta = c(0.05, 0.1), gamma = 0,
            colsample_bytree = 0.8,
            min_child_weight = 1, subsample = 0.8
        )
    )
    prob <- predict(fit, X_te, type = "prob")

```

```

    pred <- predict(fit, X_te)
  }

  cm <- confusionMatrix(pred, y_test_fct, positive = "Win")
  rc <- roc(y_test_fct, prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
  tibble(
    Model      = model_name,
    Features   = n_feat,
    Test_AUC   = as.numeric(auc(rc)),
    Test_Accuracy = as.numeric(cm$overall["Accuracy"]) * 100,
    CV_AUC     = max(fit$results$ROC)
  )
}

grid_subsets <- list(
  Full      = colnames(X_train),
  Forward   = fwd_selected,
  LASSO     = lasso_selected$feature,
  ElasticNet = enet_selected
)

grid_models <- c("LR", "NB", "KNN", "CART", "RF", "XGBoost")

grid_results <- map_dfr(names(grid_subsets), function(s) {
  map_dfr(grid_models, function(m) {
    fit_model_subset(m, grid_subsets[[s]]) %>% mutate(Subset = s)
  })
})

grid_results <- grid_results %>%
  mutate(
    Model = factor(Model, levels = grid_models),
    Subset = factor(Subset, levels = names(grid_subsets))
  )

cat(sprintf(
  "Grid (%d cells) trained in %.1fs.\n",
  nrow(grid_results), as.numeric(Sys.time() - t0, units = "secs")
))

```

Grid (24 cells) trained in 65.7s.

```

grid_table <- grid_results %>%
  select(Model, Subset, Test_AUC) %>%
  pivot_wider(names_from = Subset, values_from = Test_AUC)

knitr::kable(grid_table,
  digits = 3,
  caption = "Test AUC across the 6 (model) x 4 (selection) grid"
)

```

Table 13: Test AUC across the 6 (model) x 4 (selection) grid

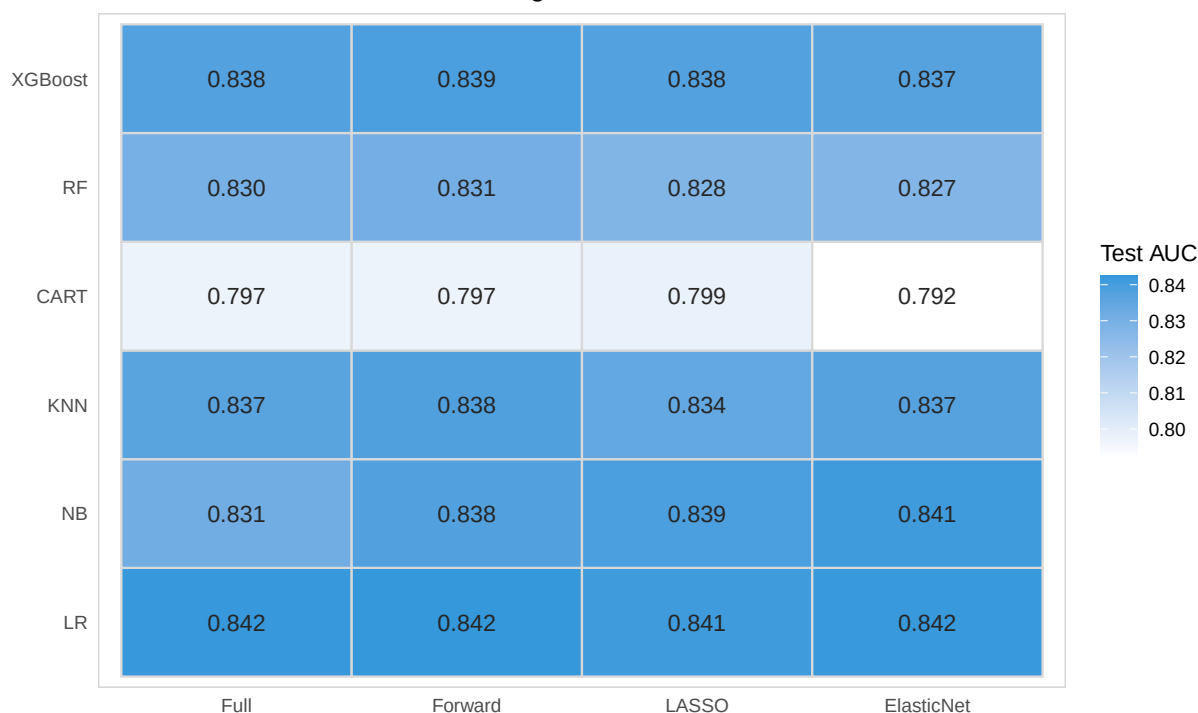
Model	Full	Forward	LASSO	ElasticNet
LR	0.842	0.842	0.841	0.842
NB	0.831	0.838	0.839	0.841
KNN	0.837	0.838	0.834	0.837
CART	0.797	0.797	0.799	0.792
RF	0.830	0.831	0.828	0.827
XGBoost	0.838	0.839	0.838	0.837

```

ggplot(grid_results, aes(x = Subset, y = Model, fill = Test_AUC)) +
  geom_tile(color = "grey85", linewidth = 0.5) +
  geom_text(aes(label = sprintf("%.3f", Test_AUC)), color = "grey15", size = 4) +
  scale_fill_gradient(
    low = "white", high = "#3498db",
    limits = c(min(grid_results$Test_AUC), max(grid_results$Test_AUC))
  ) +
  labs(
    title = "Test AUC across model x selection grid",
    x = NULL, y = NULL, fill = "Test AUC"
  ) +
  theme_minimal() +
  theme(
    panel.grid = element_blank(),
    panel.border = element_rect(color = "grey85", fill = NA)
  )

```

Test AUC across model x selection grid



The total AUC range across the 24 cells is small relative to 5-fold CV noise, and most of it is CART under-performing the rest of the field rather than a selection-method effect. Across the four selection columns LR, KNN, RF, and XGBoost shift by at most 0.004 AUC, so for the strong learners feature reduction is essentially a parsimony decision rather than a performance one.

NB is the only model where reduction clearly helps, which is exactly what the S1 multicollinearity diagnosis predicted: dropping redundant predictors makes its likelihood factorisation closer to truth. Every other model is flat (LR, KNN, XGBoost) or marginally negative (RF, CART) - trees can use overlapping features at different splits, so reduction is mildly costly rather than helpful. The empirical answer to S1's collinearity concern: feature selection here buys parsimony and interpretability, not higher held-out AUC.

Conclusions

H1 findings

H1 is confirmed. Every classifier trained on the 14 early-game features beats the 72.9% naive gold-rule baseline, and five of six exceed 83% AUC-ROC on the held-out test set. The test-set AUC values are 84.2% (LR), 83.0% (RF), 83.1% (NB), 83.7% (KNN), 79.7% (CART), and 83.8% (XGBoost). The spread between the top five is roughly 1 percentage point, meaning the

feature set matters more than the model family - once you use a competent learner, the data determines the ceiling. CART trails the rest by ~3 pp, reflecting the limits of a single tree at this prediction ceiling.

Logistic Regression is the strongest single performer, edging out XGBoost and KNN by 0.4-0.5 pp AUC. This is the result a 14-feature pool with a clear linear signal should produce: when the dominant features (resource differentials at @15) carry a near-linear relationship with the log-odds of winning, a parametric linear model captures that signal as cleanly as a tree ensemble can. The cross-model feature importance consensus identifies `golddiffat15`, `xpdiffat15`, and `csdiffat15` as universally important - the team that is ahead in gold, XP, and CS at 15 minutes wins with high probability. Binary objective flags (first dragon, first herald, first tower) add secondary, independent signal, but rank below the continuous resource metrics in importance.

These results sit in the published 76-78% accuracy band for 15-minute predictions on LoL professional data; our 83-85% AUC reflects the discriminative power of probability calibration over raw accuracy plus a broader feature set (14 vs. the 5-8 used in most prior work). Even in the 2025 season, where the reported predictive coefficient of `golddiffat15` has declined, the early-game snapshot still yields a clear 10 pp AUC gain over the naive baseline.

H2 findings

H2 requires qualification. The EDA gives clear preliminary support: bot lane (ADC) has the highest Pearson correlation with match outcome among all five roles ($r = 0.374$), ahead of jungle (0.346), mid (0.338), support (0.316), and top (0.294). The inter-role gaps are narrow, so the full H2 test decides whether the bot lane edge is real or within noise.

The decisive test is the enriched Random Forest comparison after VIF cleanup: role-level granularity shifts RF AUC by only +0.5 pp over H1 (83.0% $\rightarrow$ 83.5%), inside the 5-fold CV noise band, so team-level totals already capture most of what the role split adds. The bot-lane ordering is the firmer half of the test: the EDA's role-correlation chart gave bot lane the highest single-role correlation with `blue_win`, and the enriched RF ranks `bot_golddiffat15` 4th overall and first among per-role features - well above the best surviving feature of every other role. H2 therefore holds as an empirical ordering (bot lane leads), but not as a strong claim that bot lane alone drives prediction beyond team-level totals.

S1 vs S3 synthesis

S1 and S3 converge from two directions. S1 establishes that the 14 early-game features are collectively sufficient (83-84% AUC across five of six model families, CART trailing at 79.7%) with the signal concentrated in a compact core of continuous resource differentials at 15 min. S3 confirms and sharpens this: three independent selection methods - one algorithmic (forward stepwise by AIC) and two embedded (LASSO, Elastic Net) - all retain the same compact

consensus core, while differing only in how aggressively they prune the marginal band beyond it.

The consensus set matches S1's five-model importance consensus exactly: `golddiffat15`, `xpdiffat15`, `csdiffat15`, `winrate_diff`, `firsttower`, `firstherald`, `firstdragon`. The one disagreement of substance is `kill_diff_15`: it ranks reasonably high in unpenalised LR coefficients and tree-based importance scores, but pure LASSO ($\lambda = 1.0$) zeroes it out as its signal is largely absorbed by `golddiffat15` once L1 pressure is applied; Elastic Net ($\lambda = 0.90$) keeps it. The downstream AUC difference between the two LR fits places the feature at the very margin of usefulness.

Recommendation: train on the consensus core. It is the subset on which all three independent criteria agree, and the validation grid shows that the AUC cost relative to the full 14-feature model is small enough to be inside CV noise. Features kept only by forward stepwise but dropped by both embedded methods - the marginal-contribution band identified in the S3 findings - sit below the L1 threshold and can be dropped without measurable loss. First blood is the clearest null result: despite its prominence in esports commentary, securing the first kill carries no independent predictive value once the 15-minute economic and objective picture is in.

Closing summary

The overall gain over the trivial baseline is modest by design. The naive “team ahead in gold at 15 wins” rule already predicts 72.9% of matches; the best model reaches 75.5% accuracy (84.2% AUC), a real but small improvement. This is not a modelling failure but the practical ceiling of the problem. LoL is a 30-minute + , five-player team game in which many complex mechanics combined carry weight that no early-game snapshot can capture. The 75-78% accuracy band reported in prior work on the same task corresponds to the same ceiling, and our results fall within it.

The recommended deliverable is the seven-feature consensus model. It loses 0.3 pp AUC relative to the full 14-feature pool, trains in seconds on commodity hardware, and produces coefficients that a coach or analyst can interpret directly: gold, XP, CS, prior win-rate difference, and the three opening objectives (first tower, first herald, first dragon). The Shiny application implements both the full and reduced models so the user can verify the trade-off interactively, and the parameter sliders make the marginal contribution of each feature transparent at prediction time.

Export models for Shiny app

```

dir.create("models", showWarnings = FALSE)

saveRDS(lr_model, "./models/lr_model.rds")
saveRDS(rf_model, "./models/rf_model.rds")
saveRDS(nb_model, "./models/nb_model.rds")
saveRDS(knn_model, "./models/knn_model.rds")
saveRDS(cart_model, "./models/cart_model.rds")
saveRDS(xgb_model, "./models/xgb_model.rds")
saveRDS(preproc, "./models/preproc.rds")
saveRDS(perf_summary, "./models/perf_summary.rds")
saveRDS(overlap_df, "./models/overlap_df.rds")
saveRDS(games_model, "./models/games_model.rds")
saveRDS(X_train, "./models/X_train.rds")
saveRDS(games, "./models/games.rds")

# Game lookup table for Shiny browser.
# The Shiny app's game-picker is per-side (one Blue and one Red row per game),
# so we un-pivot games_model back into a two-rows-per-game shape. Signed diffs
# are negated for the Red row, per-team kill counts are swapped, and binary
# flags are toggled (when Blue secured the objective, Red did not; otherwise
# we keep the same value, since neither team is recorded as having secured it).
shiny_blue <- games_model %>%
  mutate(side_encoded = 1L, result = blue_win) %>%
  select(-blue_win)

shiny_red <- games_model %>%
  transmute(
    gameid,
    side_encoded = 0L,
    result = 1L - blue_win,
    golddiffat15 = -golddiffat15,
    xpdiffat15 = -xpdiffat15,
    csdiffat15 = -csdiffat15,
    kill_diff_15 = -kill_diff_15,
    assist_diff_15 = -assist_diff_15,
    gold_efficiency_15 = NA_real_, # Blue-only ratio, no Red counterpart kept
    kda_15 = NA_real_,
    plate_diff = -plate_diff,
    grub_diff = -grub_diff,
    winrate_diff = -winrate_diff,
    firstblood = 1L - firstblood,

```

```

        firstdragon = 1L - firstdragon,
        firsttherald = 1L - firsttherald,
        firsttower = 1L - firsttower
    )

per_side_features <- bind_rows(shiny_blue, shiny_red)

# Per-side team labels
team_labels <- teams %>%
  select(gameid, league, date, side, teamname) %>%
  mutate(
    side_encoded = ifelse(side == "Blue", 1L, 0L),
    side_label = side
  ) %>%
  select(gameid, league, date, teamname, side_label, side_encoded)

opp_labels <- team_labels %>%
  transmute(gameid,
    side_encoded_opp = 1L - side_encoded,
    opp_teamname = teamname
  )

game_lookup <- team_labels %>%
  left_join(opp_labels, by = c("gameid", "side_encoded" = "side_encoded_opp")) %>%
  inner_join(per_side_features, by = c("gameid", "side_encoded")) %>%
  mutate(
    date = as.Date(date),
    result_label = ifelse(result == 1, "WIN", "LOSS")
  )

saveRDS(game_lookup, "./models/game_lookup.rds")

# ROC objects for Shiny Model Comparison tab
roc_lr <- pROC::roc(y_test_fct, lr_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_rf <- pROC::roc(y_test_fct, rf_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_nb <- pROC::roc(y_test_fct, nb_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_knn <- pROC::roc(y_test_fct, knn_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_cart <- pROC::roc(y_test_fct, cart_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
roc_xgb <- pROC::roc(y_test_fct, xgb_prob[, "Win"], levels = c("Loss", "Win"), quiet = TRUE)
saveRDS(
  list(lr = roc_lr, rf = roc_rf, nb = roc_nb, knn = roc_knn, cart = roc_cart, xgb = roc_xgb),
  "./models/roc_list.rds"
)

```



```

)

# LR test predictions for Shiny (confusion matrix + prob distribution)
saveRDS(
  data.frame(actual = as.character(y_test_fct), prob_win = lr_prob[, "Win"]),
  "./models/lr_test_df.rds"
)
saveRDS(lr_cm, "./models/lr_cm.rds")

cat("Models saved to current directory.\n")

```

Models saved to current directory.

References

- [1] **Oracle's Elixir.** *2025 LoL esports match data*. Public dataset. URL: <https://oracleselixir.com/tools/downloads> (accessed 2026-04).
- [2] **Spaargaren, R. H.** (2022). *Early Game Win Prediction in League of Legends*. MSc thesis, Tilburg University. URL: <http://arno.uvt.nl/show.cgi?fid=161690>.
- [3] **Lafrance, A., & Grewal, R.** (2026). Determining the effects of League of Legends ranked tiers on outcome prediction models. *Entertainment Computing*, **57**, article 101088. DOI: <https://doi.org/10.1016/j.entcom.2026.101088>.
- [4] **Junior, J. B. S., & Campelo, C. E. C.** (2023). *League of Legends: Real-Time Result Prediction*. arXiv preprint arXiv:2309.02449. URL: <https://arxiv.org/abs/2309.02449>.
- [5] **Tsang, W.** (2025). *What's actually different about League of Legends in 2025?* Medium article. (Non peer-reviewed analyst post, cited for the temporal-trend observation only.)
- [6] **Wickham, H.** (2014). Tidy Data. *Journal of Statistical Software*, **59**(10), 1-23. DOI: <https://doi.org/10.18637/jss.v059.i10>.